

Autonomous Robotic Fishing Line Tension Regulation Using Admittance Control

Nathan Ge
Dept. of MAE

UCLA
Los Angeles, CA, USA

Chaoyi Hu
Dept. of MAE

UCLA
Los Angeles, CA, USA

Julie Nguyen
Dept. of MAE

UCLA
Los Angeles, CA, USA

Xiang Zhou
Dept. of MAE

UCLA
Los Angeles, CA, USA

Abstract—This paper presents an autonomous robotic fishing system focused on post hook-set tension regulation as a force control problem. After a fish is hooked, maintaining line tension requires balancing overpulling, which risks snapping the line, against underpulling, which can lose the fish. Our platform is a one degree-of-freedom robotic rod that measures fishing line tension with a load sensor and uses an admittance controller to convert tension feedback into rod pitch motion. The control architecture combines tension-based regulation with position compensation and an engagement supervisor that manages transitions between idle, active control, and recovery states. We implement the system in a ROS2 software stack and validate it first in MuJoCo simulation with a repeatable fish-like disturbance, then on hardware with manual line tension trials. Simulation and hardware experiments show that the robot responds intelligently to changing line tension rather than applying fixed motor commands. This work demonstrates a practical foundation for sensor-driven tension regulation in robotic fishing and motivates future extensions to multi-DOF systems, improved state switching, and more complete autonomous fishing pipelines.

Index Terms—tension regulation, admittance control, force control, robotic fishing, ROS2

I. OVERVIEW

This project explores the potential of autonomous robotic fishing framework. To carry the problem into an engineering lens to build a technical framework around, we centered our robot’s function on the fishing line tension aspect of the fishing pipeline which is modelled as a force control problem. Tension regulation is fundamentally characterized by the balance of overpulling which risks the line snapping and underpulling which can lose the fish. Good anglers will respond by dynamically adjusting the orientation of the rod with intuition and feel, keeping the line tension in a safe zone. The robot will perform this balancing act intelligently with sensor feedback and precise motor control.

Similar robotic fishing systems have been explored in previous projects. AlYaseen et al. developed an autonomous fishing rod that can detect fish bites, pull the line, and stop the motor when high resistance is detected to avoid cutting the line [1]. This is closely related to our project because both systems use the fishing-line force as an important feedback signal. Another robotic fishing pole project used an Arduino-controlled mechanical system to cast and reel automatically through smartphone or computer control [2]. These examples

show that fishing is a suitable application for automation and robotic actuation.

Fishing has also been studied as a force-feedback and haptic simulation problem. The Stanford CHARM Lab developed a haptic fishing robot/simulator to create realistic force feedback for the user [3]. This supports the motivation of our project because the key part of the fishing experience is not only moving the rod, but also reacting to the changing resistance from the fish. Our project extends this idea by using a physical robotic rod and an admittance controller to convert measured tension into rod motion.

The resultant robot platform is a 1 DOF robotic fishing system to control line tension following the hook set fishing phase. The robot employs sensor-based feedback control to manage tension regulation and disturbance rejection. The tension in the fishing line is detected by a load sensor, and the robot controls the rod pitch angle based on the error of the detected tension. The control system logic was modeled and solidified in simulation as a tethered robot in which the fish-like load is an external disturbance via the line. The project also creates a foundation for future work on more realistic multi-DOF fishing robots or autonomous fishing mechanisms.

II. CONTROL OBJECTIVES

To carry the problem into an engineering lens to build a technical framework around, fishing can be characterized by three discrete modes: fish discovery (pre hook-set), tension regulation (post hook-set), and reeling.

Fish discovery describes the phase in which the fishing line is ‘cold’ and waiting for a fish to bite. The controller looks for a disturbance signal to indicate fish interest. A jigging action that mimics live bait may be used in this phase to increase fish hooking probability. This phase also includes holes/gaps in tension sensing caused by a fish already hooked on that escapes the jurisdiction of the control domain. Algorithms/action routines may be employed to restore control over the fish.

Looking past fish discovery is the tension regulation stage, the project’s primary control objective, which centers on maintaining tension through the fishing line. The general control structure involves a physical load sensor that approximates the force through the line, and control algorithms that take this information to modulate the tension estimation data. Tension

accommodation lends itself naturally to admittance control or some sort of force regulation control methodology, both of which became primary control methods of exploration for this project. Key concepts/considerations include: Tension regulation, Virtual mass-spring-damper (MSD) admittance loops, and Highly stochastic loads handling.

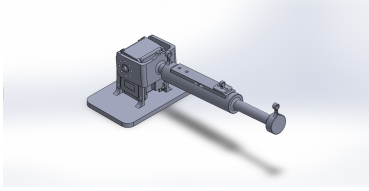
An additional reeling component may be added to successfully bring the fish to the desired endpoint. Reeling will also require additional hardware to pull the string inward. This would complicate the load sensing packaging and add mechanical complexity that comes with a reeling system. This particular system was not heavily pursued in our research, leaving it an avenue of future development.

III. METHODS

A. Hardware



(a) Physical robot.



(b) CAD of robot.

Fig. 1: Robot hardware and corresponding CAD assembly.

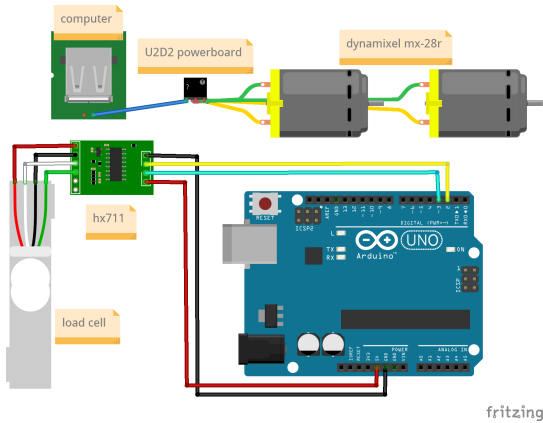


Fig. 2: Wiring diagram.

B. Mathematical Formulation

1) *System Definitions*: Let the 2-DOF robotic fishing platform be modeled as a spherical manipulator. We define the state variables and physical parameters as follows:

- q_1 : The yaw angle of the base around the global Z-axis.
- q_2 : The pitch angle of the rod relative to the horizontal XY plane.
- L : The fixed length of the fishing rod from the pitch joint to the tip.
- h : The vertical height offset of the pitch joint from the ground.

2) *Forward Kinematics*: The forward kinematics map the joint space (q_1, q_2) to the Cartesian task space (x, y, z) of the rod tip. By projecting the rod length onto the horizontal plane, we obtain the radial distance $r = L \cos(q_2)$. Decomposing this radial distance and accounting for the base height yields the position equations:

$$\begin{aligned} x &= L \cos(q_1) \cos(q_2) \\ y &= L \sin(q_1) \cos(q_2) \\ z &= h + L \sin(q_2) \end{aligned}$$

3) *Inverse Kinematics*: The inverse kinematics determine the required joint angles (q_1, q_2) to reach a desired target coordinate (x, y, z) .

The yaw angle q_1 is isolated by taking the ratio of the y and x position equations. To ensure the correct quadrant is selected, the two-argument arctangent function is utilized:

$$q_1 = \text{atan2}(y, x)$$

The pitch angle q_2 is found by relating the vertical displacement of the rod tip to the horizontal radial distance from the base, where $r = \sqrt{x^2 + y^2}$:

$$q_2 = \text{atan2}(z - h, \sqrt{x^2 + y^2})$$

4) *Analytical Jacobian*: The analytical Jacobian matrix, $\mathbf{J}$, is required for force control and admittance loops. It relates joint velocities to end-effector velocities and maps external Cartesian forces back to joint torques via the relationship $\boldsymbol{\tau} = \mathbf{J}^T \mathbf{F}_{ext}$.

The matrix is derived by computing the partial derivatives of the forward kinematic position equations with respect to the joint variables q_1 and q_2 :

$$\mathbf{J} = \begin{bmatrix} \frac{\partial x}{\partial q_1} & \frac{\partial x}{\partial q_2} \\ \frac{\partial y}{\partial q_1} & \frac{\partial y}{\partial q_2} \\ \frac{\partial z}{\partial q_1} & \frac{\partial z}{\partial q_2} \end{bmatrix} = \begin{bmatrix} -L \sin(q_1) \cos(q_2) & -L \cos(q_1) \sin(q_2) \\ L \cos(q_1) \cos(q_2) & -L \sin(q_1) \sin(q_2) \\ 0 & L \cos(q_2) \end{bmatrix}$$

C. Codebase and Simulation

Additional ROS2 code documentation can be found here: [fishing bot code documentation](#)

Building out the ROS2 codebase such that the control logic interacts seamlessly with MuJoCo simulation and hardware control

TABLE I: Table: Package Descriptions

Package	Description
Bringup	Launch configuration. Can launch simulation or hardware control based on preference.
Description	Mesh and model descriptors.
Control	Tension control package.
Planning	Used to control randomness of 'fish' behavior. Can be thought of as a fish agent (our disturbance).
Sensors	Sensor readings and publishing.
Interfaces	Develop protocol for inter-node communication.

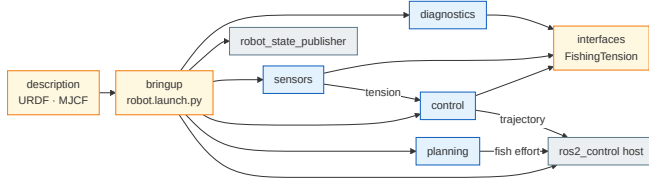


Fig. 3: Package Architecture

The following diagram shows the interaction of the self-contained ROS2 packages and their contributions to the control routine.

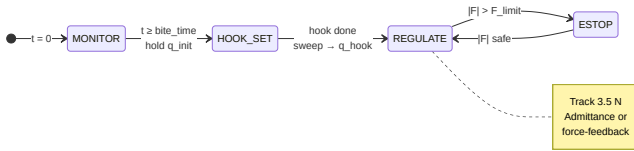


Fig. 4: Fishing State Machine

The above diagram shows the transition states enforced by our controller.



Fig. 5: Control Loop

The above diagram shows the generalized control scheme specific to our ROS2 implementation.

Simulation Pipeline

The behavior of the fish is simulated as a repeatable disturbance for controlled testing. The controllers were tested in a repeatable way with a ROS2 control structure and MuJoCo simulation.



Fig. 6: Simulation Data Flow

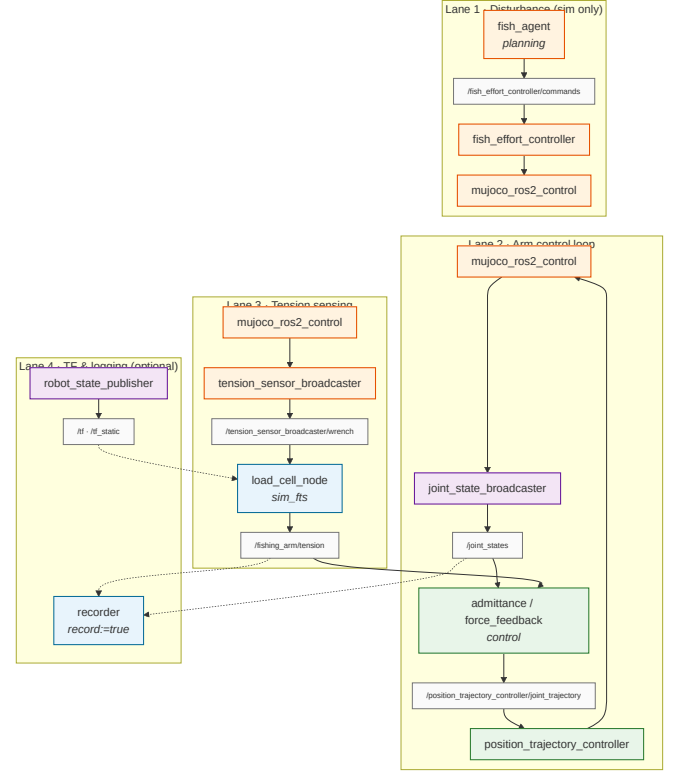


Fig. 7: Software ROS2 graph

The hardware control flow path slightly varies from the simulation pipeline.



Fig. 8: Data Flow

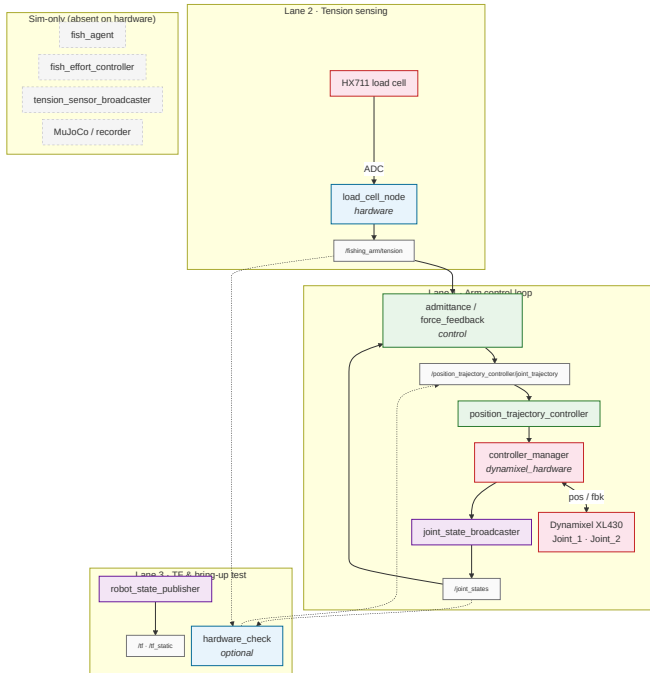


Fig. 9: Hardware ROS2 Graph

D. Control Architecture

Proportional Force Controller

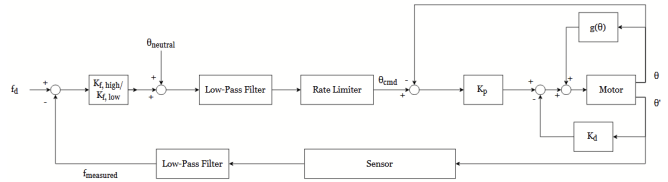


Fig. 10: Proportional Force Control diagram.

The proportional force controller directly regulates the tension of the line by using the tension error measured by the load cell multiplied by an asymmetric gain to brute-force a position command. An inner PD control is then used to track the position of the motor, and a gravity compensation term is injected into the motor command. The system measures the tension error as a difference between the desired and measured tensions

$$e_f = f_d - f_{\text{measured}}$$

An asymmetric, state-dependent gain is then used to map the force error to a corresponding position offset. If the tension error is positive and tension is too high, it uses K_f , high, and if the tension is too low, K_f , low is used. K_f , high $<$ K_f , low. This is such that the rod will ‘pull’ harder when the tension is low to pick up the slack of the line. Since the fish is hypothetically faster than the motor, the motor has to aggressively compensate for the lack of tension. However, when the tension is too high, the gain is lower to prevent the robot from slamming forwards to yield to the fish, which would cause the line to immediately go slack. The gains are tuned so that there is a balance between the rod being too stiff and unyielding (low K-value) or too soft and pliant, or over-reactive (high K-value).

The inner loop of the controller is a Proportional-Derivative controller that calculates the effort required to move the motor to the commanded position.

$$\tau = K_p(\theta_{cmd} - \theta) - K_d\theta' + g(\theta)$$

In this controller $K_p(\theta_{cmd} - \theta)$ acts as a spring to move the motor towards the command position, K_d dictates how aggressively the controller reacts to the rate of change and acts as a brake, and the gravity compensation term, $g(\theta)$, is used to counteract the downward pull of gravity. Additionally, low-pass filters and rate limiters are used to prevent high spikes in current and to reject extraneous noise or high-frequency actions.

IV. RESULTS

A. Error Assessment

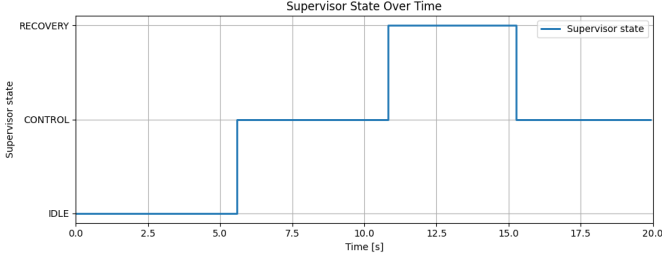


Fig. 14: Engagement and recovery supervisor state during the hardware trial.

1) *Engagement and safety logic:* The engagement supervisor was also evaluated during the hardware trial. Before the line tension reached the bite-trigger condition, the controller remained in the IDLE state and commanded zero PWM, preventing the motor from responding to sensor noise or small disturbances. In this implementation, the controller was activated only when the measured tension exceeded 0.8 N for 0.10 s, representing the initial pulling event after a fish is hooked. During active control, a low-tension condition was detected when the measured tension stayed below 1.0 N. If this condition lasted for more than 1.8 s, the supervisor entered the RECOVERY state to avoid continuously pulling against a slack line. Once the measured tension recovered above 2.0 N, the controller returned to active control. The supervisor state plot shows this sequence, with transitions from IDLE to CONTROL, then to RECOVERY, and finally back to CONTROL after the tension recovered.

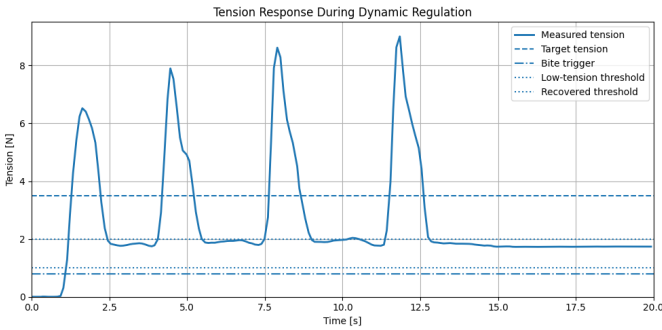


Fig. 15: Measured tension response during dynamic regulation.

2) *Dynamic Controller Response:* The dynamic controller response was evaluated using a trial with larger variations in the applied line tension. Fig. 8 shows the measured tension signal used as feedback by the controller. Since the line was pulled manually, the sharp peaks in the tension curve should not be interpreted as a standardized disturbance input. The tension signal also depends on factors such as pulling speed, pulling direction, line slack, and the contact condition of the load cell. Therefore, this figure is mainly used to show the feedback signal received by the controller during a more

dynamic pulling condition, rather than to evaluate the tension curve alone.

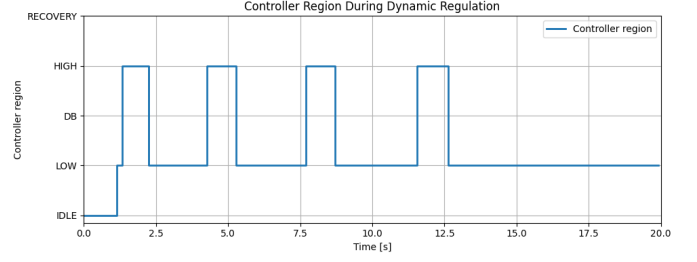


Fig. 16: Controller region switching during dynamic regulation.

The controller response to these tension changes is shown more clearly in Fig. 9. During the trial, the controller compares the measured tension with the target tension and the dead-band, then selects the corresponding control region. When the tension rises above the desired range, the controller switches to the HIGH region, which corresponds to a release action to reduce excessive line tension. When the tension drops below the desired range, the controller returns to the LOW region, where the motor pulls back to recover line tension. The repeated LOW/HIGH transitions show that the controller is actively responding to the sign of the tension error rather than applying a fixed motor command.

The LOW and HIGH regions should be interpreted as two operating regions of the same tension-regulation controller, rather than two separate controllers for reeling in and letting out the line. When the measured tension is above the desired range, the controller enters the HIGH region and commands a release action to reduce excessive line tension. When the measured tension is below the desired range, the controller enters the LOW region and commands a pull-back action to rebuild tension. In this sense, the controller is switching its response direction based on the sign of the tension error, while still using one overall feedback-control structure.

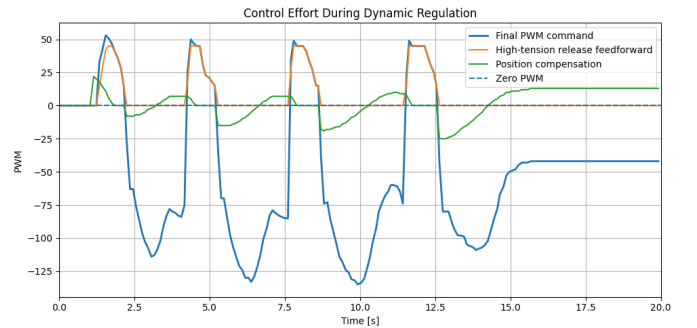


Fig. 17: PWM command and control components during dynamic regulation.

Fig. 10 shows how these feedback decisions are converted into actuator commands. The blue curve is the final PWM command sent to the Dynamixel motor, while the orange and

green curves show two major contributing terms: the high-tension release feedforward and the position compensation term. The zero-PWM line separates the two actuation directions. In this implementation, positive PWM commands are associated with release actions during high-tension events, while negative PWM commands are associated with pull-back corrections when the tension becomes too low.

The positive PWM pulses in Fig. 10 appears when the controller enters the HIGH region, showing that the controller is actively trying to release the rod and reduce excessive tension. After the tension decreases, the final PWM command becomes negative, which corresponds to the LOW-region pull-back action. The position compensation term changes more slowly because it depends mainly on the rod position rather than the instantaneous tension peak. As a result, the final PWM command is the combined output of tension-based correction and position-based compensation.

The PWM command in this hardware trial appears relatively step-like because the prototype was implemented using PWM mode on the Dynamixel motor, rather than a calibrated torque or current-control mode. In addition, the controller uses a deadband-based switching logic, so the control effort changes more noticeably when the measured tension moves between the LOW, deadband, and HIGH regions. Therefore, the PWM signal should not be interpreted as an ideal continuous torque command. Instead, it is used here to show the direction and relative strength of the motor response produced by the tension-regulation logic.

The asymmetry between the positive and negative PWM commands is also intentional. During release, the goal is to reduce excessive tension without moving the rod forward too aggressively. A very aggressive release could either drive the rod close to the forward limit or reduce the line tension too quickly and create slack. For this reason, the release command was kept relatively conservative. During pull-back, however, the motor needs more authority to rebuild line tension while overcoming gravity, friction, and mechanical resistance in the hardware setup. This explains why the negative pull-back commands are larger in magnitude than the positive release commands.

Overall, the agreement between Fig. 8, Fig. 9, and Fig. 10 shows the intended closed-loop behavior of the controller. Fig. 8 provides the measured tension feedback, Fig. 9 shows the corresponding HIGH/LOW region selection based on the tension error, and Fig. 10 shows the motor command generated from this feedback logic. Together, these results indicate that the robot actively responds to changes in line tension by switching control modes and commanding the actuator in the expected direction.

For an actual fishing application, large force oscillations would not be desirable because they could increase the risk of excessive line loading or temporary loss of line tension. In this project, the rapid-pull trial was mainly used as a stress test to check whether the controller could detect sudden tension changes and respond in the correct direction. The results show the expected switching behavior, but they also suggest that

smoother mode transitions and improved inner-loop actuation would be useful for reducing oscillations in future versions of the system.

3) *Post-Transient Regulation Performance:* The post-transient regulation trial was used to evaluate the controller under a more sustained pulling condition after the initial engagement. In this test, an initial manual pull was applied to create a large tension deviation, and the line was then held as steadily as possible. Although the input was still applied by hand and should not be treated as a standardized step input, this trial better represents the controller behavior after the large initial disturbance has passed. It was therefore used to evaluate whether the system could bring the measured tension back near the 3.5 N target and maintain it in the later part of the trial.

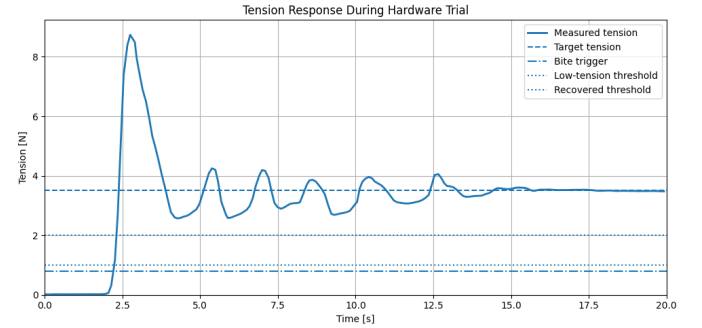


Fig. 18: Measured tension response after the initial engagement transient.

Fig. 11 shows the measured tension response during this trial. The target tension was set to 3.5 N. During the initial engagement phase, the measured tension contains a large transient peak, which is mainly caused by the initial line engagement and manual pulling input. After this transient and the following correction motions, the controller brings the measured tension back toward the target range. In the later part of the trial, the tension remains close to the desired value, showing that the controller can maintain the line tension near the target under a relatively stable pulling condition.

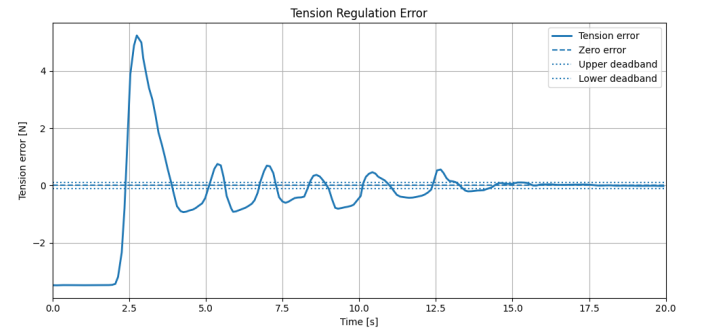


Fig. 19: Tension regulation error after the initial transient.

The tension regulation error is shown in Fig. 12. Instead of treating the full response as a standard settling-time test, the final portion of the trial was used as a post-transient

evaluation window. Over this window, the measured tension had an average value of 3.52 N compared with the 3.5 N target. The mean absolute tension error was 0.025 N, and the RMSE was 0.036 N, corresponding to approximately 0.72% and 1.03% of the target tension, respectively. These values show that once the large transient response had passed, the controller was able to regulate the line tension with a small remaining error.

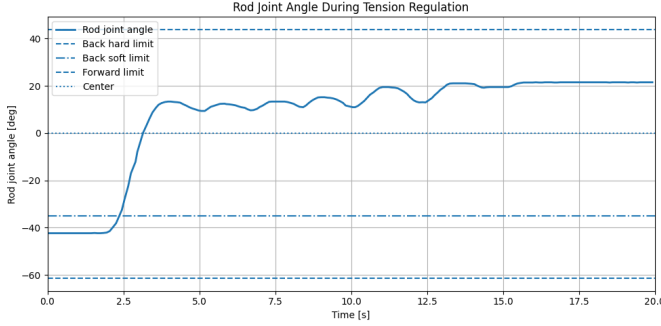


Fig. 20: Rod joint angle during post-transient tension regulation.

Fig. 13 shows the rod joint angle during the same trial, reported in degrees. The joint angle provides additional context for the tension regulation behavior shown in Fig. 11 and Fig. 12. During the initial engagement and correction phase, the tension error is relatively large, and the actuator changes the rod angle to establish line tension and reduce the error. As the measured tension approaches the target in the later part of the trial, the tension error becomes smaller and the rod joint angle also remains near a stable operating posture. This trend indicates that the actuator adjustment is consistent with the reduction of the tension error. However, the joint angle is not expected to match the tension error exactly, since the measured tension also depends on the manual pulling input, line slack, load-cell contact, and friction. Therefore, Fig. 3.3 is used to show the actuator motion and final rod posture, while Fig. 12 remains the main performance metric for tension regulation.

V. CURRENT CHALLENGES

Although the hardware test shows that the controller can regulate the line tension, the current setup still has several practical limitations. One major limitation is that the external line tension was applied by hand during the experiment rather than generated by a standardized disturbance source. This made the test closer to an actual fishing interaction, but it also means that the measured tension curve is affected by the pulling speed, pulling direction, and line slack. As a result, some sharp peaks and sudden drops in the tension signal should be interpreted as part of the manual interaction with the line, instead of being attributed to the controller alone. Therefore, the hardware results are mainly used to demonstrate the controller behavior under realistic manual interaction rather than to provide a standardized disturbance-response benchmark.

Another important limitation comes from the indirect relationship between motor motion and measured line tension. Unlike a pure position-control task, where the motor angle can be measured and controlled directly, the line tension depends on several physical factors between the actuator and the sensor. For example, line slack, fishing-line compliance, load-cell contact condition, and friction in the mechanism can all affect the measured tension even when the motor command is the same. As a result, small motor motions do not always produce an immediate or proportional change in tension.

This limitation is also related to the PWM-based actuation used in the current implementation. Since the motor is commanded through PWM rather than a precise torque or position controller, small corrective commands may need to overcome static friction, gravity, and mechanical resistance before visible rod motion occurs. This can make the actuator response less smooth during fine tension corrections. Therefore, the tension regulation result should be understood as the combined behavior of the controller, motor actuation, fishing line, and load-cell measurement, rather than as a direct one-to-one mapping between joint angle and tension.

VI. FUTURE WORK

Upon further understanding of the problem, it became clear that this apparently simple tension management task has incredibly deep and rich control ideas embedded beneath the surface. This led to a deep dive of potential control tools that may bolster the robustness of the control system.

A. Algorithms

First let's consider extensive ideas on the algorithmic front. For one, if we consider taking the 1DOF control problem to the more realistic 2DOF planarized case, nonlinear coupling dynamics and orthogonal subspaces appear. But less obvious is added complication regarding sensing capabilities. Considering our limited 1D load sensing framework, we will require sensor fusion/interpretation techniques to ensure optimal tension estimation in the higher dimensional space (2D). In other words, instead of a direct linear force/tension estimation relation for the 1DOF case, we need to be aware of a more precise tension vector to perform control feedback on. If we use load sensor info and kinematic state via encoders we may employ kinematic variation/dithering to back out the tension vector estimation. But if we decide to use joint torques as well, a sensor fusion technique such as an extended kalman filter may be necessary. Furthermore, the concept of hybrid control may come into play to allow admittance in the tension management force adaptive axis and rigid positional control in perpendicular axes.

Another crucial facet of the control problem is state switching capability. Discrete states are naturally embedded within the fishing control problem, and makes the problem incredibly control rich. The transition from fish searching to hook set, or switching from having fish to losing tension are both examples of discrete switching behavior. The problem can be summarized as this: how do we ensure stable and robust discrete state

switching capability? A robust Finite State Machine (FSM) must be prescribed. The current FSM implementation uses basic force thresholding that induces highly volatile “chatter” behavior, or rapid, unstable, and unnecessary state switching behavior. A few stability insurance techniques come to mind to augment our FSM. We can first employ common FSM techniques, two of which being hysteresis and debouncing. Hysteresis may help better characterize switching conditions by enforcing anisotropic force thresholds for entering and exiting a state. Debouncing adds time-based thresholding/conditioning to limit false/rapid triggering.

Another region of focus in the FSM is the transition points themselves. We could apply smoothing filters to help initiate bumpless transfer and eliminate unwanted transient spikes in control effort. A crucial function of the controller is to also place the system under regions of controllability. In this particular problem, we want to avoid fish line snapping or losing tension control on the fishing being hooked onto the line. We can use tension break detection (rate of force change) and accompanying funnel synthesis to bound the tracking error control behavior. The idea of funnel synthesis can be applied to the immediate high error case where tension is suddenly lost. Funnel synthesis drops the robot controller into a safe, heavily damped recovery ‘funnel’ in case of line break preventing the robotic arm from violently whipping backward and breaking motors. This funnel synthesis concept can also be applied for control regain.

Additionally, the state behaviors themselves have to be tended to. If admittance control is being used we can use gain scheduling to customize dynamic behaviors specific to a state’s concerns.

AI has relevance to control enhancement here as well. We can augment the control loop with an adaptive neural network feedforward term that learns the stochastic frequency of the fish’s pulling behavior, effectively predicting the required q_d before the tension error grows too large. Our preexisting sim-real frameworks can also be set up to build a Sim-to-real training pipeline. We use our simulation framework as an AI training tool to optimize fish tensioning behavior in the real robot. This can be primarily achieved in two ways: either we use direct RL methods to build a neural-based controller, or employ layered RL to tune controller parameters of a pre-designed control framework. Our simulation pipeline can also simply be used for adaptive control tuning without AI.

B. Noise reduction and filtering techniques

To properly target fishes for catching in an unpredictable open ocean scenario, we may require probabilistic state estimation (HMM, Bayesian filtering). This concept is often important in partial observability/limited info scenarios such as real world RL training cases. Frequency and derivative of the force may be used to distinguish between fake disturbances (boat rocking, snag on seaweed) or an actual fish. However, in the case the environment we are operating in (in our scenario it would be the ocean or a body of water) has no partial observability or obstruction concerns, we may simply bypass

the problem state estimation seeks to solve by using computer vision to distinguish between a fish and ignorable entities.

C. Hardware

A few hardware-specific augmentations could be worth studying in the future, to observe their impact on tension management characteristics (transient behaviors, error attenuation, control effort).

The introduction of a redundantly actuated serial joint is one such augmentation with significant control implications. The idea was initially inspired by a desire to emulate rod compliance with active control capability. Then, the larger control picture became clear. The most practical use of the redundant serial joint is the implementation of a macro-micro manipulator structure. The lower heavy base joint handles low-frequency, high-amplitude positioning (like tracking boat’s movement). A smaller, redundantly actuated serial joint near the tip can be tuned with a much higher control loop frequency to handle high-frequency stochastic disturbances. As a result, we receive the benefit of inertial decoupling: we get improved torque efficiency as jerky movements are handled by the actuator near the tip and doesn’t cause the heavy base motor downstream to respond unnecessarily and move high inertial loads.

The added joint also opens the possibility of null space optimization with the introduction of infinite joint configurations. This allows us to initiate secondary optimization objectives besides tension management such as torque minimization.

The secondary joint can also be treated as a dynamic impedance shaping element, with the redundant motor acting as a dynamically tunable physical stiffness unit that adheres to control needs.

Passive structures are another potential angle for improving dynamic performance. A compliant segment added to the end of the rod can mechanical LPF, dealing with incredibly jerky fish motion in true fishing scenarios as well as improve bumpless transfer stabilization during state transitions to reduce risky torque spikes.

The last hardware-based project enhancement has less to do with the robotic arm itself, but more so about creating an improved real-world testing rig. The idea is to create an actuatable object to mimic a fish that has planarized movement capabilities to perform varied experiments on our hardware system and its accompanying controller design

D. Practical additions

Other considerations that would bring the automated fishing system closer to a ‘market-available’ product include an actuated hooking system and a tensioning reel mechanism. This would come with its own host of hardware implementation and control framework problems.

ACKNOWLEDGMENT

The author thanks Professor Veronica Santos for guidance and instruction throughout the MAE 263C Controls of Robotics Systems course. Their insights into soft robotic modeling formed the foundation of this project.

REFERENCES

- [1] M. AlYaseen, O. Alajmi, N. Al-Attal, and A. Al-Saleh, "Autonomous Fishing Rod," College of Engineering and Applied Sciences, American University of Kuwait, Kuwait, 2019.
- [2] J. Cook, "Robotic Fishing Pole Casts and Reels Automatically," Hackster.io, 2019. [Online]. Available: Hackster.io.
- [3] Stanford CHARM Lab, "Haptic Fishing Robot," Stanford University, 2024. [Online]. Available: Stanford CHARM Lab.
- [4] Lai, G., Zhou, S., Yang, W., Wang, X., & Wang, F. (2023). Prescribed fixed-time adaptive neural control for manipulators with uncertain dynamics and actuator failures. *Mathematics*, 11(13), 2925. <https://doi.org/10.3390/math11132925> Cited by: 4

VII. APPENDIX

A. Itemized Breakdown

TABLE II: Itemized breakdown of team member contributions.

Task	Nathan	Julie	Chaoyi	Xiang
CAD / Mechanical Design	90%	10%	0%	0%
ROS2 / Software Architecture	100%	0%	0%	0%
Fabrication / 3D Printing	0%	45%	45%	10%
Sensor / Wiring / Debugging	10%	10%	40%	40%
Controller Design	10%	40%	40%	10%
Simulation / Controller Testing	0%	50%	0%	50%
Hardware Controller Testing	10%	10%	15%	65%
Results / Plotting	10%	10%	10%	70%
Report / Presentation	25%	25%	25%	25%

B. Bill of Materials

TABLE III: Bill of Materials

Item	Component	Qty	Description
1	Arduino Uno R3	1	Sensor reading and hardware communication.
2	HX711	1	Load-cell signal conditioning module.
3	Load Cell	1	Measures fishing-line tension.
4	Dynamixel MX-28R	2	Smart servo motors for rod actuation.
5	U2D2	1	USB-to-Dynamixel communication adapter.
6	12 V DC Power Supply	1	Powers the Dynamixel motors.
7	USB Cable (A–B)	1	Arduino-to-computer connection.
8	Dynamixel TTL Cable	2	Daisy-chain communication between servos.
9	Jumper Wire Set	1 set	Connections between Arduino, HX711, and load cell.
10	Fishing Rod Assembly	1	Custom 1-DOF rod structure.
11	M2 Screws	10	Connect motor to 3D-printed assembly.

C. Custom code

The custom project code is submitted as separate PDF files as part of the Turnitin submission. Only code written specifically for this course project is included. External libraries and built-in package source code, such as MuJoCo, Dynamixel SDK, ROS2, NumPy, and Matplotlib libraries, are not included.

The submitted custom code consists of the following files:

- `MuJoCo_controller.pdf`: custom MuJoCo simulation code for the 1-DOF robotic fishing-rod tension regulation system. This code defines the simulated rod model, fish-like external disturbance, load-cell-style tension feedback, controller states, controller update logic, and visualization/analysis routines used to evaluate the controller response in simulation.
- `project_w_t_controller.pdf`: custom hardware controller code for the final Dynamixel-based tension regulation experiment. This code reads load-cell tension feedback through the Arduino/HX711 interface, filters the measured tension signal, applies the engagement supervisor logic, switches between LOW, deadband, and HIGH tension-control regions, computes the PWM command with tension-based correction and position compensation, sends commands to the Dynamixel motor through the U2D2 interface, and logs experimental data for result plotting.

These code files correspond to the simulation and hardware implementation used in the final report. The MuJoCo code was used for controller testing before hardware deployment, while the hardware controller code was used to collect the experimental results shown in the Results section.

D. Custom CAD designs

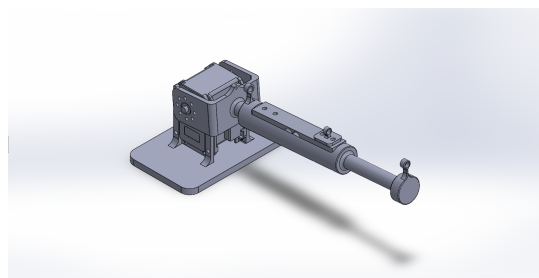


Fig. 21: CAD of robot.

E. Custom systems-level wiring diagrams

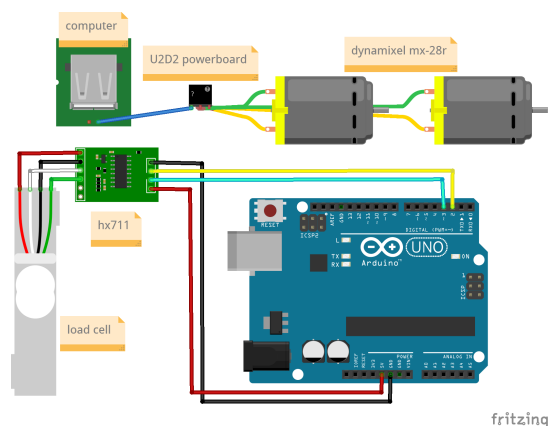


Fig. 22: Wiring diagram.

UCLA

Samueli
School of Engineering

GoFish!: A Tension Regulation Fishing Robot

TEAM 17

Nathan Ge, Chaoyi Hu, Julie Nguyen, Xiang Zhou

Overview

Project Overview & Motivation

- **Project Goal**

- Develop a 1-DOF rigid robotic fishing rod
- Simulate fish-fighting interaction under controlled testing
- Develop a control paradigm to regulate fishing-line tension near a 3.5 N target with a ± 0.1 N deadband.

- **Motivation / Literature**

- Prior fishing haptic devices show that force feedback improves realism in fishing simulation.
- Fishing-rod haptic displays have also been studied for presenting force sensation through a rod-and-line structure.

- **Applications**

- Fishing training simulator
- Entertainment / interactive game controller
- Robotics force-feedback education

- **System Concept**

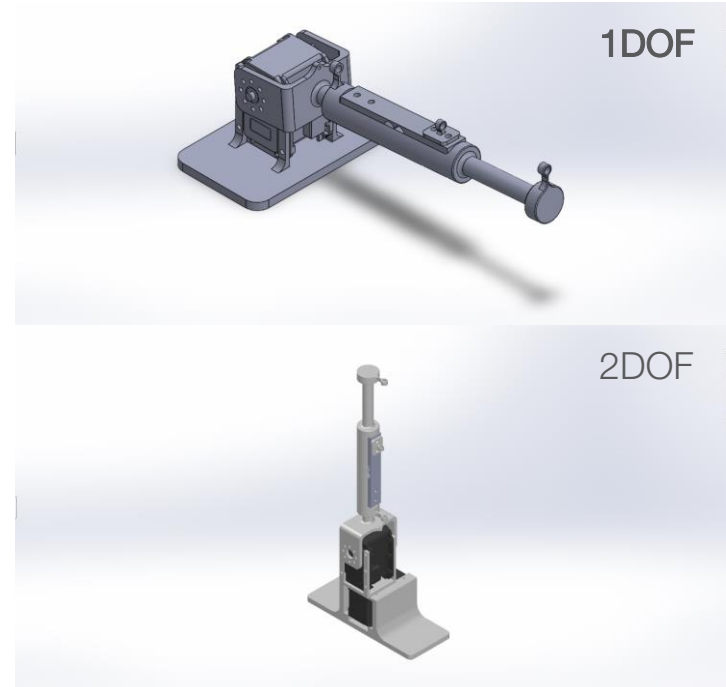
$$F_{pull} \rightarrow T_f \rightarrow \theta_{cmd} \rightarrow \theta$$

Hardware

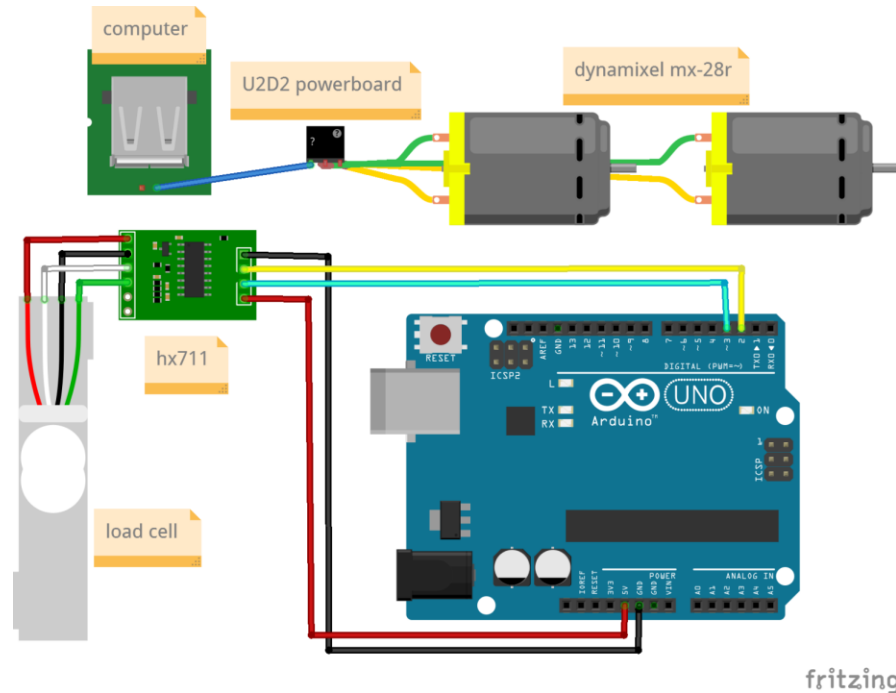
Hardware Test



GoFish: Tension Regulation Fishing Robot

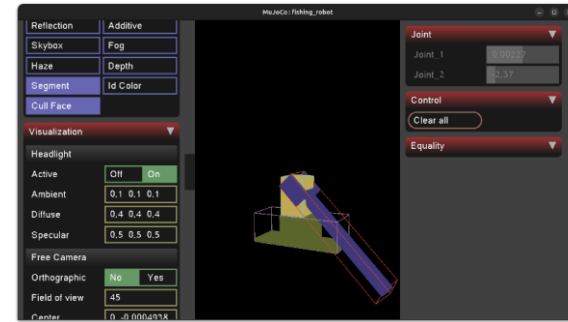
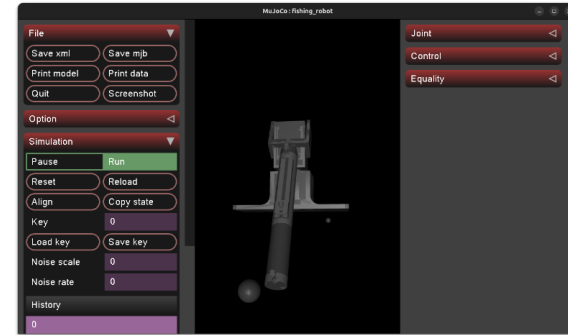
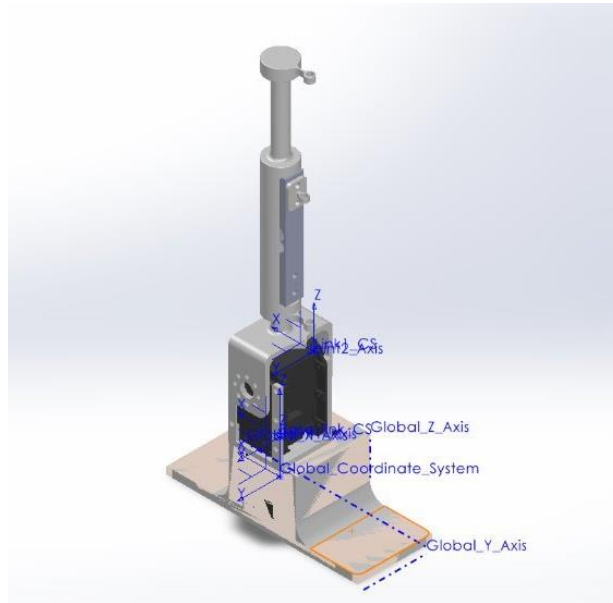


Wiring Diagram



Software

Simulation (mujoco)



Codebase (true implementation)

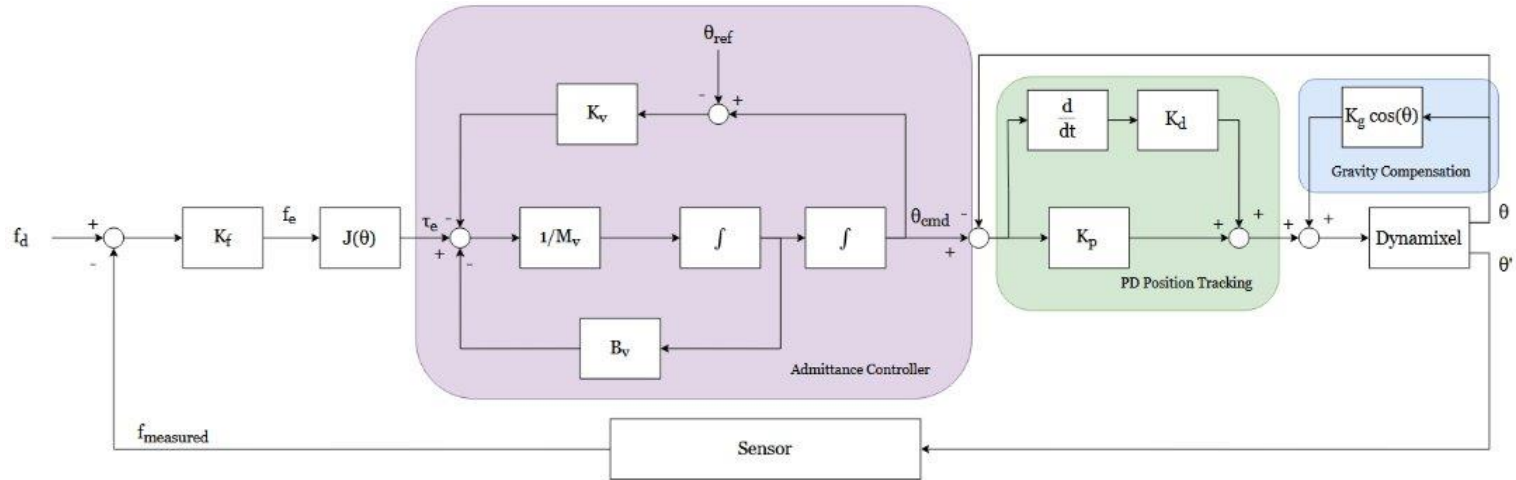
ROS2 - software architecture modularity

Building out the ROS2 codebase such that the control logic interacts seamlessly with Mujoco simulation and hardware control

- *Bringup*: launch configuration. Can launch simulation or hardware control based on preference
- *Description*: mesh and model descriptors
- *Control*: tension control package
- *Planning*: will likely be used to control randomness of 'fish' behavior. Can be thought of as a fish agent (our disturbance)
- *Sensors*: sensor readings and publishing
- *Interfaces*: develop protocol for inter-node communication

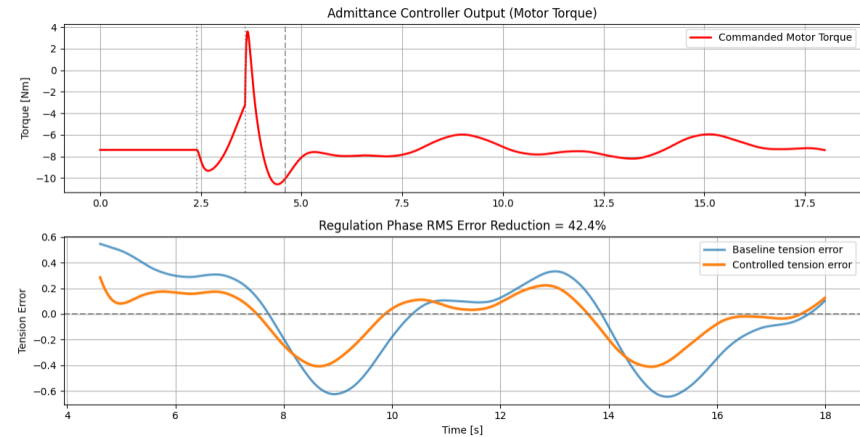
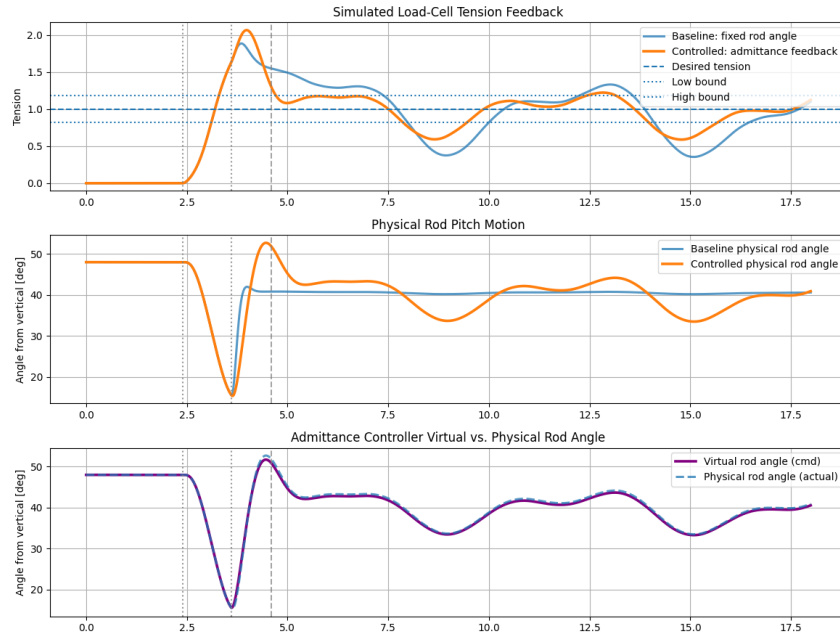
Controller & Simulation

Controller Diagram (Admittance Control)

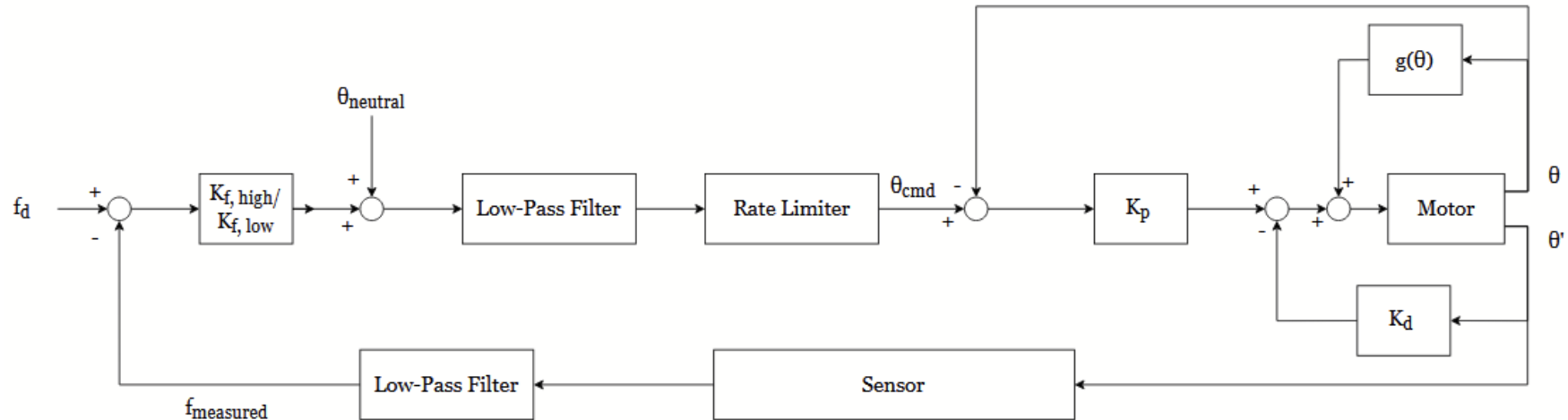


$$\mathbf{M}_v \ddot{\theta}_{cmd} + \mathbf{B}_v \dot{\theta}_{cmd} + \mathbf{K}_v (\theta_{cmd} - \theta_{ref}) = -J(\theta) \mathbf{K}_f (f_d - f_{measured})$$

Admittance Controller Results

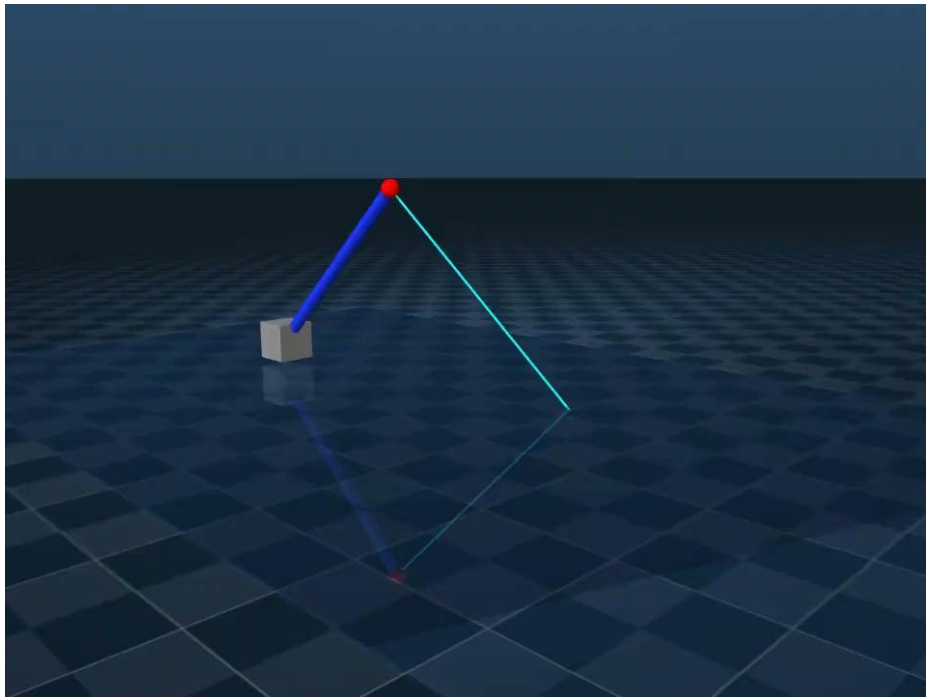


Controller Diagram (Proportional Force Control)

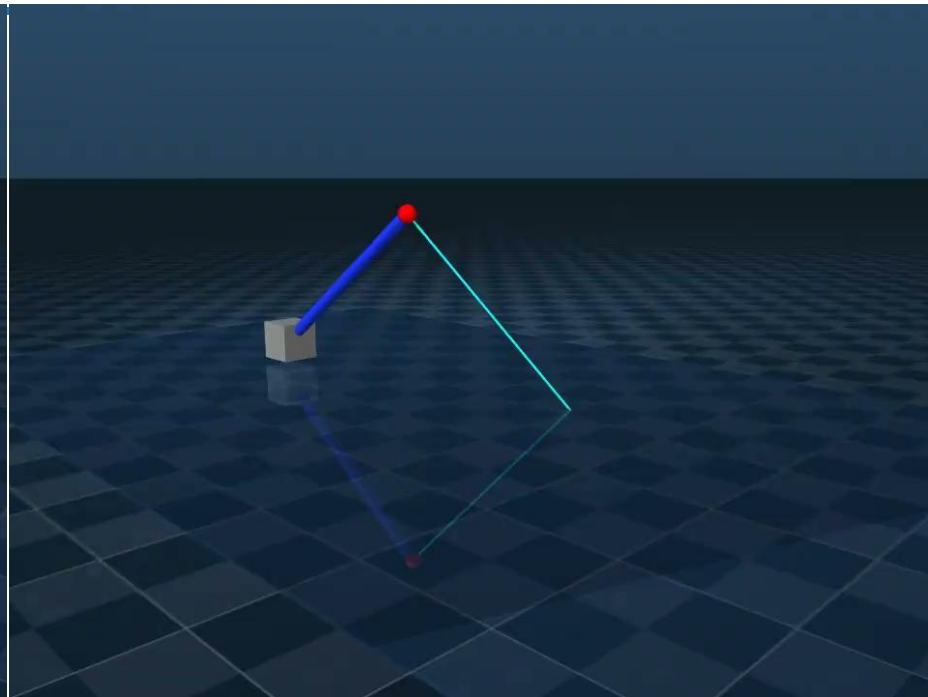


$$K_{f, \text{low/high}}(f_d - f_{\text{measured}}) + \theta_{\text{neutral}} = \theta_{\text{cmd}}$$

Admittance Controller

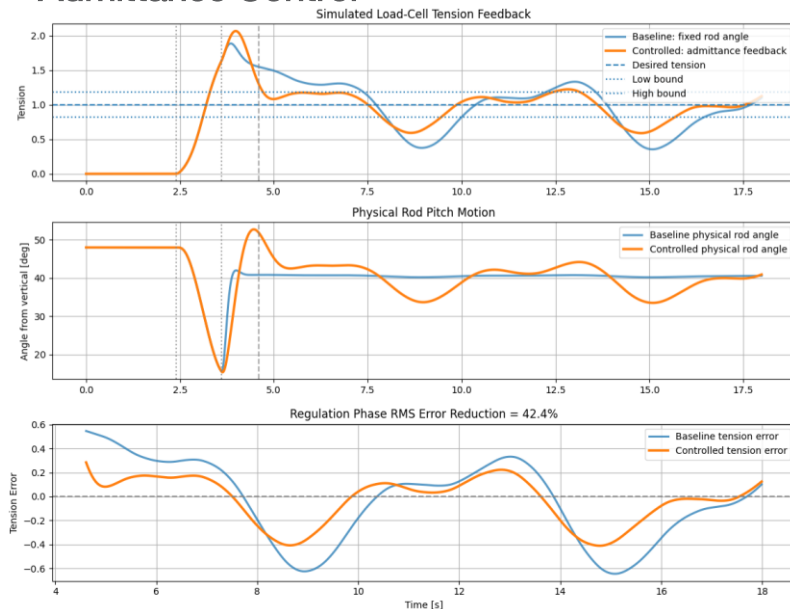


Force Controller

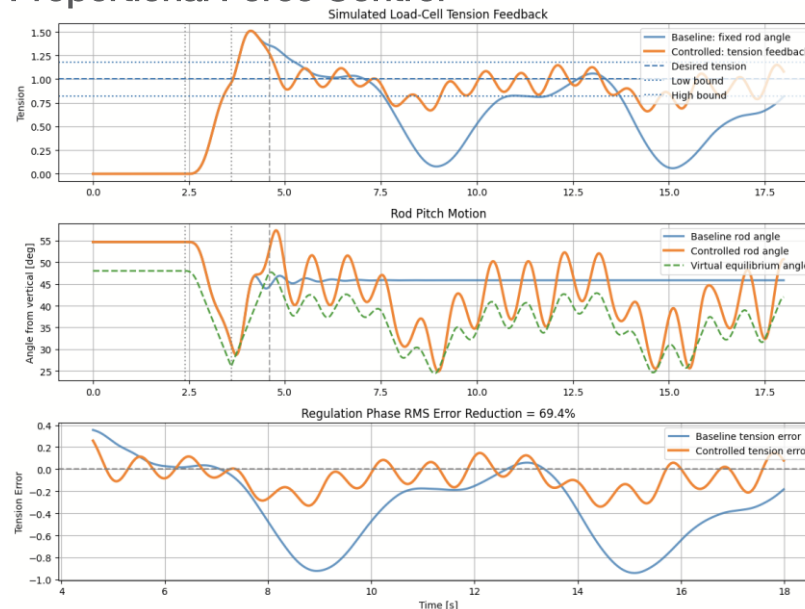


Controller Comparison

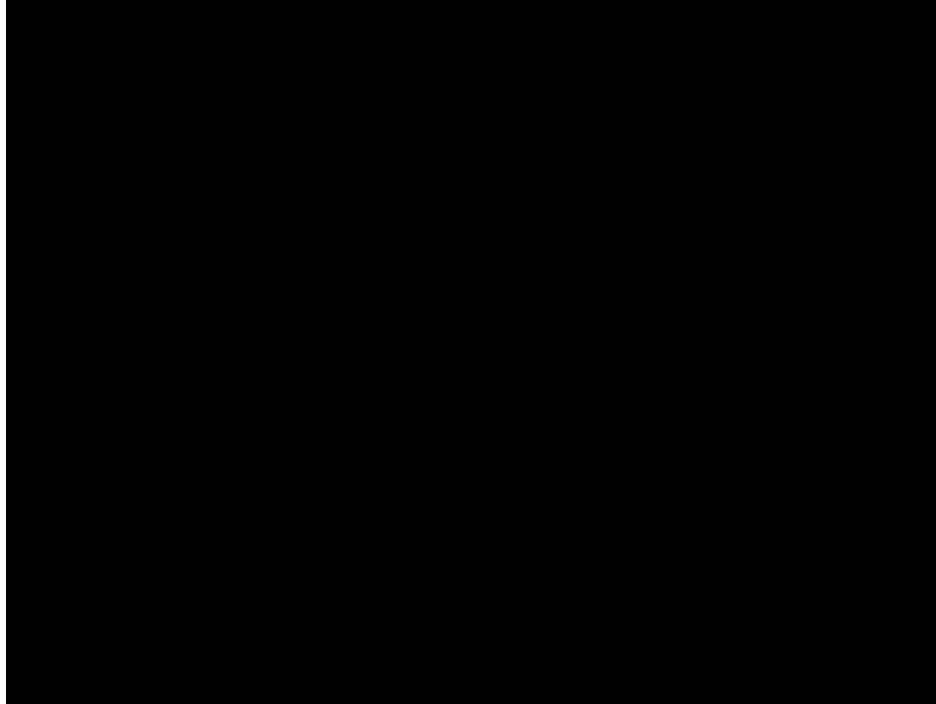
Admittance Control



Proportional Force Control



Results

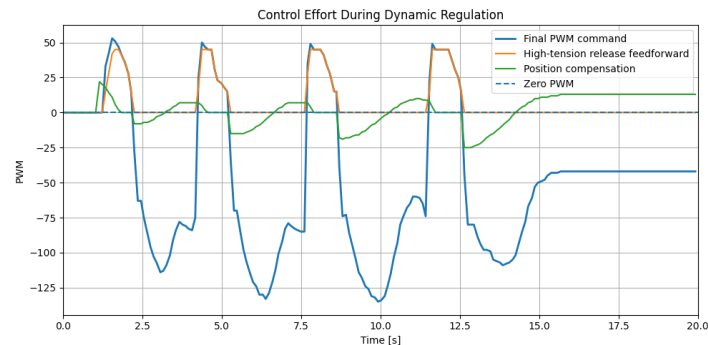
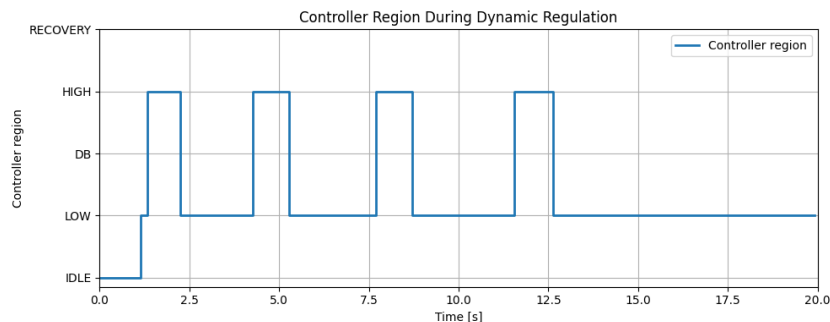


Manual pulling creates line-tension changes, and the rod responds through the Dynamixel motor.

Dynamic Controller Response

Test condition: Rapid manual pulls were applied to create sudden line-tension changes.

Key result: the controller switched between release and pull-back actions, and the PWM command changed accordingly.



Post-Transient Regulation Accuracy

Test condition:

Initial manual pull followed by steady holding to evaluate whether the controller can return the tension near the 3.5 N target.

Target tension: 3.5 N

Final 5 s window:

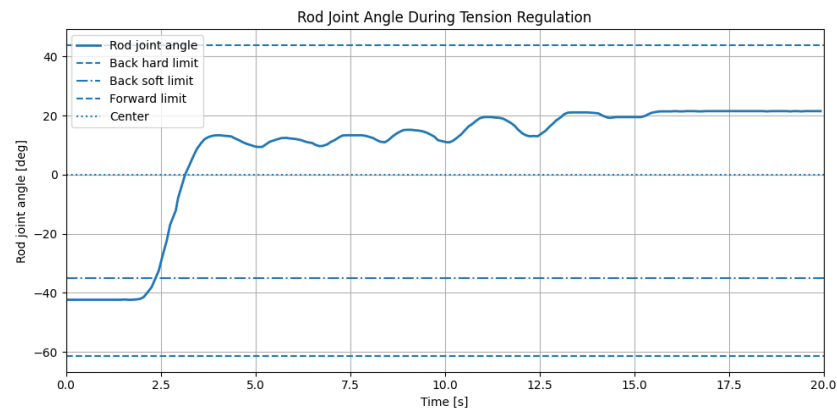
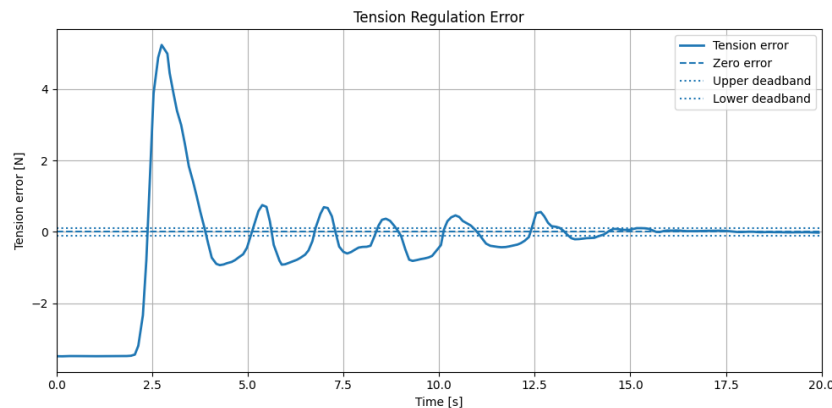
Mean tension: 3.52 N

MAE: 0.025 N (0.72%)

RMSE: 0.036 N (1.03%)

Takeaway:

After the initial engagement and correction phase, the controller reduced the tension error and maintained the measured tension close to the target



Future Developments

In truth, the fishing problem is extremely control rich in
my opinion
I will try to convey this nuance

Algorithmic extensions

2D Tension control

Planarized fish movement adds sensing complexity

- Our load sensor is 1D
 - instead of direct linear force/tension estimation relation, need to be aware of tension vector
 - use load sensor info and kinematic state (encoders) -> **kinematic variation**
- OR
- (use joint torque/position and load cell) -> sensor fusion, **extended kalman filter**
- complicates control problem
 - nonlinear coupling dynamics
 - orthogonal subspaces (hybrid control)

Hybrid control?

- Force Control (Admittance) along the axis of the fishing line to maintain perfect tension
- Rigid Position Control in the perpendicular axes
- Motivation: stiffness to reorient rod optimally, while allowing compliance on the force adaptive pulling axis

Algorithmic extensions

More robust state switching capability

Current Finite State machine uses force thresholding, which is very “chattery”

Solutions/Considerations:

- Hysteresis (anisotropic force thresholds for entering and exiting a state) & Debouncing (time based thresholding/conditioning) to limit false/rapid triggering
- Bumpless transfer
 - smoothing filters
 - transition states?
- Tension break detection (rate of force change) and **funnel synthesis (bound tracking error)**
 - Drops into a safe, heavily damped recovery ‘funnel’ in case of line break preventing the robotic arm from violently whipping backward and breaking motors
 - funnel synthesis concept also useful for control regain
- Admittance Control hardware implementation (gain scheduling)

Algorithmic extensions

Noise reduction

Probabilistic state estimation (HMM, Bayesian filter)

[important in RL, partial observability/limited info scenarios]

- Frequency and derivative of the force to distinguish between fake disturbances (boat rocking, snag on seaweed) or an actual fish

OR

Just use vision (unless there are partial observability concerns)

Algorithmic extensions (AI)

Adaptive neural network

Augmentation of control loop with an adaptive neural network feedforward term that learns the stochastic frequency of the fish's pulling behavior, effectively predicting the required q_d before the tension error grows too large.

Sim-to-real: Using the simulation as an AI training tool to optimize fish tensioning behavior.

- Direct RL: neural network
- Layered RL: controller parameter tweaking
- sim pipeline can also be used for adaptive control tuning without AI

Hardware Developments

Redundantly actuated serial joint

- Emulate rod compliance with active control capability?
- Macro-micro manipulator structure
 - **heavy base joint** handle low-frequency, high-amplitude positioning (like tracking boat's movement)
 - A **smaller, redundantly actuated serial joint** near the tip can be tuned with a much higher control loop frequency to handle high-frequency stochastic disturbances
 - **inertial decoupling**: torque efficiency, jerky movements doesn't cause heavy base motor to respond
- Null space optimization
 - infinite joint configs.
 - initiate secondary optimization objectives (torque minimization)
- Dynamic impedance shaping
 - redundant motor acts as dynamically tunable physical stiffness unit that adheres to control needs

Hardware Developments

A **compliant segment** added to the end of the rod

- tune stiffness characteristics
- **mechanical LPF**, dealing with incredibly jerky fish motion in true fishing scenarios
- **bumpless transfer stabilization** (state transitions), reduce risky torque spikes

Hardware testing rig

- Actuatable object to mimic a fish that has planarized movement capabilities to perform varied experiments on our hardware system and its accompanying controller design

Practical Additions

Bringing the product to 'market'

- Tensioning reel mechanism
- actuated hooking system

Thank You

```
In [ ]: !apt-get install -y libgl1-mesa-glx libegl1-mesa libgles2-mesa-dev libosmesa  
!pip install mujoco mediapy matplotlib numpy -q
```

```
_____ 42.5/42.5 kB 2.8 MB/s eta 0:0  
0:00  
_____ 7.3/7.3 MB 66.5 MB/s eta 0:00:00  
_____ 243.5/243.5 kB 25.9 MB/s eta 0:0  
0:00  
_____ 4.9/4.9 MB 127.5 MB/s eta 0:00:0  
0
```

```
In [ ]: import os  
os.environ["MUJOCO_GL"] = "egl"  
  
import numpy as np  
import mujoco  
import mediapy as media  
import matplotlib.pyplot as plt  
  
xml = """  
<mujoco model="one_dof_fishing_rod_3d_fish_motion">  
  <compiler angle="degree" coordinate="local"/>  
  <option timestep="0.01" gravity="0 0 -9.81"/>  
  
  <visual>  
    <global offwidth="960" offheight="720"/>  
    <headlight diffuse="0.6 0.6 0.6" ambient="0.3 0.3 0.3" specular="0 0 0"/>  
    <rgba haze="0.15 0.25 0.35 1"/>  
  </visual>  
  
  <asset>  
    <texture type="skybox" builtin="gradient"  
      rgb1="0.3 0.5 0.7" rgb2="0 0 0"  
      width="512" height="3072"/>  
    <texture type="2d" name="groundplane" builtin="checker"  
      mark="edge" rgb1="0.2 0.3 0.4" rgb2="0.1 0.2 0.3"  
      markrgb="0.8 0.8 0.8" width="300" height="300"/>  
    <material name="groundplane" texture="groundplane"  
      texuniform="true" texrepeat="5 5" reflectance="0.2"/>  
  
    <material name="blue" rgba="0.1 0.2 0.9 1"/>  
    <material name="cyan" rgba="0.0 0.8 0.9 1"/>  
    <material name="gray" rgba="0.5 0.5 0.5 1"/>  
    <material name="water" rgba="0.2 0.5 0.9 0.10"/>  
  </asset>  
  
  <worldbody>  
    <light pos="0 -3 4" dir="0 1 -1"/>  
    <geom name="floor" type="plane" size="0 0 0.05" material="groundplane"/>  
  
    <!-- Visual-only water surface -->  
    <geom name="water_surface" type="box" pos="1.4 0 0.0"  
      size="2.4 1.2 0.01" material="water"  
      contype="0" conaffinity="0"/>
```

```

<body name="base" pos="0 0 0.10">
  <geom type="box" size="0.08 0.08 0.08" material="gray"/>

  <!--
  1-DOF vertical rod:
  q = 0 deg: rod vertical upward
  q increases: rod lowers forward in +x direction
  -->
  <body name="rod" pos="0 0 0">
    <joint name="pitch" type="hinge" axis="0 1 0"
      limited="true" range="5 75" damping="1.8" armature="0.03"/>
    <geom name="rod_geom" type="capsule"
      fromto="0 0 0 0 0 1.0"
      size="0.025" material="blue"/>
    <site name="rod_tip" pos="0 0 1.0" size="0.035" rgba="1 0 0 1"/>
  </body>
</body>

<!-- Fish target controlled from Python -->
<body name="fish" mocap="true" pos="1.8 0.45 -0.50">
  <geom name="fish_geom" type="sphere" size="0.10"
    material="cyan" contype="0" conaffinity="0"/>
</body>
</worldbody>

<actuator>
  <position name="pitch_position" joint="pitch" kp="70"/>
</actuator>
</mujoco>
"""

model = mujoco.MjModel.from_xml_string(xml)
data = mujoco.MjData(model)

PITCH_JOINT_ID = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_JOINT, "pitch")
PITCH_QPOS_ID = model.jnt_qposadr[PITCH_JOINT_ID]

ROD_TIP_ID = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_SITE, "rod_tip")
FISH_BODY_ID = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_BODY, "fish")
FISH_MOCAP_ID = model.body_mocapid[FISH_BODY_ID]

renderer = mujoco.Renderer(model, height=720, width=960)

cam = mujoco.MjvCamera()
cam.type = mujoco.mjtCamera.mjCAMERA_FREE
cam.lookat[:] = [1.10, 0.05, 0.20]
cam.distance = 3.0
cam.azimuth = 135
cam.elevation = -12

print(f"MuJoCo {mujoco.__version__} | nq={model.nq} nv={model.nv} nu={model.nu}")
print("Setup complete.")

```

MuJoCo 3.8.1 | nq=1 nv=1 nu=1
Setup complete.

```

In [ ]: DT = model.opt.timestep
T_FINAL = 18.0
DT_FRAME = 0.04

STATE_MONITOR = 0
STATE_HOOK_SET = 1
STATE_REGULATE = 2

T_DESIRED = 1.0
T_LOW = 0.82
T_HIGH = 1.18

BITE_TIME = 2.4
HOOK_DURATION = 1.2

# Large enough motion for presentation
Q_INIT = np.deg2rad(48.0)
Q_HOOK = np.deg2rad(14.0)
Q_NEUTRAL = np.deg2rad(40.0)

Q_MIN = np.deg2rad(6.0)
Q_MAX = np.deg2rad(72.0)

FISH_BASE = np.array([1.80, 0.45, -0.50])

FILTER_ALPHA = 0.055

# Stronger response, but with asymmetric controller below
TENSION_TO_ANGLE_GAIN = np.deg2rad(42.0)
QEQ_ALPHA = 0.055
MAX_QEQ_STEP = np.deg2rad(0.22)

# Rod effect on simulated load-cell tension
ROD_TENSION_GAIN = 3.05
TENSION_TAU = 0.26

def set_fish_position(pos):
    data.mocap_pos[FISH_MOCAP_ID] = pos
    data.mocap_quat[FISH_MOCAP_ID] = np.array([1.0, 0.0, 0.0, 0.0])

def smoothstep(s):
    s = np.clip(s, 0.0, 1.0)
    return 3 * s**2 - 2 * s**3

def get_state(t):
    if t < BITE_TIME:
        return STATE_MONITOR
    elif t < BITE_TIME + HOOK_DURATION:
        return STATE_HOOK_SET
    else:
        return STATE_REGULATE

```

```

def fish_motion_and_pull(t, state):
    """
    Fish position is used for 3D visualization.
    Fish pull is the simulated load-cell disturbance.

    Since the current robot is only 1-DOF pitch,
    the fish's 3D motion is treated as an external disturbance,
    not as a position-tracking target.
    """
    if state == STATE_MONITOR:
        fish_pos = FISH_BASE.copy()
        fish_pull = 0.0

    elif state == STATE_HOOK_SET:
        s = smoothstep((t - BITE_TIME) / HOOK_DURATION)

        fish_pos = FISH_BASE + np.array([
            0.12 * s,
            0.06 * np.sin(np.pi * s),
            -0.10 * s
        ])

        fish_pull = 0.75 + 0.55 * np.sin(np.pi * s)

    else:
        tau = t - BITE_TIME - HOOK_DURATION

        # 3D fish-like motion:
        # x: forward/backward
        # y: left/right
        # z: up/down underwater
        fish_x = 1.90 + 0.18 * np.sin(0.75 * tau)
        fish_y = 0.45 + 0.18 * np.sin(0.55 * tau + 0.7)
        fish_z = -0.55 + 0.10 * np.sin(0.85 * tau + 1.4)

        fish_pos = np.array([fish_x, fish_y, fish_z])

        # Simulated load-cell fish pulling force.
        # This is not directly computed from fish position because
        # hardware will use an actual load cell reading.
        fish_pull = (
            1.00
            + 0.46 * np.sin(0.95 * tau)
            + 0.22 * np.sin(2.10 * tau + 0.7)
            + 0.10 * np.sin(0.38 * tau + 1.2)
        )

    return fish_pos, max(fish_pull, 0.0)

def add_visual_line(renderer, tip, fish, radius=0.005):
    scene = renderer.scene

    if scene.ngeom >= scene.maxgeom:
        return

```



```

geom = scene.geoms[scene.ngeom]

mujoco.mjv_initGeom(
    geom,
    mujoco.mjtGeom.mjGEOM_CAPSULE,
    np.zeros(3),
    np.zeros(3),
    np.eye(3).reshape(-1),
    np.array([0.0, 0.9, 0.9, 1.0])
)

mujoco.mjv_connector(
    geom,
    mujoco.mjtGeom.mjGEOM_CAPSULE,
    radius,
    np.asarray(tip, dtype=np.float64),
    np.asarray(fish, dtype=np.float64)
)

scene.ngeom += 1

def simulated_load_cell_update(t, state, fish_pull, q_actual, T_true_prev, T
    """
    Simulated load-cell signal.
    In hardware, this block will be replaced by real load-cell reading.
    """
    if state == STATE_MONITOR:
        T_target = 0.0
    else:
        # q smaller = rod raised = more line tension
        # q larger = rod lowered = less line tension
        rod_effect = ROD_TENSION_GAIN * (Q_NEUTRAL - q_actual)

        noise = 0.010 * np.sin(8.0 * t) + 0.005 * np.sin(17.0 * t + 0.4)
        T_target = max(fish_pull + rod_effect + noise, 0.0)

    beta = DT / (TENSION_TAU + DT)
    T_true = (1.0 - beta) * T_true_prev + beta * T_target

    T_filtered = (1.0 - FILTER_ALPHA) * T_filtered_prev + FILTER_ALPHA * T_t

    return T_true, T_filtered

def controller_update(t, state, q_eq_prev, T_filtered, control_enabled):
    """
    Baseline:
        Hook-setting, then hold a fixed rod angle.

    Controlled:
        High tension -> q_eq increases -> rod lowers forward.
        Low tension -> q_eq decreases -> rod raises upward.

    This version uses asymmetric feedback:

```

```

        - If tension is too low, raise the rod more actively.
        - If tension is too high, lower the rod more gently.
    """
    if state == STATE_MONITOR:
        q_target = Q_INIT

    elif state == STATE_HOOK_SET:
        s = smoothstep((t - BITE_TIME) / HOOK_DURATION)
        q_target = Q_INIT + (Q_HOOK - Q_INIT) * s

    else:
        if control_enabled:
            tension_error = T_filtered - T_DESIRED

            if tension_error < 0:
                # Tension too low: raise rod more actively
                q_target = Q_NEUTRAL + 1.25 * TENSION_TO_ANGLE_GAIN * tension_error
            else:
                # Tension too high: lower rod more gently
                q_target = Q_NEUTRAL + 0.80 * TENSION_TO_ANGLE_GAIN * tension_error
        else:
            q_target = Q_NEUTRAL

    q_target = np.clip(q_target, Q_MIN, Q_MAX)

    q_smooth = (1.0 - QEQ_ALPHA) * q_eq_prev + QEQ_ALPHA * q_target
    q_eq = np.clip(q_smooth, q_eq_prev - MAX_QEQ_STEP, q_eq_prev + MAX_QEQ_STEP)
    q_eq = np.clip(q_eq, Q_MIN, Q_MAX)

    return q_eq, q_target

def run_simulation(control_enabled=True, make_video=True):
    mujoco.mj_resetData(model, data)

    data.qpos[PITCH_QPOS_ID] = Q_INIT
    data.ctrl[0] = Q_INIT
    set_fish_position(FISH_BASE)
    mujoco.mj_forward(model, data)

    # Let the rod settle before logging
    for _ in range(700):
        data.ctrl[0] = Q_INIT
        set_fish_position(FISH_BASE)
        mujoco.mj_step(model, data)

    data.time = 0.0

    out = {
        "t": [],
        "q": [],
        "q_eq": [],
        "q_target": [],
        "T_true": [],
        "T_filtered": [],
        "fish_pull": [],
    }

```

```

        "state": [],
        "fish_pos": [],
        "rod_tip": [],
        "line_length": [],
        "frames": []
    }

    q_eq = Q_INIT
    T_true = 0.0
    T_filtered = 0.0
    t_frame = 0.0

    while data.time < T_FINAL:
        t = data.time
        state = get_state(t)

        mujoco.mj_forward(model, data)
        q_actual = data.qpos[PITCH_QPOS_ID]

        fish_pos, fish_pull = fish_motion_and_pull(t, state)
        set_fish_position(fish_pos)
        mujoco.mj_forward(model, data)

        tip = data.site_xpos[ROD_TIP_ID].copy()

        T_true, T_filtered = simulated_load_cell_update(
            t=t,
            state=state,
            fish_pull=fish_pull,
            q_actual=q_actual,
            T_true_prev=T_true,
            T_filtered_prev=T_filtered
        )

        q_eq, q_target = controller_update(
            t=t,
            state=state,
            q_eq_prev=q_eq,
            T_filtered=T_filtered,
            control_enabled=control_enabled
        )

        data.ctrl[0] = q_eq
        mujoco.mj_step(model, data)

        line_length = np.linalg.norm(fish_pos - tip)

        out["t"].append(t)
        out["q"].append(np.rad2deg(q_actual))
        out["q_eq"].append(np.rad2deg(q_eq))
        out["q_target"].append(np.rad2deg(q_target))
        out["T_true"].append(T_true)
        out["T_filtered"].append(T_filtered)
        out["fish_pull"].append(fish_pull)
        out["state"].append(state)
        out["fish_pos"].append(fish_pos.copy())

```

```

        out["rod_tip"].append(tip.copy())
        out["line_length"].append(line_length)

        if make_video and data.time >= t_frame:
            renderer.update_scene(data, camera=cam)
            add_visual_line(renderer, tip, fish_pos, radius=0.005)
            frame = renderer.render().copy()
            out["frames"].append(frame)
            t_frame += DT_FRAME

    for k in out:
        if k != "frames":
            out[k] = np.array(out[k])

    return out

print("Controller and simulation driver defined.")

```

Controller and simulation driver defined.

```

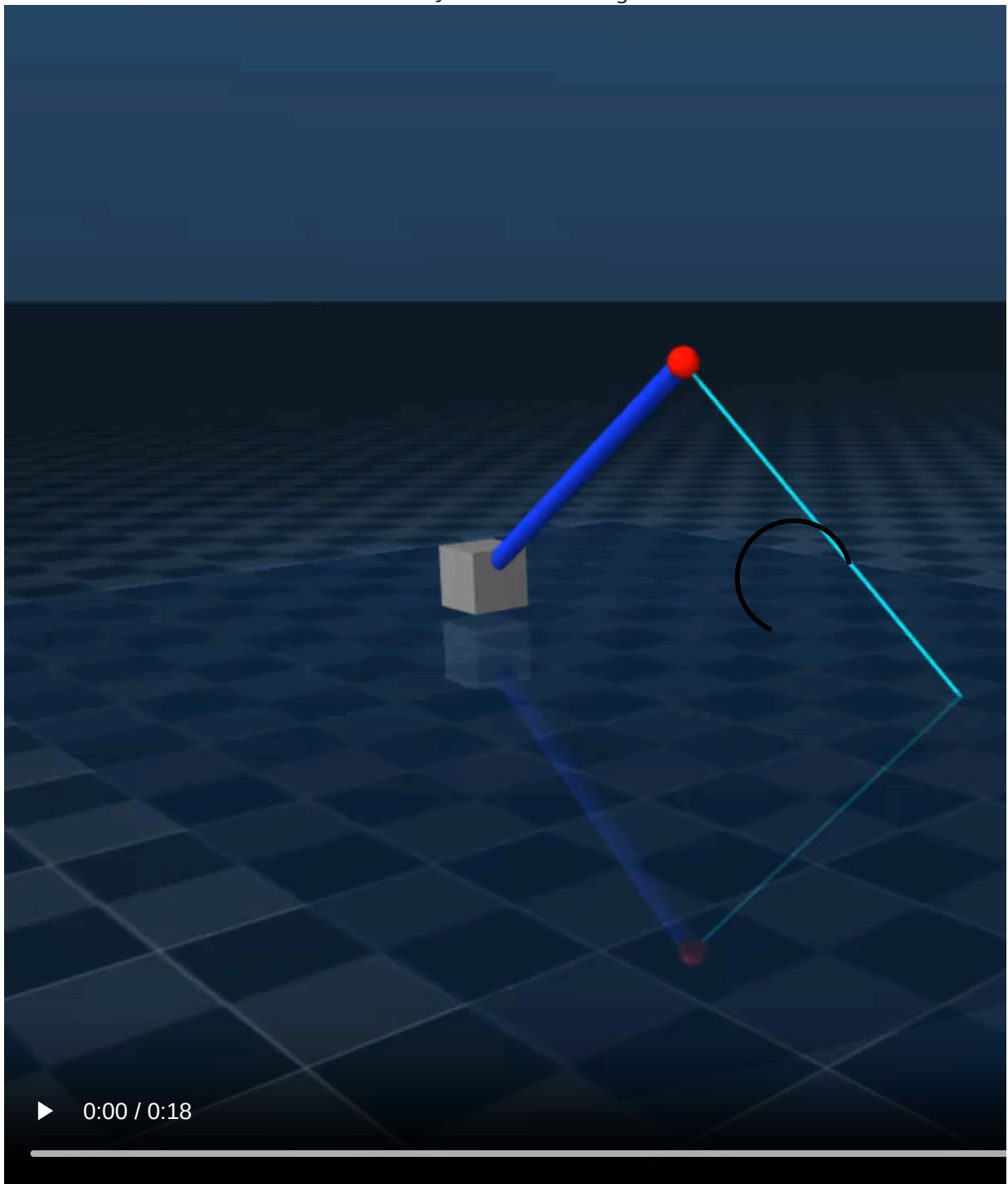
In [ ]: res_base = run_simulation(control_enabled=False, make_video=False)
        res_ctrl = run_simulation(control_enabled=True, make_video=True)

        print(f"Baseline samples: {len(res_base['t'])}")
        print(f"Controlled samples: {len(res_ctrl['t'])}")
        print(f"Controlled video frames: {len(res_ctrl['frames'])}")

        media.show_video(
            res_ctrl["frames"],
            fps=int(1 / DT_FRAME),
            title="MuJoCo 1-DOF Fishing Rod: 3D Fish Disturbance + Load-Cell Feedback"
        )

```

Baseline samples: 1800
 Controlled samples: 1800
 Controlled video frames: 451



```
In [ ]: def compute_metrics(res_base, res_ctrl):
    reg_start = BITE_TIME + HOOK_DURATION + 1.0

    mask_b = res_base["t"] > reg_start
    mask_c = res_ctrl["t"] > reg_start

    err_b = res_base["T_filtered"][mask_b] - T_DESIRED
    err_c = res_ctrl["T_filtered"][mask_c] - T_DESIRED

    rms_b = np.sqrt(np.mean(err_b**2))
    rms_c = np.sqrt(np.mean(err_c**2))
```

```

improvement = (1.0 - rms_c / rms_b) * 100.0

return rms_b, rms_c, improvement, reg_start

rms_b, rms_c, improvement, reg_start = compute_metrics(res_base, res_ctrl)

print(f"Baseline RMS tension error: {rms_b:.3f}")
print(f"Controlled RMS tension error: {rms_c:.3f}")
print(f"Error reduction: {improvement:.1f}%")

plt.figure(figsize=(12, 9))

# 1. Tension comparison
plt.subplot(3, 1, 1)
plt.plot(res_base["t"], res_base["T_filtered"],
         label="Baseline: fixed rod angle", linewidth=2.2, alpha=0.75)
plt.plot(res_ctrl["t"], res_ctrl["T_filtered"],
         label="Controlled: tension feedback", linewidth=2.6)
plt.axhline(T_DESIRED, linestyle="--", label="Desired tension")
plt.axhline(T_LOW, linestyle=":", label="Low bound")
plt.axhline(T_HIGH, linestyle=":", label="High bound")
plt.axvline(BITE_TIME, linestyle=":", color="gray", alpha=0.8)
plt.axvline(BITE_TIME + HOOK_DURATION, linestyle=":", color="gray", alpha=0.8)
plt.axvline(reg_start, linestyle="--", color="gray", alpha=0.7)
plt.ylabel("Tension")
plt.title("Simulated Load-Cell Tension Feedback")
plt.legend(loc="upper right")
plt.grid(True)

# 2. Rod angle
plt.subplot(3, 1, 2)
plt.plot(res_base["t"], res_base["q"],
         label="Baseline rod angle", linewidth=2.2, alpha=0.75)
plt.plot(res_ctrl["t"], res_ctrl["q"],
         label="Controlled rod angle", linewidth=2.6)
plt.plot(res_ctrl["t"], res_ctrl["q_eq"],
         label="Virtual equilibrium angle", linewidth=1.8)
plt.axvline(BITE_TIME, linestyle=":", color="gray", alpha=0.8)
plt.axvline(BITE_TIME + HOOK_DURATION, linestyle=":", color="gray", alpha=0.8)
plt.axvline(reg_start, linestyle="--", color="gray", alpha=0.7)
plt.ylabel("Angle from vertical [deg]")
plt.title("Rod Pitch Motion")
plt.legend(loc="upper right")
plt.grid(True)

# 3. Regulation-phase tension error only
plt.subplot(3, 1, 3)
mask_b = res_base["t"] > reg_start
mask_c = res_ctrl["t"] > reg_start

plt.plot(res_base["t"][mask_b],
         res_base["T_filtered"][mask_b] - T_DESIRED,
         label="Baseline tension error", linewidth=2.2, alpha=0.75)
plt.plot(res_ctrl["t"][mask_c],

```

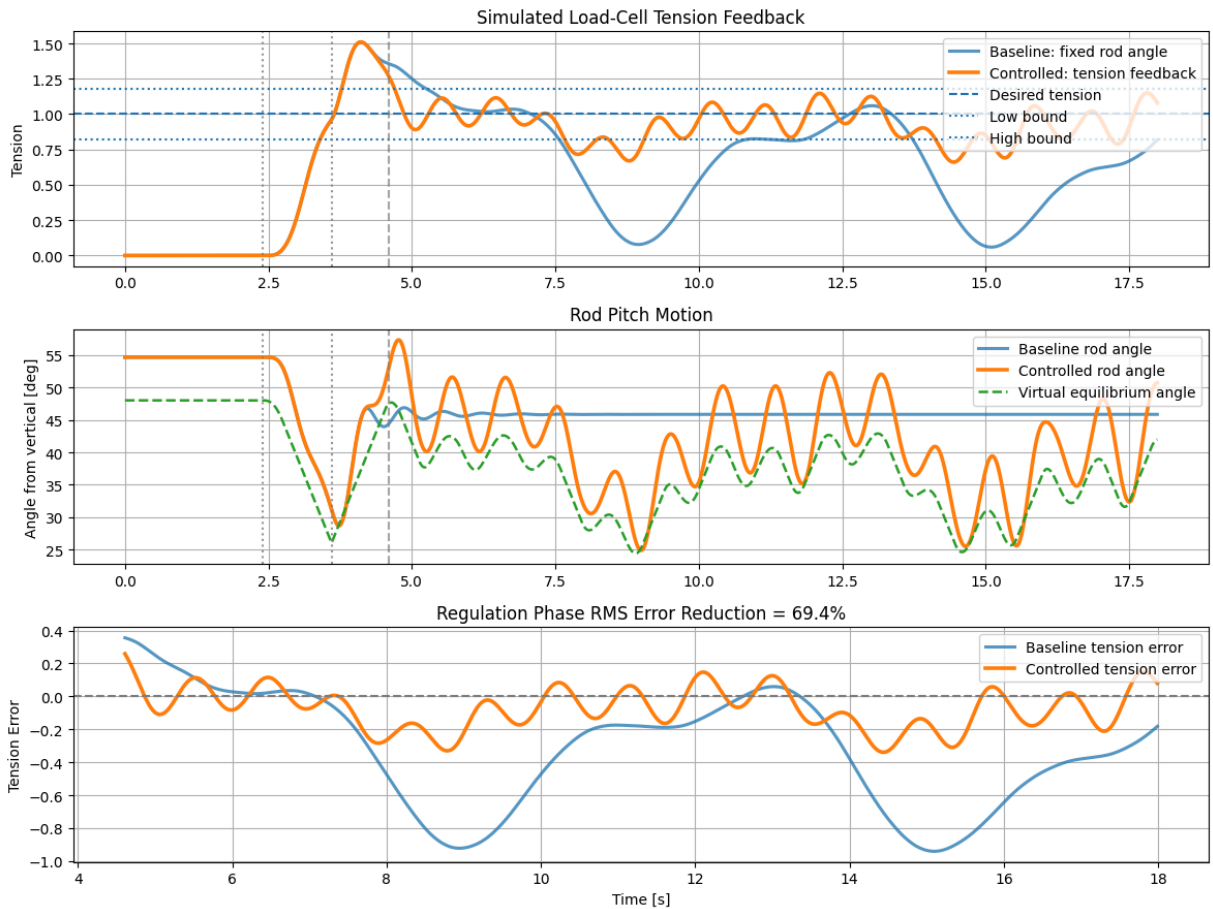
```

        res_ctrl["T_filtered"][mask_c] - T_DESIRED,
        label="Controlled tension error", linewidth=2.6)
plt.axhline(0, linestyle="--", color="gray")
plt.ylabel("Tension Error")
plt.xlabel("Time [s]")
plt.title(f"Regulation Phase RMS Error Reduction = {improvement:.1f}%")
plt.legend(loc="upper right")
plt.grid(True)

plt.tight_layout()
plt.show()

```

Baseline RMS tension error: 0.475
 Controlled RMS tension error: 0.145
 Error reduction: 69.4%



```
In [ ]: !apt-get install -y libgl1-mesa-glx libegl1-mesa libgles2-mesa-dev libosmesa6-dev
!pip install mujoco mediapy matplotlib numpy -q
```

```
In [ ]: import os
os.environ["MUJOCO_GL"] = "egl"

import numpy as np
import mujoco
import mediapy as media
import matplotlib.pyplot as plt

xml = """
<mujoco model="one_dof_fishing_rod_3d_fish_motion">
  <compiler angle="degree" coordinate="local"/>
  <option timestep="0.01" gravity="0 0 -9.81"/>

  <visual>
    <global offwidth="960" offheight="720"/>
    <headlight diffuse="0.6 0.6 0.6" ambient="0.3 0.3 0.3" specular="0 0 0"/>
    <rgba haze="0.15 0.25 0.35 1"/>
  </visual>

  <asset>
    <texture type="skybox" builtin="gradient"
      rgb1="0.3 0.5 0.7" rgb2="0 0 0"
      width="512" height="3072"/>
    <texture type="2d" name="groundplane" builtin="checker"
      mark="edge" rgb1="0.2 0.3 0.4" rgb2="0.1 0.2 0.3"
      markrgb="0.8 0.8 0.8" width="300" height="300"/>
    <material name="groundplane" texture="groundplane"
      texuniform="true" texrepeat="5 5" reflectance="0.2"/>

    <material name="blue" rgba="0.1 0.2 0.9 1"/>
    <material name="cyan" rgba="0.0 0.8 0.9 1"/>
    <material name="gray" rgba="0.5 0.5 0.5 1"/>
    <material name="water" rgba="0.2 0.5 0.9 0.10"/>
  </asset>

  <worldbody>
    <light pos="0 -3 4" dir="0 1 -1"/>
    <geom name="floor" type="plane" size="0 0 0.05" material="groundplane"/>

    <!-- Visual-only water surface -->
    <geom name="water_surface" type="box" pos="1.4 0 0.0"
      size="2.4 1.2 0.01" material="water"
      contype="0" conaffinity="0"/>

    <body name="base" pos="0 0 0.10">
      <geom type="box" size="0.08 0.08 0.08" material="gray"/>

      <!--
      1-DOF vertical rod:
      q = 0 deg: rod vertical upward
      q increases: rod lowers forward in +x direction
      -->
    </body>
  </worldbody>
</mujoco>
"""
```



```

-->
<body name="rod" pos="0 0 0">
  <joint name="pitch" type="hinge" axis="0 1 0"
    limited="true" range="5 75" damping="1.8" armature="0.03"/>
  <geom name="rod_geom" type="capsule"
    fromto="0 0 0 0 0 1.0"
    size="0.025" material="blue"/>
  <site name="rod_tip" pos="0 0 1.0" size="0.035" rgba="1 0 0 1"/>
</body>
</body>

<!-- Fish target controlled from Python -->
<body name="fish" mocap="true" pos="1.8 0.45 -0.50">
  <geom name="fish_geom" type="sphere" size="0.10"
    material="cyan" contype="0" conaffinity="0"/>
</body>
</worldbody>

<actuator>
  <motor name="pitch_torque" joint="pitch" ctrllimited="true" ctrlrange="-
</actuator>
</mujoco>
"""

model = mujoco.MjModel.from_xml_string(xml)
data = mujoco.MjData(model)

PITCH_JOINT_ID = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_JOINT, "pitch")
PITCH_QPOS_ID = model.jnt_qposadr[PITCH_JOINT_ID]

ROD_TIP_ID = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_SITE, "rod_tip")
FISH_BODY_ID = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_BODY, "fish")
FISH_MOCAP_ID = model.body_mocapid[FISH_BODY_ID]

renderer = mujoco.Renderer(model, height=720, width=960)

cam = mujoco.MjvCamera()
cam.type = mujoco.mjtCamera.mjCAMERA_FREE
cam.lookat[:] = [1.10, 0.05, 0.20]
cam.distance = 3.0
cam.azimuth = 135
cam.elevation = -12

print(f"MuJoCo {mujoco.__version__} | nq={model.nq} nv={model.nv} nu={model.nu}")
print("Setup complete.")

```

MuJoCo 3.9.0 | nq=1 nv=1 nu=1
Setup complete.

```

In [ ]: DT = model.opt.timestep
        T_FINAL = 18.0
        DT_FRAME = 0.04

        STATE_MONITOR = 0
        STATE_HOOK_SET = 1
        STATE_REGULATE = 2

```

```

T_DESIRED = 1.0
T_LOW = 0.82
T_HIGH = 1.18

BITE_TIME = 2.4
HOOK_DURATION = 1.2

# Large enough motion for presentation
Q_INIT = np.deg2rad(48.0)
Q_HOOK = np.deg2rad(14.0)
Q_NEUTRAL = np.deg2rad(40.0)

Q_MIN = np.deg2rad(6.0)
Q_MAX = np.deg2rad(72.0)

FISH_BASE = np.array([1.80, 0.45, -0.50])

FILTER_ALPHA = 0.055
PITCH_QVEL_ID = 0 # Update this index based on your MuJoCo model

# Stronger response, but with asymmetric controller below
TENSION_TO_ANGLE_GAIN = np.deg2rad(42.0)
QEQ_ALPHA = 0.055
MAX_QEQ_STEP = np.deg2rad(0.22)

# Rod effect on simulated load-cell tension
ROD_TENSION_GAIN = 3.05
TENSION_TAU = 0.26

def set_fish_position(pos):
    data.mocap_pos[FISH_MOCAP_ID] = pos
    data.mocap_quat[FISH_MOCAP_ID] = np.array([1.0, 0.0, 0.0, 0.0])

def smoothstep(s):
    s = np.clip(s, 0.0, 1.0)
    return 3 * s**2 - 2 * s**3

def get_state(t):
    if t < BITE_TIME:
        return STATE_MONITOR
    elif t < BITE_TIME + HOOK_DURATION:
        return STATE_HOOK_SET
    else:
        return STATE_REGULATE

def fish_motion_and_pull(t, state):
    """
    Fish position is used for 3D visualization.
    Fish pull is the simulated load-cell disturbance.

    Since the current robot is only 1-DOF pitch,

```

the fish's 3D motion is treated as an external disturbance,
not as a position-tracking target.

"""

if state == STATE_MONITOR:

 fish_pos = FISH_BASE.copy()

 fish_pull = 0.0

elif state == STATE_HOOK_SET:

 s = smoothstep((t - BITE_TIME) / HOOK_DURATION)

 fish_pos = FISH_BASE + np.array([

 0.12 * s,

 0.06 * np.sin(np.pi * s),

 -0.10 * s

])

 fish_pull = 0.75 + 0.55 * np.sin(np.pi * s)

else:

 tau = t - BITE_TIME - HOOK_DURATION

3D fish-like motion:

x: forward/backward

y: left/right

z: up/down underwater

 fish_x = 1.90 + 0.18 * np.sin(0.75 * tau)

 fish_y = 0.45 + 0.18 * np.sin(0.55 * tau + 0.7)

 fish_z = -0.55 + 0.10 * np.sin(0.85 * tau + 1.4)

 fish_pos = np.array([fish_x, fish_y, fish_z])

Simulated load-cell fish pulling force.

This is not directly computed from fish position because

hardware will use an actual load cell reading.

 fish_pull = (

 1.00

 + 0.46 * np.sin(0.95 * tau)

 + 0.22 * np.sin(2.10 * tau + 0.7)

 + 0.10 * np.sin(0.38 * tau + 1.2)

)

return fish_pos, max(fish_pull, 0.0)

def add_visual_line(renderer, tip, fish, radius=0.005):

 scene = renderer.scene

if scene.ngeom >= scene.maxgeom:

return

 geom = scene.geoms[scene.ngeom]

 mujoco.mjv_initGeom(

 geom,

 mujoco.mjtGeom.mjGEOM_CAPSULE,

 np.zeros(3),

```

        np.zeros(3),
        np.eye(3).reshape(-1),
        np.array([0.0, 0.9, 0.9, 1.0])
    )

    mujoco.mjv_connector(
        geom,
        mujoco.mjtGeom.mjGEOM_CAPSULE,
        radius,
        np.asarray(tip, dtype=np.float64),
        np.asarray(fish, dtype=np.float64)
    )

    scene.ngeom += 1

def simulated_load_cell_update(t, state, fish_pull, q_actual, T_true_prev, T
"""
    Simulated load-cell signal.
    In hardware, this block will be replaced by real load-cell reading.
"""
    if state == STATE_MONITOR:
        T_target = 0.0
    else:
        # q smaller = rod raised = more line tension
        # q larger = rod lowered = less line tension
        rod_effect = ROD_TENSION_GAIN * (Q_NEUTRAL - q_actual)

        noise = 0.010 * np.sin(8.0 * t) + 0.005 * np.sin(17.0 * t + 0.4)
        T_target = max(fish_pull + rod_effect + noise, 0.0)

    beta = DT / (TENSION_TAU + DT)
    T_true = (1.0 - beta) * T_true_prev + beta * T_target

    T_filtered = (1.0 - FILTER_ALPHA) * T_filtered_prev + FILTER_ALPHA * T_t

    return T_true, T_filtered

```

In []: **class** AdmittanceController:

```

    """
    Outer loop:
        Virtual mass-spring-damper driven by force error.

    Inner loop:
        PD position tracking
        Gravity Compensation
    """

    def __init__(self, dt):

        self.dt = dt

        self.theta_cmd = Q_INIT

```

```

self.theta_dot_cmd = 0.0

# Virtual admittance parameters
self.M_v = 0.6
self.B_v = 5.5
self.K_v = 16.0

# Force scaling
self.K_f = 5.0

# Inner position tracking controller
self.Kp_track = 140.0
self.Kd_track = 20.0
self.Ki_track = 0.0

self.pos_int = 0.0

# Gravity compensation
self.K_g = 9.95

self.J_filtered = 0.0
self.J_alpha = 0.05

def get_jacobian(self, theta, tip_pos, fish_pos):
    """
    Jacobian relating rod pitch motion to line length.
    """

    L = 1.0

    line_vec = fish_pos - tip_pos
    distance = np.linalg.norm(line_vec)

    if distance < 1e-6:
        return 0.0

    u = line_vec / distance

    dx_dtheta = L * np.cos(theta)
    dz_dtheta = -L * np.sin(theta)

    return dx_dtheta * u[0] + dz_dtheta * u[2]

def update(
    self,
    t,
    state,
    f_measured,
    theta,
    theta_dot,
    tip_pos,

```

```

        fish_pos,
        control_enabled,
    ):

        if state == STATE_MONITOR or not control_enabled:
            f_desired = 0.0
        else:
            f_desired = T_DESIRED

        force_error = f_desired - f_measured

        J = self.get_jacobian(theta, tip_pos, fish_pos)

        self.J_filtered += self.J_alpha * (J - self.J_filtered)

        tau_force = -(self.J_filtered * force_error) * self.K_f

        tau_spring = self.K_v * (Q_NEUTRAL - self.theta_cmd)

        theta_ddot_cmd = (
            tau_force
            + tau_spring
            - self.B_v * self.theta_dot_cmd
        ) / self.M_v

        # Integrate

        self.theta_dot_cmd += theta_ddot_cmd * self.dt
        self.theta_cmd += self.theta_dot_cmd * self.dt

        # Keep virtual rod inside physical limits

        self.theta_cmd = np.clip(
            self.theta_cmd,
            Q_MIN,
            Q_MAX,
        )

        e_pos = self.theta_cmd - theta
        e_vel = self.theta_dot_cmd - theta_dot

        self.pos_int += e_pos * self.dt
        self.pos_int = np.clip(self.pos_int, -0.15, 0.15)

        torque_cmd = (
            self.Kp_track * e_pos

```

```

        + self.Kd_track * e_vel
        + self.Ki_track * self.pos_int
    )

    torque_cmd += -self.K_g * np.sin(theta)

    torque_cmd = np.clip(torque_cmd, -20.0, 20.0)

    return torque_cmd, self.theta_dot_cmd

def run_simulation(control_enabled=True, make_video=True):
    mujoco.mj_resetData(model, data)
    controller = AdmittanceController(dt=DT)

    data.qpos[PITCH_QPOS_ID] = Q_INIT
    data.qvel[PITCH_QVEL_ID] = 0.0
    set_fish_position(FISH_BASE)
    mujoco.mj_forward(model, data)

    # Settling loop
    for _ in range(700):
        q_actual = data.qpos[PITCH_QPOS_ID]
        q_dot_actual = data.qvel[PITCH_QVEL_ID]
        settle_torque = 50.0 * (Q_INIT - q_actual) - 5.0 * q_dot_actual - cc
        data.ctrl[0] = settle_torque
        set_fish_position(FISH_BASE)
        mujoco.mj_step(model, data)

    data.time = 0.0
    out = {"t": [], "q": [], "q_dot_cmd": [], "torque_cmd": [], "T_true": [],
          "T_filtered": [], "fish_pull": [], "state": [], "fish_pos": [],
          "rod_tip": [], "line_length": [], "frames": [], "theta_virtual":

    T_true = 0.0
    T_filtered = 0.0
    t_frame = 0.0

    while data.time < T_FINAL:
        t = data.time
        state = get_state(t)

        mujoco.mj_forward(model, data)
        q_actual = data.qpos[PITCH_QPOS_ID]
        q_dot_actual = data.qvel[PITCH_QVEL_ID]

        fish_pos, fish_pull = fish_motion_and_pull(t, state)
        set_fish_position(fish_pos)
        mujoco.mj_forward(model, data)

        tip = data.site_xpos[ROD_TIP_ID].copy()

        T_true, T_filtered = simulated_load_cell_update(
            t=t, state=state, fish_pull=fish_pull,
            q_actual=q_actual, T_true_prev=T_true, T_filtered_prev=T_filtered

```

```

)

J_actual = controller.get_jacobian(q_actual, tip, fish_pos)

if state != STATE_MONITOR:
    data.qfrc_applied[PITCH_QVEL_ID] = J_actual * T_true
else:
    data.qfrc_applied[PITCH_QVEL_ID] = 0.0

if state == STATE_HOOK_SET:
    s = smoothstep((t - BITE_TIME) / HOOK_DURATION)
    target = Q_INIT + (Q_HOOK - Q_INIT) * s

    torque_cmd = 150.0 * (target - q_actual) - 15.0 * q_dot_actual -
    q_dot_cmd = 0.0

    controller.theta_cmd = q_actual
    controller.theta_dot_cmd = q_dot_actual
    controller.pos_int = 0.0

elif state == STATE_REGULATE:
    if control_enabled:
        # Admittance Controller
        torque_cmd, q_dot_cmd = controller.update(
            t=t, state=state, f_measured=T_filtered,
            theta=q_actual, theta_dot=q_dot_actual,
            tip_pos=tip, fish_pos=fish_pos, control_enabled=True
        )
    else:
        # BASELINE: Just rigidly hold the neutral angle
        target = Q_NEUTRAL
        torque_cmd = 100.0 * (target - q_actual) - 10.0 * q_dot_actu
        q_dot_cmd = 0.0

else:
    target = Q_INIT
    torque_cmd = 100.0 * (target - q_actual) - 10.0 * q_dot_actual -
    q_dot_cmd = 0.0

data.ctrl[0] = torque_cmd
mujoco.mj_step(model, data)

line_length = np.linalg.norm(fish_pos - tip)

out["t"].append(t)
out["q"].append(np.rad2deg(q_actual))
out["q_dot_cmd"].append(np.rad2deg(q_dot_cmd))
out["torque_cmd"].append(torque_cmd)
out["T_true"].append(T_true)
out["T_filtered"].append(T_filtered)
out["fish_pull"].append(fish_pull)
out["state"].append(state)
out["fish_pos"].append(fish_pos.copy())
out["rod_tip"].append(tip.copy())

```



```

        out["line_length"].append(line_length)

        if control_enabled:
            out["theta_virtual"].append(np.rad2deg(controller.theta_cmd))
        else:
            out["theta_virtual"].append(np.nan) # Append NaN if control is r

        if make_video and data.time >= t_frame:
            renderer.update_scene(data, camera=cam)
            add_visual_line(renderer, tip, fish_pos, radius=0.005)
            out["frames"].append(renderer.render().copy())
            t_frame += DT_FRAME

    for k in out:
        if k != "frames": out[k] = np.array(out[k])

    return out
print("Control and simulation setup completed")

```

Control and simulation setup completed

```

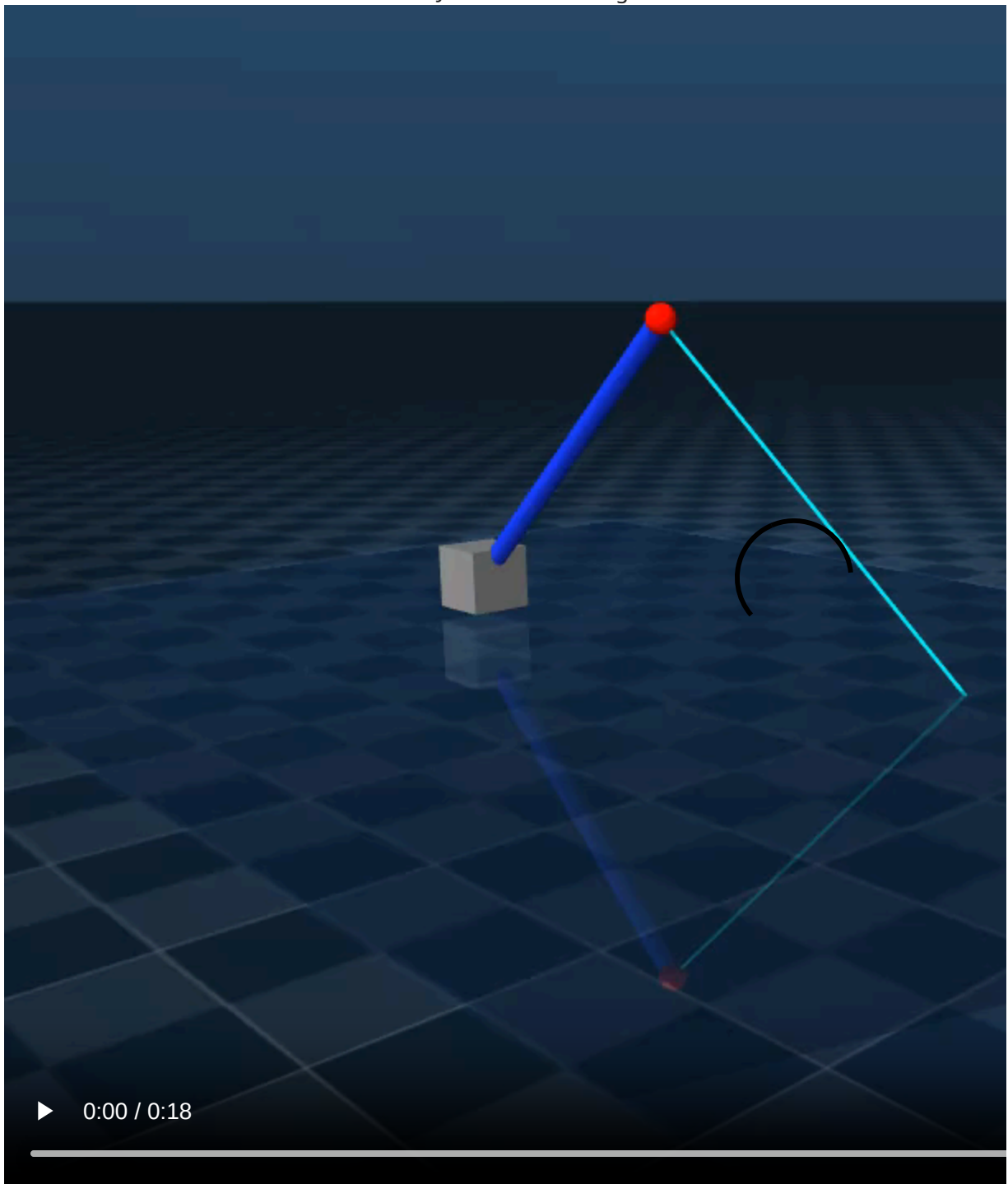
In [ ]: res_base = run_simulation(control_enabled=False, make_video=False)
        res_ctrl = run_simulation(control_enabled=True, make_video=True)

        print(f"Baseline samples: {len(res_base['t'])}")
        print(f"Controlled samples: {len(res_ctrl['t'])}")
        print(f"Controlled video frames: {len(res_ctrl['frames'])}")

        media.show_video(
            res_ctrl["frames"],
            fps=int(1 / DT_FRAME),
            title="MuJoCo 1-DOF Fishing Rod: 3D Fish Disturbance + Load-Cell Feedback"
        )

```

Baseline samples: 1800
 Controlled samples: 1800
 Controlled video frames: 451



```
In [ ]: def compute_metrics(res_base, res_ctrl):  
    reg_start = BITE_TIME + HOOK_DURATION + 1.0  
  
    mask_b = res_base["t"] > reg_start  
    mask_c = res_ctrl["t"] > reg_start  
  
    err_b = res_base["T_filtered"][mask_b] - T_DESIRED  
    err_c = res_ctrl["T_filtered"][mask_c] - T_DESIRED  
  
    rms_b = np.sqrt(np.mean(err_b**2))  
    rms_c = np.sqrt(np.mean(err_c**2))
```

```

    improvement = (1.0 - rms_c / rms_b) * 100.0

    return rms_b, rms_c, improvement, reg_start

rms_b, rms_c, improvement, reg_start = compute_metrics(res_base, res_ctrl)

print(f"Baseline RMS tension error:    {rms_b:.3f}")
print(f"Controlled RMS tension error: {rms_c:.3f}")
print(f"Error reduction:                {improvement:.1f}%")

# Increased figure height to accommodate 5 subplots
plt.figure(figsize=(12, 15))

# Tension comparison
plt.subplot(5, 1, 1)
plt.plot(res_base["t"], res_base["T_filtered"],
         label="Baseline: fixed rod angle", linewidth=2.2, alpha=0.75)
plt.plot(res_ctrl["t"], res_ctrl["T_filtered"],
         label="Controlled: admittance feedback", linewidth=2.6)
plt.axhline(T_DESIRED, linestyle="--", label="Desired tension")
plt.axhline(T_LOW, linestyle=":", label="Low bound")
plt.axhline(T_HIGH, linestyle=":", label="High bound")
plt.axvline(BITE_TIME, linestyle=":", color="gray", alpha=0.8)
plt.axvline(BITE_TIME + HOOK_DURATION, linestyle=":", color="gray", alpha=0.8)
plt.axvline(reg_start, linestyle="--", color="gray", alpha=0.7)
plt.ylabel("Tension")
plt.title("Simulated Load-Cell Tension Feedback")
plt.legend(loc="upper right")
plt.grid(True)

# Rod angle (showing just actual rod physics for baseline)
plt.subplot(5, 1, 2)
plt.plot(res_base["t"], res_base["q"],
         label="Baseline physical rod angle", linewidth=2.2, alpha=0.75)
plt.plot(res_ctrl["t"], res_ctrl["q"],
         label="Controlled physical rod angle", linewidth=2.6)
plt.axvline(BITE_TIME, linestyle=":", color="gray", alpha=0.8)
plt.axvline(BITE_TIME + HOOK_DURATION, linestyle=":", color="gray", alpha=0.8)
plt.axvline(reg_start, linestyle="--", color="gray", alpha=0.7)
plt.ylabel("Angle from vertical [deg]")
plt.title("Physical Rod Pitch Motion")
plt.legend(loc="upper right")
plt.grid(True)

# Virtual rod angle vs Physical rod angle
plt.subplot(5, 1, 3)
plt.plot(res_ctrl["t"], res_ctrl["theta_virtual"],
         label="Virtual rod angle (cmd)", linewidth=2.6, color='purple')
plt.plot(res_ctrl["t"], res_ctrl["q"],
         label="Physical rod angle (actual)", linewidth=2.2, alpha=0.75, linestyle='solid')
plt.axvline(BITE_TIME, linestyle=":", color="gray", alpha=0.8)
plt.axvline(BITE_TIME + HOOK_DURATION, linestyle=":", color="gray", alpha=0.8)
plt.axvline(reg_start, linestyle="--", color="gray", alpha=0.7)
plt.ylabel("Angle from vertical [deg]")
plt.title("Admittance Controller Virtual vs. Physical Rod Angle")

```

```

plt.legend(loc="upper right")
plt.grid(True)

# Motor Torque Command
plt.subplot(5, 1, 4)
plt.plot(res_ctrl["t"], res_ctrl["torque_cmd"],
         label="Commanded Motor Torque", color="red", linewidth=2.0)
plt.axvline(BITE_TIME, linestyle=":", color="gray", alpha=0.8)
plt.axvline(BITE_TIME + HOOK_DURATION, linestyle=":", color="gray", alpha=0.8)
plt.axvline(reg_start, linestyle="--", color="gray", alpha=0.7)
plt.ylabel("Torque [Nm]")
plt.title("Admittance Controller Output (Motor Torque)")
plt.legend(loc="upper right")
plt.grid(True)

# Regulation-phase tension error only
plt.subplot(5, 1, 5)
mask_b = res_base["t"] > reg_start
mask_c = res_ctrl["t"] > reg_start

plt.plot(res_base["t"][mask_b],
         res_base["T_filtered"][mask_b] - T_DESIRED,
         label="Baseline tension error", linewidth=2.2, alpha=0.75)
plt.plot(res_ctrl["t"][mask_c],
         res_ctrl["T_filtered"][mask_c] - T_DESIRED,
         label="Controlled tension error", linewidth=2.6)
plt.axhline(0, linestyle="--", color="gray")
plt.ylabel("Tension Error")
plt.xlabel("Time [s]")
plt.title(f"Regulation Phase RMS Error Reduction = {improvement:.1f}%")
plt.legend(loc="upper right")
plt.grid(True)

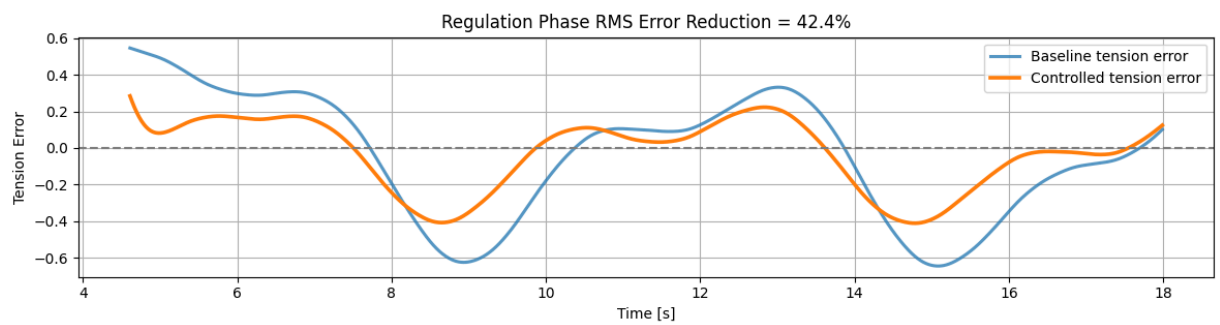
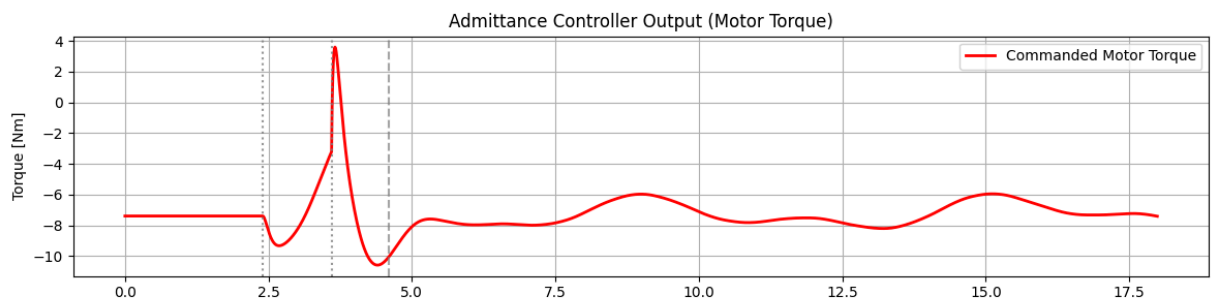
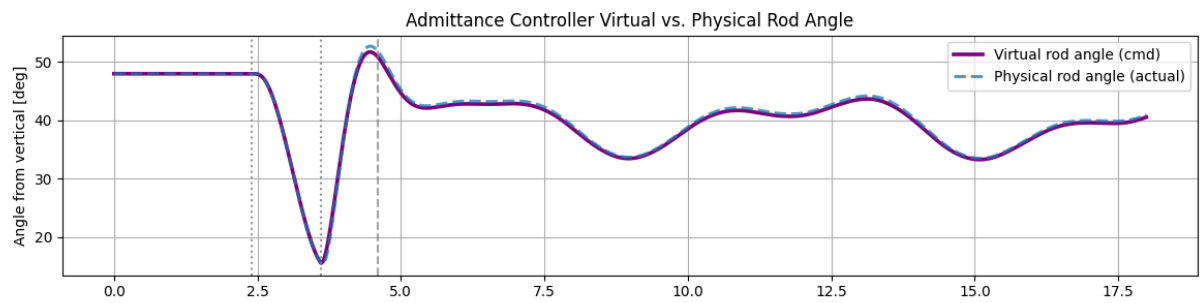
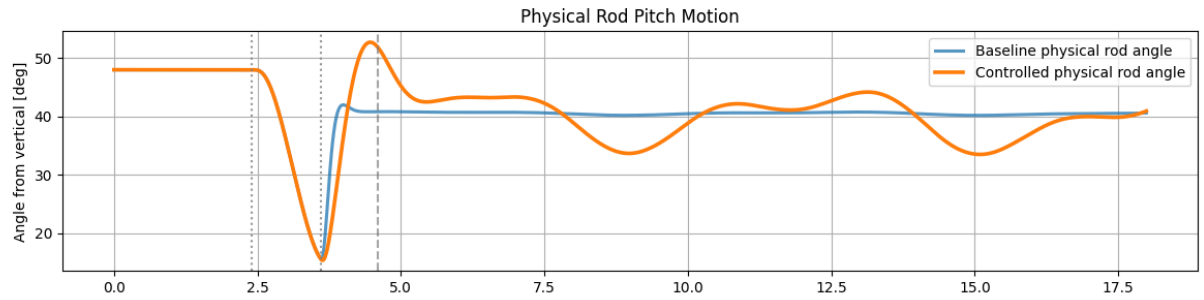
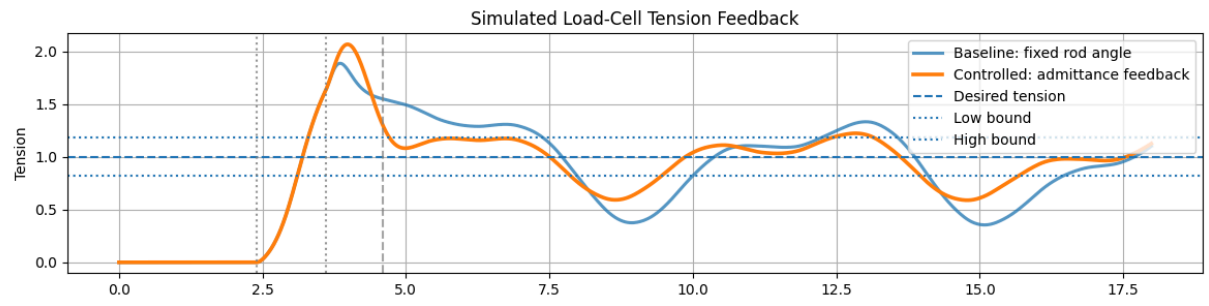
plt.tight_layout()
plt.show()

```

```

Baseline RMS tension error: 0.338
Controlled RMS tension error: 0.195
Error reduction: 42.4%

```



```
In [ ]: class PIVelocityController:
    """
    PI velocity controller.
    Maps velocity error to a PWM percentage (pwm command)
    """

    def __init__(self, K_p: float, K_i: float, max_pwm_percent: float = 100.0):
        self.K_p = float(K_p)
        self.K_i = float(K_i)
        self.max_pwm = float(max_pwm_percent)

        self.error_prev = 0.0
        self.error_integral = 0.0

    def update(self, target_vel_rad_s: float, actual_vel_rad_s: float, dt: float):
        error = target_vel_rad_s - actual_vel_rad_s

        # Tustin Discretization for the Integral Term
        #  $I[k] = I[k-1] + (K_i * dt / 2) * (e[k] + e[k-1])$ 
        self.error_integral += (self.K_i * dt / 2.0) * (error + self.error_prev)

        # Anti-windup: clamp the integral term
        self.error_integral = max(min(self.error_integral, self.max_pwm), -self.max_pwm)

        # Calculate total PWM and clamp final output
        pwm_cmd = (self.K_p * error) + self.error_integral
        pwm_cmd = max(min(pwm_cmd, self.max_pwm), -self.max_pwm)

        # Save state for the next discrete step
        self.error_prev = error

        return pwm_cmd
```

```
In [ ]: class AdmittanceFishingController:
    def __init__(self, M_v, B_v, pi_Kp, pi_Ki, desired_tension_N, L, dxl_id, control_freq_Hz, max_duration_s):
        self.M_v = float(M_v) # Virtual Mass
        self.B_v = float(B_v) # Virtual Damper
        self.desired_tension_N = float(desired_tension_N)
        self.L = float(L) # Length of Rod
        self.dxl_id = dxl_id # Dynamixel ID

        self.dt = 1.0 / float(control_freq_Hz)
        self.max_duration_s = float(max_duration_s)
        self.loop_manager = FixedFrequencyLoopManager(control_freq_Hz)

        # Tustin discretization
        # Continuous domain:  $V(s)/E(s) = 1 / (M_v*s + B_v)$ 
        # Applying Tustin transform:  $s = (2/dt) * (1 - z^{-1}) / (1 + z^{-1})$ 
        self._tustin_a = (2.0 * self.M_v / self.dt) + self.B_v
        self._tustin_b = (2.0 * self.M_v / self.dt) - self.B_v

        # State history for the discrete filter
```

```

self._e_prev = 0.0
self._v_prev = 0.0

# Data Logging
self.tension_history = []
self.velocity_cmd_history = []
self.time_stamps = []

# Instantiate the inner loop PI controller
self.pi_controller = PIVelocityController(pi_Kp, pi_Ki, max_pwm_perc

self.prev_time = time.time()

# - Sensor Reads -
def _read_load_cell(self):
    """
    Reads line tension
    """
    # TODO: READ SENSOR DO THIS IDK HOW
    tension_measured
    return tension_measured

def _read_motor_state(self):
    """
    Reads present position and velocity from the Dynamixel.
    """
    # I took this from minilab 2 mostly idk if it's how you do this
    groupSyncReadState.txRxPacket()

    v_ticks = groupSyncReadState.getData(self.dxl_id, ADDR_PRESENT_VELOC
    p_ticks = groupSyncReadState.getData(self.dxl_id, ADDR_PRESENT_POSIT

    q_rad = pos_ticks_to_rad(p_ticks)
    qdot_rad_per_s = vel_ticks_to_rad_per_s(v_ticks)

    return q_rad, qdot_rad_per_s

def _apply_pwm_percent(self, pct: float):
    # IDK if this is right or if we even need this im ngl
    """Write the scaled goal-PWM register to the motor."""
    pct = max(min(pct, 100.0), -100.0)
    pwm_raw = int(round((pct / 100.0) * PWM_FULL_SCALE))
    if pwm_raw < 0:
        pwm_raw += 1 << 16
    packetHandler.write2ByteTxRx(portHandler, self.dxl_id, ADDR_GOAL_PWM

def control_loop(self):
    print(f"Starting Cascaded Admittance Loop. Target: {self.desired_ter
    t_start = time.time()
    self.prev_time = t_start

    while True:
        now = time.time()
        dt = now - self.prev_time if now > self.prev_time else self.dt
        self.prev_time = now

```

```

t = now - t_start

if t >= self.max_duration_s:
    break

# 1. Gather Sensor Feedback
q_rad, actual_vel_rad_s = self._read_motor_state()
tension_actual_N = self._read_load_cell()

# 2. Outer Loop: Admittance (Force -> Target Velocity)
force_error = self.desired_tension_N - tension_actual_N
torque_error = force_error * self.L

v_cmd = (1.0 / self._tustin_a) * (torque_error + self._e_prev +

self._e_prev = torque_error
self._v_prev = v_cmd

# 3. Inner Loop: PI Control (Velocity Error -> PWM Command)
pwm_cmd = self.pi_controller.update(
    target_vel_rad_s=v_cmd,
    actual_vel_rad_s=actual_vel_rad_s,
    dt=dt
)

# 4. Command Actuator
self._apply_pwm_percent(pwm_cmd)

self.loop_manager.sleep()

```



```
In [ ]: !pip install -q pyserial dynamixel-sdk numpy matplotlib
```

```
In [ ]: import time
import math
import re
from collections import deque

import serial
import numpy as np
import matplotlib.pyplot as plt

from dynamixel_sdk import *

print("Imports OK")
```

```
In [ ]: ARDUINO_PORT = "COM3"
ARDUINO_BAUD = 9600
ARDUINO_TIMEOUT_S = 0.05

SENSOR_PATTERN = re.compile(r"raw:\s*(-?\d+)")

SENSITIVITY_N_PER_COUNT = 4.8e-5
SENSOR_SIGN = 1.0

FILTER_WINDOW = 2
TARE_SAMPLES = 20

DXL_ID = 1
DXL_PORT = "COM4"
DXL_BAUD = 57600
PROTOCOL_VERSION = 2.0

ADDR_OPERATING_MODE = 11
ADDR_TORQUE_ENABLE = 64
ADDR_GOAL_PWM = 100
ADDR_PRESENT_VELOCITY = 128
ADDR_PRESENT_POSITION = 132
ADDR_PRESENT_INPUT_VOLTAGE = 144
ADDR_PWM_LIMIT = 36

PWM_CONTROL_MODE = 16
TORQUE_ENABLE = 1
TORQUE_DISABLE = 0

PWM_FULL_SCALE = 885

MAX_PWM = 300

MIN_EFFECTIVE_PWM_FORWARD = 25
MIN_EFFECTIVE_PWM_BACKWARD = 65

MIN_POS_TICKS = 2300
MAX_POS_TICKS = 3500
```

```

USE_POSITION_COMP = True
CENTER_POS_TICKS = 3000
KG_POS = 0.08

TARGET_TENSION_N = 3.5
TENSION_DEADBAND_N = 0.10

CONTROL_FREQ_HZ = 20.0
MAX_DURATION_S = 20.0

LOW_TENSION_POS_GAIN_TICKS_PER_N = 400.0
HIGH_TENSION_POS_GAIN_TICKS_PER_N = 160.0

MAX_TARGET_OFFSET_TICKS = 240
TARGET_ALPHA = 0.30
MAX_TARGET_STEP_TICKS = 30

RESET_TARGET_ON_REGION_REVERSAL = True

KP_POS_TO_PWM = 0.70
KD_VEL_TO_PWM = 30.0

POSITION_ERROR_DEADBAND_TICKS = 12
MIN_PWM_ENABLE_ERROR_TICKS = 35

MAX_PWM_STEP = 30

HIGH_TENSION_RELEASE_PWM_GAIN = 16.0
HIGH_TENSION_RELEASE_PWM_MAX = 45
HIGH_TENSION_RELEASE_MIN_PWM = 15

BITE_TRIGGER_TENSION_N = 0.8
BITE_CONFIRM_TIME_S = 0.10

LOW_TENSION_RECOVERY_N = 1.0
TENSION_RECOVERED_N = 2.0
LOW_TENSION_TIMEOUT_S = 1.8
SOFT_BACK_LIMIT_TICKS = 2600

print("Config loaded")

```

```

In [ ]: class FixedFrequencyLoopManager:
    def __init__(self, frequency_Hz):
        self.period_s = 1.0 / float(frequency_Hz)
        self._next_tick = None

    def sleep(self):
        now = time.time()

        if self._next_tick is None:
            self._next_tick = now + self.period_s
            return

        sleep_time = self._next_tick - now

        if sleep_time > 0:

```

```

        time.sleep(sleep_time)

        self._next_tick += self.period_s

def parse_sensor_line(line):
    match = SENSOR_PATTERN.search(line)

    if not match:
        return None

    return int(match.group(1))

def signed_velocity_to_rad_s(vel_raw):
    if vel_raw >= (1 << 31):
        vel_raw -= (1 << 32)

    return vel_raw * 0.229 * 2.0 * math.pi / 60.0

def clip_pwm(pwm):
    return int(max(-MAX_PWM, min(MAX_PWM, pwm)))

def apply_min_effective_pwm(pwm):
    pwm = int(pwm)

    if pwm > 0:
        return max(pwm, MIN_EFFECTIVE_PWM_FORWARD)

    if pwm < 0:
        return min(pwm, -MIN_EFFECTIVE_PWM_BACKWARD)

    return 0

def position_comp_pwm(pos):
    if not USE_POSITION_COMP:
        return 0

    comp = KG_POS * (CENTER_POS_TICKS - pos)
    return int(round(comp))

def signed_pwm_to_unsigned(pwm):
    pwm = int(pwm)

    if pwm < 0:
        return pwm + (1 << 16)

    return pwm

print("Helpers ready")

```

```

In [ ]: class LoadCellSerialReader:
    def __init__(
        self,
        port,
        baud,
        timeout_s=0.05,
        filter_window=2,
        tare_samples=20,
        sensitivity_N_per_count=4.8e-5
    ):
        self.port = port
        self.baud = baud
        self.timeout_s = timeout_s
        self.filter_window = filter_window
        self.tare_samples = tare_samples
        self.sensitivity = sensitivity_N_per_count

        self.ser = None
        self.buffer = deque(maxlen=filter_window)
        self.zero_offset = None

    def open(self):
        self.ser = serial.Serial(
            self.port,
            self.baud,
            timeout=self.timeout_s
        )

        time.sleep(2.0)
        self.ser.reset_input_buffer()

        print(f"Arduino connected on {self.port}")
        print("Taring load cell. Do not pull the line...")

        self.zero_offset = self.tare()

        print(f"Tare complete. zero_offset = {self.zero_offset:.1f}")

    def read_raw_once(self):
        while True:
            line = self.ser.readline().decode("utf-8", errors="ignore").stri

            if not line:
                continue

            raw = parse_sensor_line(line)

            if raw is not None:
                return raw, line

    def tare(self):
        samples = []

        while len(samples) < self.tare_samples:
            raw, _ = self.read_raw_once()

```

```

        samples.append(raw)

    return float(np.mean(samples))

def raw_to_tension(self, filtered_raw):
    return SENSOR_SIGN * self.sensitivity * (filtered_raw - self.zero_of

def read(self):
    line = self.ser.readline().decode("utf-8", errors="ignore").strip()

    if not line:
        return None, None, None, line

    raw = parse_sensor_line(line)

    if raw is None:
        return None, None, None, line

    self.buffer.append(raw)

    filtered_raw = float(np.mean(self.buffer))
    tension_N = self.raw_to_tension(filtered_raw)

    return raw, filtered_raw, tension_N, line

def close(self):
    if self.ser is not None:
        self.ser.close()
        print("Arduino serial closed")

```

```

In [ ]: class DynamixelPWMInterface:
    def __init__(self, dxl_id, port, baud):
        self.dxl_id = dxl_id
        self.port = port
        self.baud = baud

        self.portHandler = PortHandler(self.port)
        self.packetHandler = PacketHandler(PROTOCOL_VERSION)

    def open(self):
        if not self.portHandler.openPort():
            raise RuntimeError(f"Failed to open Dynamixel port {self.port}")

        if not self.portHandler.setBaudRate(self.baud):
            raise RuntimeError(f"Failed to set Dynamixel baudrate {self.baud}

        print(f"Dynamixel port opened: {self.port}")

        _, comm_result, _ = self.packetHandler.ping(
            self.portHandler,
            self.dxl_id
        )

        if comm_result != COMM_SUCCESS:
            raise RuntimeError(f"Failed to ping Dynamixel ID {self.dxl_id}")

```

```

print(f"Dynamixel ID {self.dxl_id} connected")

def set_pwm_limit(self, limit=885):
    self.packetHandler.write1ByteTxRx(
        self.portHandler,
        self.dxl_id,
        ADDR_TORQUE_ENABLE,
        TORQUE_DISABLE
    )

    comm, err = self.packetHandler.write2ByteTxRx(
        self.portHandler,
        self.dxl_id,
        ADDR_PWM_LIMIT,
        int(limit)
    )

    pwm_limit, _, _ = self.packetHandler.read2ByteTxRx(
        self.portHandler,
        self.dxl_id,
        ADDR_PWM_LIMIT
    )

    print("set PWM limit:", pwm_limit, "comm:", comm, "err:", err)

def enable_pwm_mode(self):
    self.packetHandler.write1ByteTxRx(
        self.portHandler,
        self.dxl_id,
        ADDR_TORQUE_ENABLE,
        TORQUE_DISABLE
    )

    self.packetHandler.write1ByteTxRx(
        self.portHandler,
        self.dxl_id,
        ADDR_OPERATING_MODE,
        PWM_CONTROL_MODE
    )

    self.packetHandler.write1ByteTxRx(
        self.portHandler,
        self.dxl_id,
        ADDR_TORQUE_ENABLE,
        TORQUE_ENABLE
    )

    self.write_pwm(0)
    time.sleep(0.1)

    mode, _, _ = self.packetHandler.read1ByteTxRx(
        self.portHandler,
        self.dxl_id,
        ADDR_OPERATING_MODE
    )

```

```

torque, _, _ = self.packetHandler.read1ByteTxRx(
    self.portHandler,
    self.dxl_id,
    ADDR_TORQUE_ENABLE
)

voltage_raw, _, _ = self.packetHandler.read2ByteTxRx(
    self.portHandler,
    self.dxl_id,
    ADDR_PRESENT_INPUT_VOLTAGE
)

print("PWM mode enabled")
print("mode:", mode)
print("torque:", torque)
print("input voltage:", voltage_raw / 10.0, "V")

def write_pwm(self, pwm):
    pwm = clip_pwm(pwm)
    pwm_unsigned = signed_pwm_to_unsigned(pwm)

    comm, err = self.packetHandler.write2ByteTxRx(
        self.portHandler,
        self.dxl_id,
        ADDR_GOAL_PWM,
        pwm_unsigned
    )

    return comm, err

def read_velocity_rad_s(self):
    vel_raw, _, _ = self.packetHandler.read4ByteTxRx(
        self.portHandler,
        self.dxl_id,
        ADDR_PRESENT_VELOCITY
    )

    return signed_velocity_to_rad_s(vel_raw)

def read_position_ticks(self):
    pos, _, _ = self.packetHandler.read4ByteTxRx(
        self.portHandler,
        self.dxl_id,
        ADDR_PRESENT_POSITION
    )

    return pos

def stop(self):
    try:
        self.write_pwm(0)
        time.sleep(0.1)

        self.packetHandler.write1ByteTxRx(
            self.portHandler,
            self.dxl_id,

```

```

        ADDR_TORQUE_ENABLE,
        TORQUE_DISABLE
    )

    except Exception:
        pass

    print("Dynamixel stopped")

def close(self):
    self.portHandler.closePort()
    print("Dynamixel port closed")

```

```

In [ ]: class VirtualEquilibriumFishingController:
    def __init__(
        self,
        sensor_reader,
        motor_interface,
        desired_tension_N,
        control_freq_Hz,
        max_duration_s
    ):
        self.sensor = sensor_reader
        self.motor = motor_interface

        self.desired_tension_N = float(desired_tension_N)
        self.dt_nominal = 1.0 / float(control_freq_Hz)

        self.max_duration_s = float(max_duration_s)
        self.loop_manager = FixedFrequencyLoopManager(control_freq_Hz)

        self.prev_time = None
        self.prev_tension_N = None
        self.prev_pwm_raw = 0

        self.virtual_target_pos = None

        self.control_enabled = False
        self.bite_timer_s = 0.0
        self.low_tension_timer_s = 0.0

        self.prev_region = "IDLE"

        self.time_history = []
        self.raw_history = []
        self.filtered_raw_history = []
        self.position_history = []
        self.virtual_target_history = []
        self.position_error_history = []
        self.tension_history = []
        self.raw_tension_history = []
        self.tension_error_history = []
        self.tension_rate_history = []
        self.controller_pwm_history = []
        self.release_ff_pwm_history = []
        self.pos_comp_pwm_history = []

```



```

self.pwm_history = []
self.region_history = []
self.supervisor_state_history = []
self.low_tension_timer_history = []

def apply_position_limits(self, pwm_raw, pos):
    limited = False
    reason = ""

    if pos <= MIN_POS_TICKS and pwm_raw < 0:
        pwm_raw = 0
        limited = True
        reason = "BACK_LIMIT"

    if pos >= MAX_POS_TICKS and pwm_raw > 0:
        pwm_raw = 0
        limited = True
        reason = "FORWARD_LIMIT"

    return pwm_raw, limited, reason

def slew_limit_pwm(self, pwm_raw):
    lower = self.prev_pwm_raw - MAX_PWM_STEP
    upper = self.prev_pwm_raw + MAX_PWM_STEP

    pwm_limited = int(round(max(lower, min(upper, pwm_raw))))

    return pwm_limited

def disable_control(self, current_pos):
    self.control_enabled = False
    self.bite_timer_s = 0.0
    self.low_tension_timer_s = 0.0
    self.virtual_target_pos = float(current_pos)
    self.prev_pwm_raw = 0
    self.prev_region = "IDLE"

def update_engagement_supervisor(self, tension_N, dt, pos):
    if not self.control_enabled:
        self.low_tension_timer_s = 0.0

        if tension_N > BITE_TRIGGER_TENSION_N:
            self.bite_timer_s += dt
        else:
            self.bite_timer_s = 0.0

        if self.bite_timer_s >= BITE_CONFIRM_TIME_S:
            self.control_enabled = True
            self.bite_timer_s = 0.0
            self.low_tension_timer_s = 0.0
            self.virtual_target_pos = float(pos)
            self.prev_pwm_raw = 0
            self.prev_region = "IDLE"
            return True

    self.virtual_target_pos = float(pos)

```

```

        self.prev_pwm_raw = 0
        return False

    if tension_N < LOW_TENSION_RECOVERY_N:
        self.low_tension_timer_s += dt
    elif tension_N > TENSION_RECOVERED_N:
        self.low_tension_timer_s = 0.0
    else:
        self.low_tension_timer_s = max(
            0.0,
            self.low_tension_timer_s - dt
        )

    low_tension_timeout = (
        tension_N < BITE_TRIGGER_TENSION_N
        and self.low_tension_timer_s >= LOW_TENSION_TIMEOUT_S
    )

    soft_back_limit_reached = (
        tension_N < LOW_TENSION_RECOVERY_N
        and pos <= SOFT_BACK_LIMIT_TICKS
    )

    if low_tension_timeout or soft_back_limit_reached:
        self.disable_control(pos)
        return False

    return True

def compute_raw_virtual_target(self, tension_error_N, current_pos):
    if tension_error_N > TENSION_DEADBAND_N:
        region = "HIGH"
        effective_error = tension_error_N - TENSION_DEADBAND_N
        offset = HIGH_TENSION_POS_GAIN_TICKS_PER_N * effective_error

    elif tension_error_N < -TENSION_DEADBAND_N:
        region = "LOW"
        effective_error = tension_error_N + TENSION_DEADBAND_N
        offset = LOW_TENSION_POS_GAIN_TICKS_PER_N * effective_error

    else:
        region = "DB"
        offset = 0.0

    offset = max(
        -MAX_TARGET_OFFSET_TICKS,
        min(MAX_TARGET_OFFSET_TICKS, offset)
    )

    if region == "DB":
        target_raw = float(current_pos)
    else:
        target_raw = CENTER_POS_TICKS + offset

    target_raw = max(
        MIN_POS_TICKS,

```

```

        min(MAX_POS_TICKS, target_raw)
    )

    return target_raw, region

def reset_virtual_target_on_region_reversal(self, region, current_pos):
    if not RESET_TARGET_ON_REGION_REVERSAL:
        return

    low_to_high = self.prev_region == "LOW" and region == "HIGH"
    high_to_low = self.prev_region == "HIGH" and region == "LOW"

    if low_to_high or high_to_low:
        self.virtual_target_pos = float(current_pos)
        self.prev_pwm_raw = 0

def update_virtual_target(self, target_raw, region, current_pos):
    if self.virtual_target_pos is None:
        self.virtual_target_pos = float(current_pos)

    if region == "DB":
        self.virtual_target_pos = float(current_pos)
        return self.virtual_target_pos

    target_smooth = (
        (1.0 - TARGET_ALPHA) * self.virtual_target_pos
        + TARGET_ALPHA * target_raw
    )

    lower = self.virtual_target_pos - MAX_TARGET_STEP_TICKS
    upper = self.virtual_target_pos + MAX_TARGET_STEP_TICKS

    target_limited = max(lower, min(upper, target_smooth))

    self.virtual_target_pos = float(target_limited)

    return self.virtual_target_pos

def position_error_to_pwm(self, position_error_ticks, motor_vel_rad_s, r
    if region == "DB":
        return 0

    if abs(position_error_ticks) < POSITION_ERROR_DEADBAND_TICKS:
        return 0

    pwm = (
        KP_POS_TO_PWM * position_error_ticks
        - KD_VEL_TO_PWM * motor_vel_rad_s
    )

    controller_pwm = int(round(pwm))
    controller_pwm = clip_pwm(controller_pwm)

    if region == "HIGH" and controller_pwm < 0:
        controller_pwm = 0

```

```

    if region == "LOW" and controller_pwm > 0:
        controller_pwm = 0

    if abs(position_error_ticks) > MIN_PWM_ENABLE_ERROR_TICKS:
        if abs(controller_pwm) > 0:
            controller_pwm = apply_min_effective_pwm(controller_pwm)

    return controller_pwm

def high_tension_release_feedforward_pwm(self, tension_error_N, region):
    if region != "HIGH":
        return 0

    over_tension = max(0.0, tension_error_N - TENSION_DEADBAND_N)

    release_pwm = int(round(
        HIGH_TENSION_RELEASE_PWM_GAIN * over_tension
    ))

    release_pwm = min(HIGH_TENSION_RELEASE_PWM_MAX, release_pwm)

    if release_pwm > 0:
        release_pwm = max(HIGH_TENSION_RELEASE_MIN_PWM, release_pwm)

    return release_pwm

def compute_final_pwm(self, controller_pwm, pos_comp, region):
    if region == "DB":
        pwm_raw = clip_pwm(pos_comp)
        return pwm_raw

    pwm_raw = controller_pwm + pos_comp
    pwm_raw = clip_pwm(pwm_raw)

    if region == "HIGH" and pwm_raw < 0:
        pwm_raw = 0
        if self.prev_pwm_raw < 0:
            self.prev_pwm_raw = 0

    if region == "LOW" and pwm_raw > 0:
        pwm_raw = 0
        if self.prev_pwm_raw > 0:
            self.prev_pwm_raw = 0

    if self.prev_pwm_raw * pwm_raw < 0:
        self.prev_pwm_raw = 0

    pwm_raw = self.slew_limit_pwm(pwm_raw)

    if region == "HIGH" and pwm_raw < 0:
        pwm_raw = 0

    if region == "LOW" and pwm_raw > 0:
        pwm_raw = 0

    return pwm_raw

```

```

def run_idle_output(self, pos):
    supervisor_state = "IDLE"
    region = "IDLE"

    target_pos = float(pos)
    position_error_ticks = 0.0
    controller_pwm = 0
    release_ff_pwm = 0
    pos_comp = 0
    pwm_raw = 0

    self.virtual_target_pos = float(pos)
    self.prev_pwm_raw = 0

    return (
        supervisor_state,
        region,
        target_pos,
        position_error_ticks,
        controller_pwm,
        release_ff_pwm,
        pos_comp,
        pwm_raw
    )

def run_control_output(self, tension_N, pos, motor_vel):
    if self.low_tension_timer_s > 0.0 and tension_N < LOW_TENSION_RECOVER:
        supervisor_state = "RECOVERY"
    else:
        supervisor_state = "CONTROL"

    tension_error_N = tension_N - self.desired_tension_N

    target_raw, region = self.compute_raw_virtual_target(
        tension_error_N=tension_error_N,
        current_pos=pos
    )

    self.reset_virtual_target_on_region_reversal(
        region=region,
        current_pos=pos
    )

    target_pos = self.update_virtual_target(
        target_raw=target_raw,
        region=region,
        current_pos=pos
    )

    position_error_ticks = target_pos - pos

    controller_pwm = self.position_error_to_pwm(
        position_error_ticks=position_error_ticks,
        motor_vel_rad_s=motor_vel,
        region=region
    )

```

```

    )

    release_ff_pwm = self.high_tension_release_feedforward_pwm(
        tension_error_N=tension_error_N,
        region=region
    )

    if region == "HIGH":
        controller_pwm = max(controller_pwm, release_ff_pwm)

    pos_comp = position_comp_pwm(pos)

    if region == "HIGH" and controller_pwm > 0 and pos_comp < 0:
        pos_comp = 0

    pwm_raw = self.compute_final_pwm(
        controller_pwm=controller_pwm,
        pos_comp=pos_comp,
        region=region
    )

    return (
        supervisor_state,
        region,
        target_pos,
        position_error_ticks,
        controller_pwm,
        release_ff_pwm,
        pos_comp,
        pwm_raw
    )

def run(self):
    print("Starting virtual-equilibrium fishing controller with engagement")
    print(f"Target tension: {self.desired_tension_N:.2f} N")
    print(f"Bite trigger: {BITE_TRIGGER_TENSION_N:.2f} N for {BITE_CONFIDENCE_N:.2f} s")
    print(f"Low-tension recovery threshold: {LOW_TENSION_RECOVERY_N:.2f} N")
    print(f"Recovered tension threshold: {TENSION_RECOVERED_N:.2f} N")
    print(f"Low-tension timeout: {LOW_TENSION_TIMEOUT_S:.2f} s")
    print(f"Soft back limit: {SOFT_BACK_LIMIT_TICKS} ticks")
    print(f"Deadband: +/- {TENSION_DEADBAND_N:.2f} N")
    print(f"Low-tension gain: {LOW_TENSION_POS_GAIN_TICKS_PER_N:.1f} ticks/N")
    print(f"High-tension gain: {HIGH_TENSION_POS_GAIN_TICKS_PER_N:.1f} ticks/N")
    print(f"High-tension release gain: {HIGH_TENSION_RELEASE_PWM_GAIN:.1f}")
    print(f"High-tension release max: {HIGH_TENSION_RELEASE_PWM_MAX}")
    print(f"High-tension release min: {HIGH_TENSION_RELEASE_MIN_PWM}")
    print(f"Max target offset: {MAX_TARGET_OFFSET_TICKS} ticks")
    print(f"Target alpha: {TARGET_ALPHA:.2f}")
    print(f"Max target step: {MAX_TARGET_STEP_TICKS} ticks")
    print(f"Reset target on reversal: {RESET_TARGET_ON_REGION_REVERSAL}")
    print(f"Safe position range: {MIN_POS_TICKS} to {MAX_POS_TICKS}")
    print(f"Position compensation: {USE_POSITION_COMP}, center={CENTER_POS_TICKS}")
    print("Press Ctrl+C to stop\n")

    t_start = time.time()
    self.prev_time = t_start

```

```

while True:
    now = time.time()
    t = now - t_start
    dt = max(now - self.prev_time, 1e-6)
    self.prev_time = now

    if t > self.max_duration_s:
        print("Max duration reached")
        self.motor.write_pwm(0)
        break

    raw, filtered_raw, tension_raw_N, line = self.sensor.read()

    if tension_raw_N is None:
        if line:
            print(f"Skipped sensor line: {line}")
            self.loop_manager.sleep()
            continue

    tension_N = max(0.0, tension_raw_N)

    if self.prev_tension_N is None:
        tension_rate = 0.0
    else:
        tension_rate = (tension_N - self.prev_tension_N) / dt

    self.prev_tension_N = tension_N

    pos = self.motor.read_position_ticks()
    motor_vel = self.motor.read_velocity_rad_s()

    engaged = self.update_engagement_supervisor(
        tension_N=tension_N,
        dt=dt,
        pos=pos
    )

    if engaged:
        (
            supervisor_state,
            region,
            target_pos,
            position_error_ticks,
            controller_pwm,
            release_ff_pwm,
            pos_comp,
            pwm_raw
        ) = self.run_control_output(
            tension_N=tension_N,
            pos=pos,
            motor_vel=motor_vel
        )
    else:
        (
            supervisor_state,

```

```

        region,
        target_pos,
        position_error_ticks,
        controller_pwm,
        release_ff_pwm,
        pos_comp,
        pwm_raw
    ) = self.run_idle_output(pos)

pwm_raw, limited, limit_reason = self.apply_position_limits(
    pwm_raw,
    pos
)

self.prev_pwm_raw = pwm_raw

comm, err = self.motor.write_pwm(pwm_raw)

tension_error_N = tension_N - self.desired_tension_N

self.time_history.append(t)
self.raw_history.append(raw)
self.filtered_raw_history.append(filtered_raw)
self.position_history.append(pos)
self.virtual_target_history.append(target_pos)
self.position_error_history.append(position_error_ticks)
self.tension_history.append(tension_N)
self.raw_tension_history.append(tension_raw_N)
self.tension_error_history.append(tension_error_N)
self.tension_rate_history.append(tension_rate)
self.controller_pwm_history.append(controller_pwm)
self.release_ff_pwm_history.append(release_ff_pwm)
self.pos_comp_pwm_history.append(pos_comp)
self.pwm_history.append(pwm_raw)
self.region_history.append(region)
self.supervisor_state_history.append(supervisor_state)
self.low_tension_timer_history.append(self.low_tension_timer_s)

self.prev_region = region

limit_msg = f" | {limit_reason}" if limited else ""

print(
    f"t={t:.2f}s | "
    f"sup={supervisor_state} | "
    f"region={region} | "
    f"pos={pos} | "
    f"target={target_pos:.1f} | "
    f"posErr={position_error_ticks:.1f} | "
    f"raw={raw} | "
    f"filtered={filtered_raw:.1f} | "
    f"zero={self.sensor.zero_offset:.1f} | "
    f"tension={tension_N:.3f} N | "
    f"rawT={tension_raw_N:.3f} N | "
    f"terr={tension_error_N:.3f} N | "
    f"dT={tension_rate:.2f} N/s | "

```



```

        f"lowTimer={self.low_tension_timer_s:.2f}s | "
        f"v_motor={motor_vel:.3f} | "
        f"releaseFF={release_ff_pwm} | "
        f"ctrlPWM={controller_pwm} | "
        f"posComp={pos_comp} | "
        f"PWM={pwm_raw} | "
        f"dxl_err={err}"
        f"{limit_msg}"
    )

    self.loop_manager.sleep()

```

```

In [ ]: sensor = LoadCellSerialReader(
    port=ARDUINO_PORT,
    baud=ARDUINO_BAUD,
    timeout_s=ARDUINO_TIMEOUT_S,
    filter_window=FILTER_WINDOW,
    tare_samples=TARE_SAMPLES,
    sensitivity_N_per_count=SENSITIVITY_N_PER_COUNT
)

motor = DynamixelPWMInterface(
    dxl_id=DXL_ID,
    port=DXL_PORT,
    baud=DXL_BAUD
)

controller = None

try:
    sensor.open()

    motor.open()
    motor.set_pwm_limit(885)
    motor.enable_pwm_mode()

    controller = VirtualEquilibriumFishingController(
        sensor_reader=sensor,
        motor_interface=motor,
        desired_tension_N=TARGET_TENSION_N,
        control_freq_Hz=CONTROL_FREQ_HZ,
        max_duration_s=MAX_DURATION_S
    )

    controller.run()

except KeyboardInterrupt:
    print("\nStopped by user")

finally:
    try:
        motor.stop()
        motor.close()
    except Exception:
        pass

```

```

try:
    sensor.close()
except Exception:
    pass

print("Trial finished")

```

```

In [ ]: if controller is None or len(controller.time_history) == 0:
        print("No data to plot")
    else:
        t = np.asarray(controller.time_history)
        tension = np.asarray(controller.tension_history)
        raw_tension = np.asarray(controller.raw_tension_history)
        tension_error = np.asarray(controller.tension_error_history)
        tension_rate = np.asarray(controller.tension_rate_history)

        pos = np.asarray(controller.position_history)
        target = np.asarray(controller.virtual_target_history)
        pos_error = np.asarray(controller.position_error_history)

        pwm = np.asarray(controller.pwm_history)
        ctrl_pwm = np.asarray(controller.controller_pwm_history)
        release_ff = np.asarray(controller.release_ff_pwm_history)
        pos_comp = np.asarray(controller.pos_comp_pwm_history)
        low_timer = np.asarray(controller.low_tension_timer_history)

        plt.figure(figsize=(8, 4))
        plt.plot(t, tension, label="Tension")
        plt.plot(t, raw_tension, label="Raw tension", alpha=0.5)
        plt.axhline(TARGET_TENSION_N, linestyle="--", label="Target")
        plt.axhline(BITE_TRIGGER_TENSION_N, linestyle="-. ", label="Bite trigger")
        plt.axhline(LOW_TENSION_RECOVERY_N, linestyle=":", label="Recovery thresh")
        plt.axhline(TENSION_RECOVERED_N, linestyle=":", label="Recovered thresh")
        plt.xlabel("Time [s]")
        plt.ylabel("Tension [N]")
        plt.title("Tension vs Time")
        plt.legend()
        plt.grid(True)
        plt.show()

        plt.figure(figsize=(8, 4))
        plt.plot(t, tension_error, label="Tension error")
        plt.axhline(0.0, linestyle="--", label="Zero error")
        plt.axhline(TENSION_DEADBAND_N, linestyle=":", label="Upper deadband")
        plt.axhline(-TENSION_DEADBAND_N, linestyle=":", label="Lower deadband")
        plt.xlabel("Time [s]")
        plt.ylabel("Tension error [N]")
        plt.title("Tension Error")
        plt.legend()
        plt.grid(True)
        plt.show()

        plt.figure(figsize=(8, 4))
        plt.plot(t, pos, label="Motor position")
        plt.plot(t, target, label="Virtual target position")
        plt.axhline(MIN_POS_TICKS, linestyle="--", label="Back hard limit")

```

```

plt.axhline(SOFT_BACK_LIMIT_TICKS, linestyle="-. ", label="Back soft limit")
plt.axhline(MAX_POS_TICKS, linestyle="--", label="Forward limit")
plt.axhline(CENTER_POS_TICKS, linestyle=":", label="Center")
plt.xlabel("Time [s]")
plt.ylabel("Position [ticks]")
plt.title("Position and Virtual Target")
plt.legend()
plt.grid(True)
plt.show()

```

```

plt.figure(figsize=(8, 4))
plt.plot(t, pos_error, label="Position error")
plt.axhline(0.0, linestyle="--", label="Zero error")
plt.axhline(POSITION_ERROR_DEADBAND_TICKS, linestyle=":", label="Positive error limit")
plt.axhline(-POSITION_ERROR_DEADBAND_TICKS, linestyle=":", label="Negative error limit")
plt.xlabel("Time [s]")
plt.ylabel("Position error [ticks]")
plt.title("Virtual Target Tracking Error")
plt.legend()
plt.grid(True)
plt.show()

```

```

plt.figure(figsize=(8, 4))
plt.plot(t, pwm, label="Final PWM")
plt.plot(t, ctrl_pwm, label="Controller PWM")
plt.plot(t, release_ff, label="High-tension release FF")
plt.plot(t, pos_comp, label="Position compensation PWM")
plt.xlabel("Time [s]")
plt.ylabel("PWM")
plt.title("PWM Components")
plt.legend()
plt.grid(True)
plt.show()

```

```

plt.figure(figsize=(8, 4))
plt.plot(t, low_timer, label="Low-tension timer")
plt.axhline(LOW_TENSION_TIMEOUT_S, linestyle="--", label="Timeout")
plt.xlabel("Time [s]")
plt.ylabel("Timer [s]")
plt.title("Low-Tension Recovery Timer")
plt.legend()
plt.grid(True)
plt.show()

```

```

plt.figure(figsize=(8, 4))
plt.plot(t, tension_rate, label="Tension rate")
plt.axhline(0.0, linestyle="--", label="Zero rate")
plt.xlabel("Time [s]")
plt.ylabel("dT/dt [N/s]")
plt.title("Tension Rate")
plt.legend()
plt.grid(True)
plt.show()

```

```

print("Supervisor states used:", sorted(set(controller.supervisor_state_history)))
print("Regions used:", sorted(set(controller.region_history)))

```

```

In [ ]: # =====
# Supplementary final report plots
# Cleaner selected figures for Results section
# =====

import numpy as np
import matplotlib.pyplot as plt

if controller is None or len(controller.time_history) == 0:
    print("No data to plot")
else:
    # =====
    # Time windows
    # =====
    t_min = 0.0
    t_max_main = 20.0      # tension error, PWM, region
    t_max_position = 14.0  # position response

    t = np.asarray(controller.time_history)

    mask_main = (t >= t_min) & (t <= t_max_main)
    mask_pos = (t >= t_min) & (t <= t_max_position)

    t_main = t[mask_main]
    t_pos = t[mask_pos]

    # =====
    # Unit conversion: ticks to degrees
    # CENTER_POS_TICKS is treated as 0 deg reference
    # =====
    TICKS_PER_REV = 4096.0

    def ticks_to_deg(ticks):
        return (np.asarray(ticks) - CENTER_POS_TICKS) * 360.0 / TICKS_PER_REV

    def tick_error_to_deg(error_ticks):
        return np.asarray(error_ticks) * 360.0 / TICKS_PER_REV

    # =====
    # 1. Tension Error
    # Better performance/error assessment than position error
    # =====
    tension_error = np.asarray(controller.tension_error_history)[mask_main]

    plt.figure(figsize=(10, 5))
    plt.plot(t_main, tension_error, label="Tension error", linewidth=2)
    plt.axhline(0.0, linestyle="--", label="Zero error")
    plt.axhline(TENSION_DEADBAND_N, linestyle=":", label="Upper deadband")
    plt.axhline(-TENSION_DEADBAND_N, linestyle=":", label="Lower deadband")

    plt.xlabel("Time [s]")
    plt.ylabel("Tension error [N]")
    plt.title("Tension Regulation Error")
    plt.xlim(t_min, t_max_main)
    plt.grid(True)

```

```

plt.legend(loc="best")
plt.tight_layout()
plt.savefig("report_tension_error.png", dpi=300)
plt.show()

# =====
# 2. Rod Joint Position Response in Degrees
# Use degree instead of ticks to satisfy report requirement
# =====
pos_ticks = np.asarray(controller.position_history)[mask_pos]
target_ticks = np.asarray(controller.virtual_target_history)[mask_pos]

pos_deg = ticks_to_deg(pos_ticks)
target_deg = ticks_to_deg(target_ticks)

min_pos_deg = ticks_to_deg(MIN_POS_TICKS)
soft_back_deg = ticks_to_deg(SOFT_BACK_LIMIT_TICKS)
max_pos_deg = ticks_to_deg(MAX_POS_TICKS)
center_deg = 0.0

plt.figure(figsize=(10, 5))
plt.plot(t_pos, pos_deg, label="Rod joint position", linewidth=2)
plt.plot(t_pos, target_deg, label="Virtual target", linewidth=2)

plt.axhline(min_pos_deg, linestyle="--", label="Back hard limit")
plt.axhline(soft_back_deg, linestyle="-. ", label="Back soft limit")
plt.axhline(max_pos_deg, linestyle="--", label="Forward limit")
plt.axhline(center_deg, linestyle=":", label="Center")

plt.xlabel("Time [s]")
plt.ylabel("Rod joint position [deg]")
plt.title("Rod Joint Position Response to Virtual Target")
plt.xlim(t_min, t_max_position)
plt.grid(True)
plt.legend(loc="lower right")
plt.tight_layout()
plt.savefig("report_position_response_deg.png", dpi=300)
plt.show()

# =====
# 3. Simplified Control Effort / PWM
# Cleaner than plotting every component
# =====
pwm = np.asarray(controller.pwm_history)[mask_main]
release_ff = np.asarray(controller.release_ff_pwm_history)[mask_main]
pos_comp = np.asarray(controller.pos_comp_pwm_history)[mask_main]

plt.figure(figsize=(10, 5))
plt.plot(t_main, pwm, label="Final PWM command", linewidth=2)
plt.plot(t_main, release_ff, label="High-tension release feedforward", 1
plt.plot(t_main, pos_comp, label="Position compensation", linewidth=1.5)
plt.axhline(0.0, linestyle="--", label="Zero PWM")

plt.xlabel("Time [s]")
plt.ylabel("PWM")
plt.title("Control Effort")

```

```

plt.xlim(t_min, t_max_main)
plt.grid(True)
plt.legend(loc="best")
plt.tight_layout()
plt.savefig("report_control_effort_pwm.png", dpi=300)
plt.show()

# =====
# 4. Controller Region Over Time
# Shows IDLE / LOW / DB / HIGH switching logic
# =====
region_map = {
    "IDLE": 0,
    "LOW": 1,
    "DB": 2,
    "HIGH": 3,
    "RECOVERY": 4,
}

region_history = np.asarray(controller.region_history)[mask_main]
region_num = np.asarray([region_map.get(str(r), -1) for r in region_hist

plt.figure(figsize=(10, 4))
plt.step(t_main, region_num, where="post", label="Controller region", li

plt.yticks(
    [0, 1, 2, 3, 4],
    ["IDLE", "LOW", "DB", "HIGH", "RECOVERY"]
)

plt.xlabel("Time [s]")
plt.ylabel("Controller region")
plt.title("Controller Region Over Time")
plt.xlim(t_min, t_max_main)
plt.grid(True)
plt.legend(loc="best")
plt.tight_layout()
plt.savefig("report_controller_region.png", dpi=300)
plt.show()

print("Saved report-ready supplementary figures:")
print("report_tension_error.png")
print("report_position_response_deg.png")
print("report_control_effort_pwm.png")
print("report_controller_region.png")

```

```

In [ ]: # =====
# Supervisor State Over Time
# Shows IDLE / CONTROL / RECOVERY state-machine behavior
# =====

import numpy as np
import matplotlib.pyplot as plt

if controller is None or len(controller.time_history) == 0:
    print("No data to plot")

```

```

else:
    t_min = 0.0
    t_max = 20.0

    t = np.asarray(controller.time_history)
    mask = (t >= t_min) & (t <= t_max)

    t_plot = t[mask]
    supervisor_history = np.asarray(controller.supervisor_state_history)[mas

    supervisor_map = {
        "IDLE": 0,
        "CONTROL": 1,
        "RECOVERY": 2,
    }

    supervisor_num = np.asarray([
        supervisor_map.get(str(s), -1) for s in supervisor_history
    ])

    plt.figure(figsize=(10, 4))
    plt.step(
        t_plot,
        supervisor_num,
        where="post",
        label="Supervisor state",
        linewidth=2
    )

    plt.yticks(
        [0, 1, 2],
        ["IDLE", "CONTROL", "RECOVERY"]
    )

    plt.xlabel("Time [s]")
    plt.ylabel("Supervisor state")
    plt.title("Supervisor State Over Time")
    plt.xlim(t_min, t_max)
    plt.grid(True)
    plt.legend(loc="best")
    plt.tight_layout()
    plt.savefig("report_supervisor_state.png", dpi=300)
    plt.show()

    print("Saved: report_supervisor_state.png")

```

```

In [ ]: # =====
# Paper-ready plotting cell
# Compatible with current notebook structure
# - No raw tension
# - Actual joint angle only
# - Position error also plotted in degrees
# - Saves figures + CSV using numpy only
# =====

import numpy as np

```

```

import matplotlib.pyplot as plt
import csv

if controller is None or len(controller.time_history) == 0:
    print("No data to plot. Please run the controller first.")
else:
    # =====
    # Time window
    # =====
    t_min = 0.0
    t_max = 20.0

    t_all = np.asarray(controller.time_history)
    mask = (t_all >= t_min) & (t_all <= t_max)

    t = t_all[mask]

    # =====
    # Load histories
    # =====
    tension = np.asarray(controller.tension_history)[mask]
    tension_error = np.asarray(controller.tension_error_history)[mask]

    pos_ticks = np.asarray(controller.position_history)[mask]
    target_ticks = np.asarray(controller.virtual_target_history)[mask]
    pos_error_ticks = np.asarray(controller.position_error_history)[mask]

    pwm = np.asarray(controller.pwm_history)[mask]
    ctrl_pwm = np.asarray(controller.controller_pwm_history)[mask]
    release_ff = np.asarray(controller.release_ff_pwm_history)[mask]
    pos_comp = np.asarray(controller.pos_comp_pwm_history)[mask]

    region_history = np.asarray(controller.region_history)[mask]

    if hasattr(controller, "supervisor_state_history"):
        supervisor_history = np.asarray(controller.supervisor_state_history)
    else:
        supervisor_history = None

    if hasattr(controller, "low_tension_timer_history"):
        low_timer = np.asarray(controller.low_tension_timer_history)[mask]
    else:
        low_timer = None

    # =====
    # Convert ticks to degrees
    # 4096 ticks = 360 deg
    # CENTER_POS_TICKS is treated as 0 deg
    # =====
    TICKS_PER_REV = 4096.0

    def ticks_to_deg(ticks):
        return (np.asarray(ticks) - CENTER_POS_TICKS) * 360.0 / TICKS_PER_REV

    def tick_error_to_deg(error_ticks):
        return np.asarray(error_ticks) * 360.0 / TICKS_PER_REV

```



```

joint_angle_deg = ticks_to_deg(pos_ticks)
target_angle_deg = ticks_to_deg(target_ticks)
pos_error_deg = tick_error_to_deg(pos_error_ticks)

min_angle_deg = ticks_to_deg(MIN_POS_TICKS)
soft_back_angle_deg = ticks_to_deg(SOFT_BACK_LIMIT_TICKS)
max_angle_deg = ticks_to_deg(MAX_POS_TICKS)
center_angle_deg = 0.0

pos_error_deadband_deg = POSITION_ERROR_DEADBAND_TICKS * 360.0 / TICKS_F

# =====
# Save CSV without pandas
# =====
csv_filename = "paper_trial_log.csv"

with open(csv_filename, "w", newline="") as f:
    writer = csv.writer(f)

    header = [
        "time_s",
        "tension_N",
        "tension_error_N",
        "position_ticks",
        "target_ticks",
        "joint_angle_deg",
        "target_angle_deg",
        "position_error_ticks",
        "position_error_deg",
        "final_pwm",
        "controller_pwm",
        "release_ff_pwm",
        "position_comp_pwm",
        "region",
    ]

    if supervisor_history is not None:
        header.append("supervisor_state")

    if low_timer is not None:
        header.append("low_tension_timer_s")

    writer.writerow(header)

    for i in range(len(t)):
        row = [
            t[i],
            tension[i],
            tension_error[i],
            pos_ticks[i],
            target_ticks[i],
            joint_angle_deg[i],
            target_angle_deg[i],
            pos_error_ticks[i],
            pos_error_deg[i],

```

```

        pwm[i],
        ctrl_pwm[i],
        release_ff[i],
        pos_comp[i],
        region_history[i],
    ]

    if supervisor_history is not None:
        row.append(supervisor_history[i])

    if low_timer is not None:
        row.append(low_timer[i])

    writer.writerow(row)

# =====
# 1. Tension response
# =====
plt.figure(figsize=(10, 5))
plt.plot(t, tension, label="Measured tension", linewidth=2)

plt.axhline(TARGET_TENSION_N, linestyle="--", label="Target tension")
plt.axhline(BITE_TRIGGER_TENSION_N, linestyle="-. ", label="Bite trigger")
plt.axhline(LOW_TENSION_RECOVERY_N, linestyle=":", label="Low-tension threshold")
plt.axhline(TENSION_RECOVERED_N, linestyle=":", label="Recovered threshold")

plt.xlabel("Time [s]")
plt.ylabel("Tension [N]")
plt.title("Tension Response During Hardware Trial")
plt.xlim(t_min, t_max)
plt.ylim(0, max(6.0, float(np.max(tension)) + 0.5))
plt.grid(True)
plt.legend(loc="best")
plt.tight_layout()
plt.savefig("paper_01_tension_response.png", dpi=300)
plt.show()

# =====
# 2. Tension error
# =====
plt.figure(figsize=(10, 5))
plt.plot(t, tension_error, label="Tension error", linewidth=2)

plt.axhline(0.0, linestyle="--", label="Zero error")
plt.axhline(TENSION_DEADBAND_N, linestyle=":", label="Upper deadband")
plt.axhline(-TENSION_DEADBAND_N, linestyle=":", label="Lower deadband")

plt.xlabel("Time [s]")
plt.ylabel("Tension error [N]")
plt.title("Tension Regulation Error")
plt.xlim(t_min, t_max)
plt.grid(True)
plt.legend(loc="best")
plt.tight_layout()
plt.savefig("paper_02_tension_error.png", dpi=300)
plt.show()

```

```

# =====
# 3. Actual rod joint angle only
# Do not plot virtual target in the main joint-angle figure
# =====
plt.figure(figsize=(10, 5))
plt.plot(t, joint_angle_deg, label="Rod joint angle", linewidth=2)

plt.axhline(min_angle_deg, linestyle="--", label="Back hard limit")
plt.axhline(soft_back_angle_deg, linestyle="-. ", label="Back soft limit")
plt.axhline(max_angle_deg, linestyle="--", label="Forward limit")
plt.axhline(center_angle_deg, linestyle=":", label="Center")

plt.xlabel("Time [s]")
plt.ylabel("Rod joint angle [deg]")
plt.title("Rod Joint Angle During Tension Regulation")
plt.xlim(t_min, t_max)
plt.grid(True)
plt.legend(loc="best")
plt.tight_layout()
plt.savefig("paper_03_actual_joint_angle_deg.png", dpi=300)
plt.show()

# =====
# 4. Position error in degrees
# Backup / optional figure
# =====
plt.figure(figsize=(10, 5))
plt.plot(t, pos_error_deg, label="Position error", linewidth=2)

plt.axhline(0.0, linestyle="--", label="Zero error")
plt.axhline(pos_error_deadband_deg, linestyle=":", label="Positive deadband")
plt.axhline(-pos_error_deadband_deg, linestyle=":", label="Negative deadband")

plt.xlabel("Time [s]")
plt.ylabel("Position error [deg]")
plt.title("Virtual Target Position Error")
plt.xlim(t_min, t_max)
plt.grid(True)
plt.legend(loc="best")
plt.tight_layout()
plt.savefig("paper_04_position_error_deg_backup.png", dpi=300)
plt.show()

# =====
# 5. Control effort / PWM
# =====
plt.figure(figsize=(10, 5))
plt.plot(t, pwm, label="Final PWM command", linewidth=2)
plt.plot(t, release_ff, label="High-tension release feedforward", linewidth=2)
plt.plot(t, pos_comp, label="Position compensation", linewidth=1.5)
plt.axhline(0.0, linestyle="--", label="Zero PWM")

plt.xlabel("Time [s]")
plt.ylabel("PWM")
plt.title("Control Effort During Tension Regulation")

```

```

plt.xlim(t_min, t_max)
plt.grid(True)
plt.legend(loc="best")
plt.tight_layout()
plt.savefig("paper_05_control_effort_pwm.png", dpi=300)
plt.show()

# =====
# 6. Controller region
# =====

region_map = {
    "IDLE": 0,
    "LOW": 1,
    "DB": 2,
    "HIGH": 3,
    "RECOVERY": 4,
}

region_num = np.asarray([
    region_map.get(str(r), -1) for r in region_history
])

plt.figure(figsize=(10, 4))
plt.step(t, region_num, where="post", label="Controller region", linewidth=2)

plt.yticks(
    [0, 1, 2, 3, 4],
    ["IDLE", "LOW", "DB", "HIGH", "RECOVERY"]
)

plt.xlabel("Time [s]")
plt.ylabel("Controller region")
plt.title("Controller Region Over Time")
plt.xlim(t_min, t_max)
plt.grid(True)
plt.legend(loc="best")
plt.tight_layout()
plt.savefig("paper_06_controller_region.png", dpi=300)
plt.show()

# =====
# 7. Supervisor state
# =====

if supervisor_history is not None:
    supervisor_map = {
        "IDLE": 0,
        "CONTROL": 1,
        "RECOVERY": 2,
    }

    supervisor_num = np.asarray([
        supervisor_map.get(str(s), -1) for s in supervisor_history
    ])

    plt.figure(figsize=(10, 4))
    plt.step(t, supervisor_num, where="post", label="Supervisor state",

```

```

plt.yticks(
    [0, 1, 2],
    ["IDLE", "CONTROL", "RECOVERY"]
)

plt.xlabel("Time [s]")
plt.ylabel("Supervisor state")
plt.title("Supervisor State Over Time")
plt.xlim(t_min, t_max)
plt.grid(True)
plt.legend(loc="best")
plt.tight_layout()
plt.savefig("paper_07_supervisor_state.png", dpi=300)
plt.show()

# =====
# 8. Low-tension recovery timer
# Only useful when recovery behavior occurs
# =====
if low_timer is not None:
    plt.figure(figsize=(10, 5))
    plt.plot(t, low_timer, label="Low-tension timer", linewidth=2)
    plt.axhline(LOW_TENSION_TIMEOUT_S, linestyle="--", label="Timeout th

    plt.xlabel("Time [s]")
    plt.ylabel("Timer [s]")
    plt.title("Low-Tension Recovery Timer")
    plt.xlim(t_min, t_max)
    plt.grid(True)
    plt.legend(loc="best")
    plt.tight_layout()
    plt.savefig("paper_08_low_tension_timer.png", dpi=300)
    plt.show()

print("Saved paper-ready figures:")
print("paper_01_tension_response.png")
print("paper_02_tension_error.png")
print("paper_03_actual_joint_angle_deg.png")
print("paper_04_position_error_deg_backup.png")
print("paper_05_control_effort_pwm.png")
print("paper_06_controller_region.png")
print("paper_07_supervisor_state.png")
print("paper_08_low_tension_timer.png")
print("Saved CSV:")
print(csv_filename)

```

Fishing Robot — ROS 2 Source Code

Generated 2026-06-14 07:25 UTC · git 3a3ea81 (dirty) · 60 files

Table of contents

- [software/sim/fishing-robot_sim.xml](#) (58 lines)
- [software/sim/xml-creation.py](#) (7 lines)
- [software/src/bringup/CMakeLists.txt](#) (33 lines)
- [software/src/bringup/config/controllers.yaml](#) (45 lines)
- [software/src/bringup/config/controllers_hw.yaml](#) (24 lines)
- [software/src/bringup/config/params.yaml](#) (70 lines)
- [software/src/bringup/config/params_hw.yaml](#) (15 lines)
- [software/src/bringup/launch/robot.launch.py](#) (229 lines)
- [software/src/bringup/package.xml](#) (40 lines)
- [software/src/control/control/__init__.py](#) (1 lines)
- [software/src/control/control/admittance_controller.py](#) (20 lines)
- [software/src/control/control/fishing_control_core.py](#) (294 lines)
- [software/src/control/control/fishing_controller_node.py](#) (247 lines)
- [software/src/control/control/fishing_fsm.py](#) (1 lines)
- [software/src/control/control/force_feedback_controller.py](#) (20 lines)
- [software/src/control/control/position_test_publisher.py](#) (57 lines)
- [software/src/control/launch/control.launch.py](#) (82 lines)
- [software/src/control/package.xml](#) (24 lines)
- [software/src/control/setup.cfg](#) (5 lines)
- [software/src/control/setup.py](#) (36 lines)
- [software/src/control/test/test_controller_simulation.py](#) (79 lines)
- [software/src/control/test/test_copyright.py](#) (26 lines)
- [software/src/control/test/test_flake8.py](#) (26 lines)
- [software/src/control/test/test_pep257.py](#) (24 lines)
- [software/src/description/CMakeLists.txt](#) (34 lines)
- [software/src/description/package.xml](#) (28 lines)
- [software/src/description/urdf/fishing-robot.urdf.xacro](#) (203 lines)
- [software/src/description/urdf/fishing-robot_hw.urdf](#) (114 lines)
- [software/src/description/urdf/fishing-robot_sim.urdf](#) (114 lines)
- [software/src/description/urdf/fishing-robot_sim.xml](#) (121 lines)
- [software/src/diagnostics/config/diagnostics_params.yaml](#) (34 lines)
- [software/src/diagnostics/diagnostics/__init__.py](#) (1 lines)
- [software/src/diagnostics/diagnostics/hardware_check.py](#) (189 lines)
- [software/src/diagnostics/diagnostics/recorder.py](#) (223 lines)
- [software/src/diagnostics/launch/diagnostics.launch.py](#) (23 lines)
- [software/src/diagnostics/package.xml](#) (27 lines)
- [software/src/diagnostics/scripts/motor_test.py](#) (96 lines)
- [software/src/diagnostics/scripts/render_animation.py](#) (124 lines)
- [software/src/diagnostics/scripts/sensor_test.py](#) (96 lines)
- [software/src/diagnostics/scripts/test_joint_state.py](#) (1 lines)
- [software/src/diagnostics/setup.cfg](#) (5 lines)
- [software/src/diagnostics/setup.py](#) (35 lines)
- [software/src/interfaces/CMakeLists.txt](#) (33 lines)
- [software/src/interfaces/msg/FishingTension.msg](#) (3 lines)
- [software/src/interfaces/package.xml](#) (25 lines)
- [software/src/planning/config/fish_params.yaml](#) (10 lines)
- [software/src/planning/launch/fish.launch.py](#) (23 lines)
- [software/src/planning/package.xml](#) (22 lines)
- [software/src/planning/planning/__init__.py](#) (1 lines)
- [software/src/planning/planning/fish_agent.py](#) (70 lines)
- [software/src/planning/setup.cfg](#) (5 lines)
- [software/src/planning/setup.py](#) (35 lines)
- [software/src/sensors/config/sensor_params.yaml](#) (37 lines)
- [software/src/sensors/launch/sensor.launch.py](#) (29 lines)
- [software/src/sensors/package.xml](#) (26 lines)
- [software/src/sensors/sensors/__init__.py](#) (6 lines)
- [software/src/sensors/sensors/arduino_load_cell.py](#) (94 lines)
- [software/src/sensors/sensors/load_cell_publisher.py](#) (260 lines)
- [software/src/sensors/setup.cfg](#) (5 lines)
- [software/src/sensors/setup.py](#) (35 lines)

Package: sim

software/sim/fishing-robot_sim.xml (58 lines)

```
1 <mujoco model="fishing_robot">
2   <compiler angle="radian" coordinate="local" meshdir="../meshes"/>
3
4   <asset>
5     <mesh name="base_link_collision" file="base_link_collision.STL"/>
6     <mesh name="Link_1_collision" file="Link_1_collision.STL"/>
7     <mesh name="Link_2_collision" file="Link_2_collision.STL"/>
8
9     <mesh name="base_link_visual" file="base_link.STL"/>
10    <mesh name="Link_1_visual" file="Link_1.STL"/>
11    <mesh name="Link_2_visual" file="Link_2.STL"/>
12  </asset>
13
14  <worldbody>
15    <body name="reel_anchor" pos="0 0 0">
16      <site name="reel_site" pos="0 -1.0 0" size="0.01"/>
17    </body>
18
19    <body name="base_link" pos="0 0 0">
20      <geom type="mesh" mesh="base_link_collision"/>
21      <geom type="mesh" mesh="base_link_visual" rgba="0.75 0.75 0.75 1" contype="0" conaffinity="0"/>
22
23      <body name="Link_1" pos="0 0 0.02335">
24        <inertial pos="1.98733e-05 -0.000248211 0.0252855" quat="0.997703 0.0677224 -0.00113881 0.00132904" mass="0.0497088" dia
25        <joint name="Joint_1" pos="0 0 0" axis="0 0 1" range="-3.1416 3.1416" actuatorfrange="-2.5 2.5"/>
26        <geom type="mesh" mesh="Link_1_collision"/>
27        <geom type="mesh" mesh="Link_1_visual" rgba="0.75 0.75 0.75 1" contype="0" conaffinity="0"/>
28
29        <body name="Link_2" pos="0 -0.02335 0.03875">
30          <inertial pos="-0.000376763 0.02285 0.0758976" quat="1 0 -0.000462491 0" mass="0.0374845" diaginertia="9.81279e-05 9.5
31          <joint name="Joint_2" pos="0 0 0" axis="0 1 0" range="-3.1416 3.1416" actuatorfrange="-2.5 2.5"/>
32          <geom type="mesh" mesh="Link_2_collision"/>
33          <geom type="mesh" mesh="Link_2_visual" rgba="0.75 0.75 0.75 1" contype="0" conaffinity="0"/>
34
35          <site name="rod_tip_site" pos="0 0 0.19" size="0.01"/>
36        </body>
37      </body>
38    </body>
39  </worldbody>
40
41  <!-- tendon>
42    <spatial name="fishing_line" stiffness="100" damping="1">
43      <site site="reel_site" />
44      <site site="rod_tip_site" />
45    </spatial>
46  </tendon -->
47
48  <sensor>
49    <force name="tension_sensor" site="rod_tip_site" />
50  </sensor>
51
52
53
54  <keyframe>
55    <key name="starting_pose" qpos="1.57 2.61"/>
56  </keyframe>
57
58 </mujoco>
```

software/sim/xml-creation.py (7 lines)

```
1 import mujoco
2
3 # Load the URDF file
4 model = mujoco.MjModel.from_xml_path('fishing-robot_sim.urdf')
5
6 # Save as a MuJoCo MJCF XML file
7 mujoco.mj_saveLastXML('fishing-robot_sim.xml', model)
```

Package: bringup

software/src/bringup/CMakeLists.txt (33 lines)

```
1 cmake_minimum_required(VERSION 3.8)
2 project(bringup)
3
4 if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
5     add_compile_options(-Wall -Wextra -Wpedantic)
6 endif()
7
8 # find dependencies
9 find_package(ament_cmake REQUIRED)
10 # uncomment the following section in order to fill in
11 # further dependencies manually.
12 # find_package(<dependency> REQUIRED)
13
14 if(BUILD_TESTING)
15     find_package(ament_lint_auto REQUIRED)
16     # the following line skips the linter which checks for copyrights
17     # comment the line when a copyright and license is added to all source files
18     set(ament_cmake_copyright_FOUND TRUE)
19     # the following line skips cpplint (only works in a git repo)
20     # comment the line when this package is in a git repo and when
21     # a copyright and license is added to all source files
22     set(ament_cmake_cpplint_FOUND TRUE)
23     ament_lint_auto_find_test_dependencies()
24 endif()
25
26 ament_package()
27
28 install(DIRECTORY
29     launch
30     config
31     DESTINATION share/${PROJECT_NAME})
32 )
```

software/src/bringup/config/controllers.yaml (45 lines)

```
1 controller_manager:
2   ros__parameters:
3     update_rate: 100 # Hz
4
5   joint_state_broadcaster:
6     type: joint_state_broadcaster/JointStateBroadcaster
7
8   position_trajectory_controller:
9     type: joint_trajectory_controller/JointTrajectoryController
10
11   # --- Simulation-only controllers (spawned only when use_sim:=true) ---
12   fish_effort_controller:
13     type: forward_command_controller/ForwardCommandController
14
15   tension_sensor_broadcaster:
16     type: force_torque_sensor_broadcaster/ForceTorqueSensorBroadcaster
17
18 position_trajectory_controller:
19   ros__parameters:
20     joints:
21       - Joint_1
22       - Joint_2
23     command_interfaces:
24       - position
25     state_interfaces:
26       - position
27       - velocity
28     state_publish_rate: 50.0
29     action_monitor_rate: 20.0
30     allow_partial_joints_goal: false
31
32   # The fish pulls the line via an effort command on its MuJoCo slide joint.
33   # fish_agent publishes std_msgs/Float64MultiArray on /fish_effort_controller/commands.
34   fish_effort_controller:
35     ros__parameters:
36       joints:
37         - fish_swim
38       interface_name: effort
39
```



```

40 # Publishes the rod-tip load cell as geometry_msgs/WrenchStamped on
41 # /tension_sensor_broadcaster/wrench. load_cell converts it to FishingTension.
42 tension_sensor_broadcaster:
43   ros__parameters:
44     sensor_name: tension_sensor
45     frame_id: rod_tip_link

```

software/src/bringup/config/controllers_hw.yaml (24 lines)

```

1 controller_manager:
2   ros__parameters:
3     update_rate: 100 # Hz
4
5   joint_state_broadcaster:
6     type: joint_state_broadcaster/JointStateBroadcaster
7
8   position_trajectory_controller:
9     type: joint_trajectory_controller/JointTrajectoryController
10
11 position_trajectory_controller:
12   ros__parameters:
13     joints:
14       - Joint_2
15     command_interfaces:
16       - position
17     state_interfaces:
18       - position
19       - velocity
20     state_publish_rate: 50.0
21     action_monitor_rate: 20.0
22     # Control still publishes Joint_1 hold commands; ignore the unconnected base.
23     allow_partial_joints_goal: true

```

software/src/bringup/config/params.yaml (70 lines)

```

1 # Shared fishing tension controllers (Julie admittance + Chaoyi force feedback).
2 # Select at launch: controller_type:=admittance | force_feedback | none
3
4 admittance_controller:
5   ros__parameters:
6     target_tension: 3.5
7     safety_tension_limit: 20.0
8
9     # 1DOF mode (notebook-equivalent): control pitch_joint only; lock base_joint.
10    control_dof: 1
11    pitch_joint: Joint_2
12    base_joint: Joint_1
13    joint_1_hold: 1.57 # base_joint lock angle (rad)
14
15    # Pitch setpoints (rad). Smaller = rod raised = higher tension.
16    q_init: 1.745
17    q_hook: 1.40
18    q_neutral: 1.605
19    q_min: 0.10
20    q_max: 2.20
21
22    bite_time: 2.4
23    hook_duration: 1.2
24    fsm_enabled: true
25    tension_filter_alpha: 0.055
26    control_rate_hz: 50.0
27
28    # Julie admittance (virtual mass-spring-damper)
29    m_v: 0.6
30    b_v: 5.5
31    k_v: 20.0
32    k_f: 8.0
33    kp_track: 140.0
34    kd_track: 20.0
35    ki_track: 0.0
36    k_g: 9.95
37    rod_length: 0.185
38    j_alpha: 0.05
39    pos_int_limit: 0.15
40    max_torque: 20.0
41
42 force_feedback_controller:
43   ros__parameters:
44     target_tension: 3.5
45     safety_tension_limit: 20.0

```

```

46
47     control_dof: 1
48     pitch_joint: Joint_2
49     base_joint: Joint_1
50     joint_1_hold: 1.57
51
52     q_init: 1.745
53     q_hook: 1.40
54     q_neutral: 1.605
55     q_min: 0.10
56     q_max: 2.20
57
58     bite_time: 2.4
59     hook_duration: 1.2
60     fsm_enabled: true
61     tension_filter_alpha: 0.055
62     control_rate_hz: 50.0
63
64     # Chaoyi asymmetric proportional feedback (tuned for 3.5 N target in MuJoCo)
65     tension_to_angle_gain: 0.261799 # rad/N (~15 deg/N)
66     qeq_alpha: 0.04
67     max_qeq_step: 0.00384 # rad (~0.22 deg)
68     raise_gain: 1.25
69     lower_gain: 0.80

```

software/src/bringup/config/params_hw.yaml (15 lines)

```

1 # Hardware bring-up overrides (layered on params.yaml when use_sim:=false).
2 # Skips the sim-style bite/hook timeline so tension regulation starts immediately.
3
4 admittance_controller:
5   ros__parameters:
6     fsm_enabled: false
7     bite_time: 0.0
8     hook_duration: 0.0
9
10 force_feedback_controller:
11   ros__parameters:
12     fsm_enabled: false
13     bite_time: 0.0
14     hook_duration: 0.0

```

software/src/bringup/launch/robot.launch.py (229 lines)

```

1 from launch import LaunchDescription
2 from launch.actions import (
3     DeclareLaunchArgument, IncludeLaunchDescription, OpaqueFunction,
4     RegisterEventHandler, Shutdown, TimerAction)
5 from launch.conditions import IfCondition, UnlessCondition
6 from launch.event_handlers import OnProcessExit
7 from launch.launch_description_sources import PythonLaunchDescriptionSource
8 from launch.substitutions import (
9     Command, FindExecutable, LaunchConfiguration, PathJoinSubstitution,
10    PythonExpression)
11 from launch_ros.actions import Node
12 from launch_ros.parameter_descriptions import ParameterValue
13 from launch_ros.substitutions import FindPackageShare
14
15
16 def generate_launch_description():
17     # 1. Architecture flag: the single sim-to-real switch for the whole stack.
18     use_sim = LaunchConfiguration('use_sim')
19     use_sim_arg = DeclareLaunchArgument(
20         'use_sim',
21         default_value='true',
22         description='Launch MuJoCo simulation (true) or the physical Dynamixel arm (false)')
23
24     # Run MuJoCo with or without its GUI window (sim only).
25     headless = LaunchConfiguration('headless')
26     headless_arg = DeclareLaunchArgument(
27         'headless', default_value='false',
28         description='Run MuJoCo without the GUI viewer window (sim only)')
29
30     # Optional self-imposed time limit: auto-shutdown the whole launch after N
31     # seconds. 0 (default) means run until Ctrl-C.
32     run_duration = LaunchConfiguration('run_duration')
33     run_duration_arg = DeclareLaunchArgument(
34         'run_duration', default_value='0',
35         description='Auto-shutdown the launch after N seconds (0 = until Ctrl-C)')
36

```

```

37 # Record the run: time-series plots + an isometric MuJoCo animation.
38 record = LaunchConfiguration('record')
39 record_arg = DeclareLaunchArgument(
40     'record', default_value='false',
41     description='Capture diagnostics plots + animation of the run (sim only)')
42
43 # Bring-up self-test: open-loop motor + sensor function checks. Disables the
44 # tension controller so nothing competes with the test commands.
45 hardware_check = LaunchConfiguration('hardware_check')
46 hardware_check_arg = DeclareLaunchArgument(
47     'hardware_check', default_value='false',
48     description='Run the motor/sensor bring-up self-test, then shut down')
49
50 # Tension controller selection: Julie admittance or Chaoyi force feedback.
51 controller_type = LaunchConfiguration('controller_type')
52 controller_type_arg = DeclareLaunchArgument(
53     'controller_type',
54     default_value='admittance',
55     description=(
56         'Tension controller: admittance (Julie), '
57         'force_feedback (Chaoyi), or none'
58     ),
59 )
60
61 # 2. Package shares.
62 description_pkg = FindPackageShare('description')
63 bringup_pkg = FindPackageShare('bringup')
64 control_pkg = FindPackageShare('control')
65 sensors_pkg = FindPackageShare('sensors')
66 planning_pkg = FindPackageShare('planning')
67 diagnostics_pkg = FindPackageShare('diagnostics')
68
69 # 3. Xacro -> URDF. Passing use_sim here is what swaps the <hardware> plugin
70 # (dynamixel_hardware vs mujoco_ros2_control) at preprocess time, BEFORE
71 # the controller manager ever loads it.
72 urdf_xacro = PathJoinSubstitution([description_pkg, 'urdf', 'fishing-robot.urdf.xacro'])
73 robot_description_content = Command(
74     [FindExecutable(name='xacro'), ' ', urdf_xacro,
75     ' use_sim:=', use_sim, ' headless:=', headless])
76 # Wrap as a string parameter so launch does not try to YAML-parse the URDF.
77 robot_description = {
78     'robot_description': ParameterValue(robot_description_content, value_type=str)}
79
80 controllers_yaml = PathJoinSubstitution([
81     bringup_pkg, 'config',
82     PythonExpression(
83         ["'controllers.yaml' if '", use_sim, "' == 'true' else 'controllers_hw.yaml'"]),
84 ])
85
86 # 4. TF broadcaster (runs in both modes).
87 robot_state_pub_node = Node(
88     package='robot_state_publisher',
89     executable='robot_state_publisher',
90     output='both',
91     parameters=[robot_description],
92 )
93
94 # 5a. REAL HARDWARE path: a standalone controller_manager owns the realtime
95 # loop and instantiates the dynamixel_hardware plugin named in the URDF.
96 controller_manager_node = Node(
97     package='controller_manager',
98     executable='ros2_control_node',
99     parameters=[robot_description, controllers_yaml],
100     output='both',
101     condition=UnlessCondition(use_sim),
102 )
103
104 # 5b. SIMULATION path: mujoco_ros2_control ships its OWN ros2_control_node
105 # (required for compatibility) that wraps MuJoCo's Simulate app: it opens
106 # the viewer, steps physics, and hosts the controller_manager. The MJCF
107 # path + initial keyframe are declared in the URDF hardware block, so we
108 # only pass robot_description, the controllers, and use_sim_time.
109 mujoco_node = Node(
110     package='mujoco_ros2_control',
111     executable='ros2_control_node',
112     parameters=[robot_description, controllers_yaml, {'use_sim_time': True}],
113     output='both',
114     condition=IfCondition(use_sim),
115 )
116
117 # 6. Controller spawners. Either host above exposes the manager at
118 # /controller_manager, so these are identical in both modes.
119 joint_state_broadcaster_spawner = Node(
120     package='controller_manager',
121     executable='spawner',

```

```

122     arguments=['joint_state_broadcaster', '--controller-manager', '/controller_manager'],
123 )
124 position_controller_spawner = Node(
125     package='controller_manager',
126     executable='spawner',
127     arguments=['position_trajectory_controller', '--controller-manager', '/controller_manager'],
128 )
129
130 # Sim-only controllers: the fish effort command forwarder and the load-cell
131 # FTS broadcaster. They exist only in the MuJoCo scene.
132 fish_effort_controller_spawner = Node(
133     package='controller_manager',
134     executable='spawner',
135     arguments=['fish_effort_controller', '--controller-manager', '/controller_manager'],
136     condition=IfCondition(use_sim),
137 )
138 tension_broadcaster_spawner = Node(
139     package='controller_manager',
140     executable='spawner',
141     arguments=['tension_sensor_broadcaster', '--controller-manager', '/controller_manager'],
142     condition=IfCondition(use_sim),
143 )
144
145 # 7. Delegation Pattern: each application layer comes from its own package's
146 # launch file rather than being re-declared inline here.
147 # The tension controller is suppressed during the bring-up self-test.
148 control_launch = IncludeLaunchDescription(
149     PythonLaunchDescriptionSource(
150         PathJoinSubstitution([control_pkg, 'launch', 'control.launch.py'])),
151     launch_arguments={
152         'controller_type': controller_type,
153         'use_sim': use_sim,
154     }.items(),
155     condition=UnlessCondition(hardware_check),
156 )
157 # The load cell runs in both modes; only its data source changes. On hardware
158 # it reads the HX711; in sim it converts the MuJoCo FTS wrench to tension.
159 sensor_source = PythonExpression(
160     ["'sim_fts' if '" + use_sim + "' == 'true' else 'hardware'"])
161 sensor_launch = IncludeLaunchDescription(
162     PythonLaunchDescriptionSource(
163         PathJoinSubstitution([sensors_pkg, 'launch', 'sensor.launch.py'])),
164     launch_arguments={'source': sensor_source}.items(),
165 )
166 # The virtual "fish" disturbance only runs in simulation.
167 fish_launch = IncludeLaunchDescription(
168     PythonLaunchDescriptionSource(
169         PathJoinSubstitution([planning_pkg, 'launch', 'fish.launch.py'])),
170     condition=IfCondition(use_sim),
171 )
172 # Diagnostics recorder: plots + animation, captured only when record:=true.
173 diagnostics_launch = IncludeLaunchDescription(
174     PythonLaunchDescriptionSource(
175         PathJoinSubstitution([diagnostics_pkg, 'launch', 'diagnostics.launch.py'])),
176     condition=IfCondition(record),
177 )
178
179 # Bring-up self-test node (motor + sensor function checks). Runs only when
180 # hardware_check:=true; when it exits, the whole launch shuts down.
181 diagnostics_params = PathJoinSubstitution(
182     [diagnostics_pkg, 'config', 'diagnostics_params.yaml'])
183 hardware_check_node = Node(
184     package='diagnostics',
185     executable='hardware_check',
186     name='hardware_check',
187     output='screen',
188     parameters=[diagnostics_params],
189     condition=IfCondition(hardware_check),
190 )
191 hardware_check_shutdown = RegisterEventHandler(
192     OnProcessExit(target_action=hardware_check_node, on_exit=[Shutdown()]),
193     condition=IfCondition(hardware_check),
194 )
195
196 # Resolve run_duration at launch time and, if positive, schedule a shutdown.
197 def _schedule_shutdown(context, *_args, **_kwargs):
198     seconds = float(run_duration.perform(context))
199     if seconds <= 0.0:
200         return []
201     return [TimerAction(
202         period=seconds,
203         actions=[Shutdown(reason=f'run_duration {seconds:g}s reached')]]
204
205 shutdown_timer = OpaqueFunction(function=_schedule_shutdown)
206

```

```

207     return LaunchDescription([
208         use_sim_arg,
209         headless_arg,
210         run_duration_arg,
211         record_arg,
212         hardware_check_arg,
213         controller_type_arg,
214         shutdown_timer,
215         robot_state_pub_node,
216         controller_manager_node,
217         mujoco_node,
218         joint_state_broadcaster_spawner,
219         position_controller_spawner,
220         fish_effort_controller_spawner,
221         tension_broadcaster_spawner,
222         control_launch,
223         sensor_launch,
224         fish_launch,
225         diagnostics_launch,
226         hardware_check_node,
227         hardware_check_shutdown,
228     ])

```

software/src/bringup/package.xml (40 lines)

```

1  <?xml version="1.0"?>
2  <?xml-model href="http://download.ros.org/schema/package_format3.xsd" schematypens="http://www.w3.org/2001/XMLSchema"?>
3  <package format="3">
4      <name>bringup</name>
5      <version>0.0.0</version>
6      <description>Top-level launch orchestration for the fishing arm: sim/real switch, ros2_control, and delegation to application
7      <maintainer email="nathange784@gmail.com">nzge</maintainer>
8      <license>MIT</license>
9
10     <buildtool_depend>ament_cmake</buildtool_depend>
11
12     <!-- ros2_control stack hosted/spawned by robot.launch.py -->
13     <exec_depend>controller_manager</exec_depend>
14     <exec_depend>joint_state_broadcaster</exec_depend>
15     <exec_depend>joint_trajectory_controller</exec_depend>
16     <exec_depend>forward_command_controller</exec_depend>
17     <exec_depend>force_torque_sensor_broadcaster</exec_depend>
18
19     <!-- URDF processing + TF -->
20     <exec_depend>robot_state_publisher</exec_depend>
21     <exec_depend>xacro</exec_depend>
22
23     <!-- Simulation backend (built from source; not in rosdistro, see note below) -->
24     <!-- <exec_depend>mujoco_ros2_control</exec_depend> -->
25
26     <!-- Application packages whose launch files this maestro includes -->
27     <exec_depend>control</exec_depend>
28     <exec_depend>sensors</exec_depend>
29     <exec_depend>planning</exec_depend>
30     <exec_depend>description</exec_depend>
31     <exec_depend>diagnostics</exec_depend>
32
33     <test_depend>ament_lint_auto</test_depend>
34     <test_depend>ament_lint_common</test_depend>
35
36     <export>
37         <build_type>ament_cmake</build_type>
38     </export>
39 </package>

```

Package: control

software/src/control/control/__init__.py (1 lines)

```
1
```

software/src/control/control/admittance_controller.py (20 lines)

```
1 import rclpy
2
3 from control.fishing_controller_node import AdmittanceControllerNode
4
5
6 def main(args=None):
7     rclpy.init(args=args)
8     node = AdmittanceControllerNode()
9     try:
10         rclpy.spin(node)
11     except KeyboardInterrupt:
12         pass
13     finally:
14         node.destroy_node()
15         rclpy.shutdown()
16
17
18 if __name__ == '__main__':
19     main()
```

software/src/control/control/fishing_control_core.py (294 lines)

```
1 """Pure-Python fishing tension controllers ported from the MuJoCo notebooks.
2
3 Julie notebook -> AdmittanceController (virtual mass-spring-damper + PD inner loop)
4 Chaoyi notebook -> ForceFeedbackController (asymmetric tension-to-angle mapping)
5
6 The notebooks model a single pitch DOF. In ROS2, control_dof=1 maps all q_* setpoints
7 to pitch_joint (Joint_2) while base_joint (Joint_1) is held fixed at joint_1_hold.
8 """
9 from __future__ import annotations
10
11 import math
12 from dataclasses import dataclass, field
13 from enum import IntEnum
14 from typing import Optional, Tuple
15
16
17 class FishingState(IntEnum):
18     MONITOR = 0
19     HOOK_SET = 1
20     REGULATE = 2
21
22
23 def smoothstep(s: float) -> float:
24     s = max(0.0, min(1.0, s))
25     return 3.0 * s * s - 2.0 * s * s * s
26
27
28 def clip(value: float, lo: float, hi: float) -> float:
29     return max(lo, min(hi, value))
30
31
32 @dataclass
33 class FishingControllerConfig:
34     """Shared configuration for both controller types."""
35
36     target_tension: float = 3.5
37     safety_tension_limit: float = 20.0
38
39     # 1 = notebook-equivalent pitch-only (Joint_2); 2 = both arm joints (future).
40     control_dof: int = 1
41     pitch_joint: str = 'Joint_2'
42     base_joint: str = 'Joint_1'
43     # Base yaw locked at this angle in 1DOF mode (matches MJCF starting_pose).
44     joint_1_hold: float = 1.57
45
46     # Pitch limits and setpoints (radians) – maps to pitch_joint in 1DOF mode.
```

```

47     q_init: float = 1.745
48     q_hook: float = 1.40
49     q_neutral: float = 1.605
50     q_min: float = 0.10
51     q_max: float = 2.20
52
53     # When false, skip MONITOR/HOOK_SET and regulate from t=0 (hardware bring-up).
54     fsm_enabled: bool = True
55
56     # State-machine timing (seconds from controller start).
57     bite_time: float = 2.4
58     hook_duration: float = 1.2
59
60     # Tension signal conditioning (matches notebook FILTER_ALPHA).
61     tension_filter_alpha: float = 0.055
62
63     control_rate_hz: float = 50.0
64
65
66 @dataclass
67 class AdmittanceConfig(FishingControllerConfig):
68     """Julie admittance controller parameters."""
69
70     m_v: float = 0.6
71     b_v: float = 5.5
72     k_v: float = 16.0
73     k_f: float = 5.0
74     kp_track: float = 140.0
75     kd_track: float = 20.0
76     ki_track: float = 0.0
77     k_g: float = 9.95
78     rod_length: float = 0.185
79     j_alpha: float = 0.05
80     pos_int_limit: float = 0.15
81     max_torque: float = 20.0
82
83
84 @dataclass
85 class ForceFeedbackConfig(FishingControllerConfig):
86     """Chaoyi proportional force-feedback controller parameters."""
87
88     tension_to_angle_gain: float = math.radians(42.0)
89     qeq_alpha: float = 0.055
90     max_qeq_step: float = math.radians(0.22)
91     raise_gain: float = 1.25
92     lower_gain: float = 0.80
93
94
95 @dataclass
96 class ControllerSnapshot:
97     state: FishingState = FishingState.MONITOR
98     tension_raw: float = 0.0
99     tension_filtered: float = 0.0
100     q_cmd: float = 0.0
101     q_dot_cmd: float = 0.0
102
103
104 def get_fishing_state(t: float, cfg: FishingControllerConfig) -> FishingState:
105     if not cfg.fsm_enabled:
106         return FishingState.REGULATE
107     if t < cfg.bite_time:
108         return FishingState.MONITOR
109     if t < cfg.bite_time + cfg.hook_duration:
110         return FishingState.HOOK_SET
111     return FishingState.REGULATE
112
113
114 def filter_tension(
115     raw: float, prev_filtered: float, alpha: float,
116 ) -> float:
117     return (1.0 - alpha) * prev_filtered + alpha * raw
118
119
120 class AdmittanceControllerCore:
121     """Virtual admittance outer loop from mojomocowwithJulieController.ipynb."""
122
123     def __init__(self, cfg: AdmittanceConfig, dt: float):
124         self.cfg = cfg
125         self.dt = dt
126         self.theta_cmd = cfg.q_init
127         self.theta_dot_cmd = 0.0
128         self.pos_int = 0.0
129         self.j_filtered = 0.0
130         self._tension_filtered = 0.0
131

```

```

132 @property
133 def tension_filtered(self) -> float:
134     return self._tension_filtered
135
136 def reset(self, q_actual: float) -> None:
137     self.theta_cmd = q_actual
138     self.theta_dot_cmd = 0.0
139     self.pos_int = 0.0
140     self.j_filtered = 0.0
141     self._tension_filtered = 0.0
142
143 def get_jacobian(
144     self,
145     theta: float,
146     tip_pos: Optional[Tuple[float, float, float]] = None,
147     fish_pos: Optional[Tuple[float, float, float]] = None,
148 ) -> float:
149     """Map pitch rate to line-length rate (notebook get_jacobian)."""
150     if tip_pos is not None and fish_pos is not None:
151         line_vec = (
152             fish_pos[0] - tip_pos[0],
153             fish_pos[1] - tip_pos[1],
154             fish_pos[2] - tip_pos[2],
155         )
156         distance = math.sqrt(sum(v * v for v in line_vec))
157         if distance < 1e-6:
158             return 0.0
159         u = tuple(v / distance for v in line_vec)
160         l_arm = self.cfg.rod_length
161         dx_dtheta = l_arm * math.cos(theta)
162         dz_dtheta = -l_arm * math.sin(theta)
163         return dx_dtheta * u[0] + dz_dtheta * u[2]
164
165     # Hardware fallback: approximate scalar Jacobian.
166     return self.cfg.rod_length * max(abs(math.cos(theta)), 0.05)
167
168 def update_tension(self, tension_raw: float) -> float:
169     self._tension_filtered = filter_tension(
170         tension_raw, self._tension_filtered, self.cfg.tension_filter_alpha,
171     )
172     return self._tension_filtered
173
174 def hook_set_target(self, t: float) -> float:
175     s = smoothstep((t - self.cfg.bite_time) / self.cfg.hook_duration)
176     return self.cfg.q_init + (self.cfg.q_hook - self.cfg.q_init) * s
177
178 def update(
179     self,
180     t: float,
181     state: FishingState,
182     f_measured: float,
183     theta: float,
184     theta_dot: float,
185     tip_pos: Optional[Tuple[float, float, float]] = None,
186     fish_pos: Optional[Tuple[float, float, float]] = None,
187     control_enabled: bool = True,
188 ) -> Tuple[float, float]:
189     """Return (position_cmd, velocity_cmd) for Joint_2."""
190     cfg = self.cfg
191
192     if state == FishingState.MONITOR or not control_enabled:
193         f_desired = 0.0
194     else:
195         f_desired = cfg.target_tension
196
197     force_error = f_desired - f_measured
198
199     j_val = self.get_jacobian(theta, tip_pos, fish_pos)
200     self.j_filtered += cfg.j_alpha * (j_val - self.j_filtered)
201
202     tau_force = -(self.j_filtered * force_error) * cfg.k_f
203     tau_spring = cfg.k_v * (cfg.q_neutral - self.theta_cmd)
204
205     theta_ddot_cmd = (
206         tau_force + tau_spring - cfg.b_v * self.theta_dot_cmd
207     ) / cfg.m_v
208
209     self.theta_dot_cmd += theta_ddot_cmd * self.dt
210     self.theta_cmd += self.theta_dot_cmd * self.dt
211     self.theta_cmd = clip(self.theta_cmd, cfg.q_min, cfg.q_max)
212
213     # Inner PD loop (notebook); on ROS the position_trajectory_controller
214     # tracks theta_cmd, but we keep theta_dot_cmd for diagnostics.
215     e_pos = self.theta_cmd - theta
216     e_vel = self.theta_dot_cmd - theta_dot

```



```

217         self.pos_int += e_pos * self.dt
218         self.pos_int = clip(self.pos_int, -cfg.pos_int_limit, cfg.pos_int_limit)
219
220         _torque = (
221             cfg.kp_track * e_pos
222             + cfg.kd_track * e_vel
223             + cfg.ki_track * self.pos_int
224             - cfg.k_g * math.sin(theta)
225         )
226         _torque = clip(_torque, -cfg.max_torque, cfg.max_torque)
227
228         return self.theta_cmd, self.theta_dot_cmd
229
230
231 class ForceFeedbackControllerCore:
232     """Asymmetric tension-to-angle controller from mojocowwithchaoyi'scontroller.ipynb."""
233
234     def __init__(self, cfg: ForceFeedbackConfig):
235         self.cfg = cfg
236         self.q_eq = cfg.q_init
237         self._tension_filtered = 0.0
238
239     @property
240     def tension_filtered(self) -> float:
241         return self._tension_filtered
242
243     def reset(self) -> None:
244         self.q_eq = self.cfg.q_init
245         self._tension_filtered = 0.0
246
247     def update_tension(self, tension_raw: float) -> float:
248         self._tension_filtered = filter_tension(
249             tension_raw, self._tension_filtered, self.cfg.tension_filter_alpha,
250         )
251         return self._tension_filtered
252
253     def update(
254         self,
255         t: float,
256         state: FishingState,
257         tension_filtered: float,
258         control_enabled: bool = True,
259     ) -> Tuple[float, float]:
260         """Return (q_eq, q_target) equilibrium pitch angles."""
261         cfg = self.cfg
262
263         if state == FishingState.MONITOR:
264             q_target = cfg.q_init
265         elif state == FishingState.HOOK_SET:
266             s = smoothstep((t - cfg.bite_time) / cfg.hook_duration)
267             q_target = cfg.q_init + (cfg.q_hook - cfg.q_init) * s
268         elif control_enabled:
269             tension_error = tension_filtered - cfg.target_tension
270             if tension_error < 0.0:
271                 q_target = (
272                     cfg.q_neutral
273                     + cfg.raise_gain * cfg.tension_to_angle_gain * tension_error
274                 )
275             else:
276                 q_target = (
277                     cfg.q_neutral
278                     + cfg.lower_gain * cfg.tension_to_angle_gain * tension_error
279                 )
280         else:
281             q_target = cfg.q_neutral
282
283         q_target = clip(q_target, cfg.q_min, cfg.q_max)
284
285         q_smooth = (1.0 - cfg.qeq_alpha) * self.q_eq + cfg.qeq_alpha * q_target
286         q_eq = clip(
287             q_smooth,
288             self.q_eq - cfg.max_qeq_step,
289             self.q_eq + cfg.max_qeq_step,
290         )
291         q_eq = clip(q_eq, cfg.q_min, cfg.q_max)
292         self.q_eq = q_eq
293         return q_eq, q_target

```

software/src/control/control/fishing_controller_node.py (247 lines)

```

1  """Shared ROS2 node base for fishing tension controllers."""
2  from __future__ import annotations
3

```

```

4 from typing import Optional, Tuple
5
6 import rclpy
7 from rclpy.node import Node
8 from rclpy.duration import Duration
9 from sensor_msgs.msg import JointState
10 from trajectory_msgs.msg import JointTrajectory, JointTrajectoryPoint
11 from interfaces.msg import FishingTension
12
13 from control.fishing_control_core import (
14     AdmittanceConfig,
15     AdmittanceControllerCore,
16     ControllerSnapshot,
17     FishingControllerConfig,
18     FishingState,
19     ForceFeedbackConfig,
20     ForceFeedbackControllerCore,
21     get_fishing_state,
22 )
23
24
25 def _load_fishing_config(node: Node, defaults: FishingControllerConfig) -> FishingControllerConfig:
26     """Load parameters from the node's namespace (params.yaml keyed by node name)."""
27     cfg = FishingControllerConfig()
28     for name in defaults.__dataclass_fields__:
29         default = getattr(defaults, name)
30         node.declare_parameter(name, default)
31         setattr(cfg, name, node.get_parameter(name).value)
32     return cfg
33
34
35 def load_admittance_config(node: Node) -> AdmittanceConfig:
36     base = _load_fishing_config(node, FishingControllerConfig())
37     cfg = AdmittanceConfig(**base.__dict__)
38     for name in (
39         'm_v', 'b_v', 'k_v', 'k_f', 'kp_track', 'kd_track', 'ki_track',
40         'k_g', 'rod_length', 'j_alpha', 'pos_int_limit', 'max_torque',
41     ):
42         default = getattr(cfg, name)
43         node.declare_parameter(name, default)
44         setattr(cfg, name, node.get_parameter(name).value)
45     return cfg
46
47
48 def load_force_feedback_config(node: Node) -> ForceFeedbackConfig:
49     base = _load_fishing_config(node, FishingControllerConfig())
50     cfg = ForceFeedbackConfig(**base.__dict__)
51     for name in (
52         'tension_to_angle_gain', 'qeq_alpha', 'max_qeq_step',
53         'raise_gain', 'lower_gain',
54     ):
55         default = getattr(cfg, name)
56         node.declare_parameter(name, default)
57         setattr(cfg, name, node.get_parameter(name).value)
58     return cfg
59
60
61 class FishingControllerBase(Node):
62     """Common subscriptions, safety checks, and trajectory publishing."""
63
64     def __init__(self, node_name: str, cfg: FishingControllerConfig | None = None):
65         super().__init__(node_name)
66         if cfg is not None:
67             self._configure(cfg)
68
69     def _configure(self, cfg: FishingControllerConfig) -> None:
70         self.cfg = cfg
71         self._start_time = self.get_clock().now()
72         self._base_pos = cfg.joint_1_hold
73         self._pitch_pos = cfg.q_init
74         self._pitch_vel = 0.0
75         self._tension_raw = 0.0
76         self._tension_received = False
77         self._estop = False
78         self._fish_pos: Optional[Tuple[float, float, float]] = None
79         self._tip_pos: Optional[Tuple[float, float, float]] = None
80         self.snapshot = ControllerSnapshot(q_cmd=cfg.q_init)
81
82         self._traj_pub = self.create_publisher(
83             JointTrajectory,
84             '/position_trajectory_controller/joint_trajectory',
85             10,
86         )
87         self.create_subscription(
88             FishingTension, '/fishing_arm/tension', self._tension_cb, 10,

```

```

89         )
90         self.create_subscription(JointState, '/joint_states', self._joint_state_cb, 10)
91
92         period = 1.0 / cfg.control_rate_hz
93         self.create_timer(period, self._control_timer)
94
95         if cfg.control_dof == 2:
96             self.get_logger().warn(
97                 f'{node_name}: control_dof=2 is not implemented yet; '
98                 f'behaving as 1DOF on {cfg.pitch_joint}.',
99             )
100         elif cfg.control_dof != 1:
101             self.get_logger().error(
102                 f'{node_name}: unsupported control_dof={cfg.control_dof}.',
103             )
104         if cfg.fsm_enabled:
105             self.get_logger().info(
106                 f'{self.get_name()} ready - 1DOF on {cfg.pitch_joint}, FSM enabled '
107                 f'(REGULATE after {cfg.bite_time + cfg.hook_duration:.1f}s), '
108                 f'target={cfg.target_tension:.2f} N.',
109             )
110         else:
111             self.get_logger().info(
112                 f'{self.get_name()} ready - 1DOF on {cfg.pitch_joint}, FSM disabled, '
113                 f'regulating from t=0, target={cfg.target_tension:.2f} N.',
114             )
115
116         def _elapsed(self) -> float:
117             return (self.get_clock().now() - self._start_time).nanoseconds / 1e9
118
119         def _tension_cb(self, msg: FishingTension) -> None:
120             self._tension_raw = msg.tension_newtons
121             self._tension_received = True
122
123         def _joint_state_cb(self, msg: JointState) -> None:
124             names = list(msg.name)
125             cfg = self.cfg
126             if cfg.base_joint in names:
127                 self._base_pos = msg.position[names.index(cfg.base_joint)]
128             if cfg.pitch_joint in names:
129                 idx = names.index(cfg.pitch_joint)
130                 self._pitch_pos = msg.position[idx]
131                 if idx < len(msg.velocity):
132                     self._pitch_vel = msg.velocity[idx]
133
134         def _check_safety(self) -> bool:
135             if not self._tension_received:
136                 return False
137             if abs(self._tension_raw) > self.cfg.safety_tension_limit:
138                 if not self._estop:
139                     self.get_logger().warn(
140                         f'Tension {self._tension_raw:.2f} N exceeds safety limit '
141                         f'{self.cfg.safety_tension_limit:.2f} N. Holding position.',
142                     )
143                     self._estop = True
144                 return False
145             self._estop = False
146             return True
147
148         def _publish_position(self, pitch_cmd: float) -> None:
149             """Publish trajectory commands for the active control DOF."""
150             cfg = self.cfg
151             traj = JointTrajectory()
152             point = JointTrajectoryPoint()
153
154             if cfg.control_dof == 1:
155                 # 1DOF: notebook logic drives pitch only; base joint stays fixed.
156                 traj.joint_names = [cfg.base_joint, cfg.pitch_joint]
157                 point.positions = [cfg.joint_1_hold, pitch_cmd]
158             else:
159                 # Reserved for future 2DOF - pitch_cmd reused as pitch setpoint only.
160                 traj.joint_names = [cfg.pitch_joint]
161                 point.positions = [pitch_cmd]
162
163             point.time_from_start = Duration(seconds=0.1).to_msg()
164             traj.points.append(point)
165             self._traj_pub.publish(traj)
166             self.snapshot.q_cmd = pitch_cmd
167
168         def _control_timer(self) -> None:
169             raise NotImplementedError
170
171
172 class AdmittanceControllerNode(FishingControllerBase):
173     """ROS2 wrapper for Julie's admittance controller."""

```

```

174
175 def __init__(self):
176     super().__init__('admittance_controller')
177     cfg = load_admittance_config(self)
178     self._configure(cfg)
179     dt = 1.0 / cfg.control_rate_hz
180     self._core = AdmittanceControllerCore(cfg, dt)
181     self._core.reset(self._pitch_pos)
182     self.get_logger().info('Admittance (Julie) controller active.')
183
184 def _control_timer(self) -> None:
185     if not self._check_safety():
186         return
187
188     t = self._elapsed()
189     state = get_fishing_state(t, self.cfg)
190     tension = self._core.update_tension(self._tension_raw)
191
192     self.snapshot.state = state
193     self.snapshot.tension_raw = self._tension_raw
194     self.snapshot.tension_filtered = tension
195
196     if state == FishingState.HOOK_SET:
197         q_cmd = self._core.hook_set_target(t)
198         self._core.reset(self._pitch_pos)
199         self._core.theta_cmd = q_cmd
200     elif state == FishingState.REGULATE:
201         q_cmd, q_dot = self._core.update(
202             t=t,
203             state=state,
204             f_measured=tension,
205             theta=self._pitch_pos,
206             theta_dot=self._pitch_vel,
207             tip_pos=self._tip_pos,
208             fish_pos=self._fish_pos,
209             control_enabled=True,
210         )
211         self.snapshot.q_dot_cmd = q_dot
212     else:
213         q_cmd = self.cfg.q_init
214
215     self._publish_position(q_cmd)
216
217
218 class ForceFeedbackControllerNode(FishingControllerBase):
219     """ROS2 wrapper for Chaoyi's proportional force-feedback controller."""
220
221     def __init__(self):
222         super().__init__('force_feedback_controller')
223         cfg = load_force_feedback_config(self)
224         self._configure(cfg)
225         self._core = ForceFeedbackControllerCore(cfg)
226         self.get_logger().info('Force-feedback (Chaoyi) controller active.')
227
228     def _control_timer(self) -> None:
229         if not self._check_safety():
230             return
231
232         t = self._elapsed()
233         state = get_fishing_state(t, self.cfg)
234         tension = self._core.update_tension(self._tension_raw)
235
236         self.snapshot.state = state
237         self.snapshot.tension_raw = self._tension_raw
238         self.snapshot.tension_filtered = tension
239
240         q_cmd, _ = self._core.update(
241             t=t,
242             state=state,
243             tension_filtered=tension,
244             control_enabled=True,
245         )
246         self._publish_position(q_cmd)

```

software/src/control/control/fishing_fsm.py (1 lines)

1

software/src/control/control/force_feedback_controller.py (20 lines)

```
1 import rclpy
2
3 from control.fishing_controller_node import ForceFeedbackControllerNode
4
5
6 def main(args=None):
7     rclpy.init(args=args)
8     node = ForceFeedbackControllerNode()
9     try:
10         rclpy.spin(node)
11     except KeyboardInterrupt:
12         pass
13     finally:
14         node.destroy_node()
15         rclpy.shutdown()
16
17
18 if __name__ == '__main__':
19     main()
```

software/src/control/control/position_test_publisher.py (57 lines)

```
1 #!/usr/bin/env python3
2 import rclpy
3 from rclpy.node import Node
4 from trajectory_msgs.msg import JointTrajectory, JointTrajectoryPoint
5 import math
6
7 class PositionTestPublisher(Node):
8     def __init__(self):
9         super().__init__('position_test_publisher')
10
11         self.trajectory_pub = self.create_publisher(
12             JointTrajectory,
13             '/position_trajectory_controller/joint_trajectory',
14             10
15         )
16
17         # 10Hz control loop step to update smooth trajectory points
18         self.timer = self.create_timer(0.1, self.timer_callback)
19         self.start_time = self.get_clock().now()
20
21         # Operational limits for safety profile
22         self.max_amplitude = 0.5 # radians (~28 degrees)
23         self.frequency = 0.2 # Hz (slow cycles ttimertimero preserve internal motor gears)
24
25         self.get_logger().info("Dynamixel position validation node initialized.")
26
27     def timer_callback(self):
28         current_time = self.get_clock().now()
29         elapsed_time = (current_time - self.start_time).nanoseconds / 1e9
30
31         # Calculate smooth sinusoidal target configuration
32         target_position = self.max_amplitude * math.sin(2.0 * math.pi * self.frequency * elapsed_time)
33
34         msg = JointTrajectory()
35         msg.joint_names = ['Joint_1']
36
37         point = JointTrajectoryPoint()
38         point.positions = [target_position]
39         point.time_from_start.sec = 0
40         point.time_from_start.nanosec = int(1e8) # Command arriving 100ms smoothly from start
41
42         msg.points.append(point)
43         self.trajectory_pub.publish(msg)
44
45 def main(args=None):
46     rclpy.init(args=args)
47     node = PositionTestPublisher()
48     try:
49         rclpy.spin(node)
50     except KeyboardInterrupt:
51         pass
52     finally:
53         node.destroy_node()
54         rclpy.shutdown()
55
56 if __name__ == '__main__':
57     main()
```

```

1 from launch import LaunchDescription
2 from launch.actions import DeclareLaunchArgument
3 from launch.conditions import IfCondition
4 from launch.substitutions import (
5     LaunchConfiguration, PathJoinSubstitution, PythonExpression,
6 )
7 from launch_ros.actions import Node
8 from launch_ros.substitutions import FindPackageShare
9
10
11 def generate_launch_description():
12     controller_type = LaunchConfiguration('controller_type')
13     controller_type_arg = DeclareLaunchArgument(
14         'controller_type',
15         default_value='admittance',
16         description=(
17             'Tension controller: admittance (Julie), '
18             'force_feedback (Chaoyi), or none'
19         ),
20     )
21
22     use_sim = LaunchConfiguration('use_sim')
23     use_sim_arg = DeclareLaunchArgument(
24         'use_sim',
25         default_value='true',
26         description='Layer params_hw.yaml when false (skip FSM on real hardware)',
27     )
28
29     bringup_pkg = FindPackageShare('bringup')
30     params_file = PathJoinSubstitution([bringup_pkg, 'config', 'params.yaml'])
31     params_hw_file = PathJoinSubstitution([bringup_pkg, 'config', 'params_hw.yaml'])
32
33     admittance_hw = PythonExpression([
34         '"', use_sim, '" == 'false' and '"', controller_type, '" == 'admittance'",
35     ])
36     admittance_sim = PythonExpression([
37         '"', use_sim, '" == 'true' and '"', controller_type, '" == 'admittance'",
38     ])
39     ffb_hw = PythonExpression([
40         '"', use_sim, '" == 'false' and '"', controller_type, '" == 'force_feedback'",
41     ])
42     ffb_sim = PythonExpression([
43         '"', use_sim, '" == 'true' and '"', controller_type, '" == 'force_feedback'",
44     ])
45
46     return LaunchDescription([
47         controller_type_arg,
48         use_sim_arg,
49         Node(
50             package='control',
51             executable='admittance_node',
52             name='admittance_controller',
53             output='screen',
54             parameters=[params_file, params_hw_file],
55             condition=IfCondition(admittance_hw),
56         ),
57         Node(
58             package='control',
59             executable='admittance_node',
60             name='admittance_controller',
61             output='screen',
62             parameters=[params_file],
63             condition=IfCondition(admittance_sim),
64         ),
65         Node(
66             package='control',
67             executable='force_feedback_node',
68             name='force_feedback_controller',
69             output='screen',
70             parameters=[params_file, params_hw_file],
71             condition=IfCondition(ffb_hw),
72         ),
73         Node(
74             package='control',
75             executable='force_feedback_node',
76             name='force_feedback_controller',
77             output='screen',
78             parameters=[params_file],
79             condition=IfCondition(ffb_sim),
80         ),
81     ])

```

software/src/control/package.xml (24 lines)

```

1 <?xml version="1.0"?>
2 <?xml-model href="http://download.ros.org/schema/package_format3.xsd" schematypens="http://www.w3.org/2001/XMLSchema"?>
3 <package format="3">
4   <name>control</name>
5   <version>0.0.0</version>
6   <description>Tension controllers for the robotic fishing arm (admittance and force feedback).</description>
7   <maintainer email="nathange784@gmail.com">nzge</maintainer>
8   <license>MIT</license>
9
10  <exec_depend>rclpy</exec_depend>
11  <exec_depend>sensor_msgs</exec_depend>
12  <exec_depend>trajectory_msgs</exec_depend>
13  <exec_depend>interfaces</exec_depend>
14
15  <test_depend>ament_copyright</test_depend>
16  <test_depend>ament_flake8</test_depend>
17  <test_depend>ament_pep257</test_depend>
18  <test_depend>python3-pytest</test_depend>
19
20  <export>
21    <build_type>ament_python</build_type>
22  </export>
23 </package>

```

software/src/control/setup.cfg (5 lines)

```

1 [develop]
2 script_dir=$base/lib/control
3 [install]
4 install_scripts=$base/lib/control

```

software/src/control/setup.py (36 lines)

```

1 import os
2 from glob import glob
3 from setuptools import find_packages, setup
4
5 package_name = 'control'
6
7 setup(
8     name=package_name,
9     version='0.0.0',
10    packages=find_packages(exclude=['test']),
11    data_files=[
12        ('share/ament_index/resource_index/packages',
13         ['resource/' + package_name]),
14        ('share/' + package_name, ['package.xml']),
15        (os.path.join('share', package_name, 'launch'), glob('launch/*.launch.py')),
16    ],
17    install_requires=['setuptools'],
18    zip_safe=True,
19    maintainer='nzge',
20    maintainer_email='nathange784@gmail.com',
21    description='Tension controllers for the robotic fishing arm',
22    license='MIT',
23    extras_require={
24        'test': [
25            'pytest',
26        ],
27    },
28    entry_points={
29        'console_scripts': [
30            'admittance_node = control.admittance_controller:main',
31            'force_feedback_node = control.force_feedback_controller:main',
32            'position_test_publisher = control.position_test_publisher:main',
33        ],
34    },
35 )

```

software/src/control/test/test_controller_simulation.py (79 lines)

```

1 """Headless simulation tests for admittance and force-feedback controllers."""
2 from __future__ import annotations
3
4 import importlib.util
5 import json

```

```

6 import sys
7 from pathlib import Path
8
9 import numpy as np
10 import pytest
11
12 REPO_ROOT = Path(__file__).resolve().parents[4]
13 SCRIPT = REPO_ROOT / 'scripts' / 'simulate_controllers.py'
14
15
16 def _load_sim_module():
17     sys.path.insert(0, str(REPO_ROOT / 'software/src/control'))
18     spec = importlib.util.spec_from_file_location('simulate_controllers', SCRIPT)
19     module = importlib.util.module_from_spec(spec)
20     assert spec.loader is not None
21     spec.loader.exec_module(module)
22     return module
23
24
25 pytest.importorskip('mujoco')
26
27
28 @pytest.fixture(scope='module')
29 def sim():
30     return _load_sim_module()
31
32
33 @pytest.mark.parametrize('controller_type', ['admittance', 'force_feedback', 'baseline'])
34 def test_controller_simulation_produces_finite_metrics(sim, controller_type, tmp_path):
35     adm_cfg, ffb_cfg = sim.configs_from_params_yaml()
36     cfg = ffb_cfg if controller_type == 'force_feedback' else adm_cfg
37
38     data = sim.run_simulation(controller_type, cfg, duration=8.0, settle_steps=100)
39     metrics = sim.compute_metrics(data['time_s'], data['tension_filtered_N'], cfg)
40
41     assert data['time_s'].size > 100
42     assert np.all(np.isfinite(data['tension_filtered_N']))
43     assert np.isfinite(metrics['rms_error_N'])
44
45     sim.save_recording(tmp_path, controller_type, data, cfg, metrics)
46     assert (tmp_path / f'{controller_type}.npz').exists()
47     assert (tmp_path / f'{controller_type}.csv').exists()
48     with (tmp_path / f'{controller_type}_metrics.json').open() as f:
49         saved = json.load(f)
50     assert 'rms_error_N' in saved
51
52
53 def test_plot_and_export_bundle(sim, tmp_path):
54     adm_cfg, ffb_cfg = sim.configs_from_params_yaml()
55     runs = {}
56     metrics = {}
57     for name, ctrl, cfg in (
58         ('baseline', 'baseline', adm_cfg),
59         ('admittance', 'admittance', adm_cfg),
60         ('force_feedback', 'force_feedback', ffb_cfg),
61     ):
62         data = sim.run_simulation(ctrl, cfg, duration=6.0, settle_steps=50)
63         metrics[name] = sim.compute_metrics(data['time_s'], data['tension_filtered_N'], cfg)
64         runs[name] = data
65         sim.save_recording(tmp_path, name, data, cfg, metrics[name])
66
67     pytest.importorskip('matplotlib')
68     sim.plot_results(tmp_path, runs, adm_cfg, metrics)
69
70     for stem in (
71         'comparison_tension',
72         'comparison_position',
73         'admittance_detail',
74         'force_feedback_detail',
75         'metrics_summary',
76     ):
77         assert (tmp_path / f'{stem}.png').exists()
78         assert (tmp_path / f'{stem}.pdf').exists()

```

software/src/control/test/test_copyright.py (26 lines)

```

1 # Copyright 2015 Open Source Robotics Foundation, Inc.
2 #
3 # Licensed under the Apache License, Version 2.0 (the "License");
4 # you may not use this file except in compliance with the License.
5 # You may obtain a copy of the License at
6 #
7 #     http://www.apache.org/licenses/LICENSE-2.0

```



```

8 #
9 # Unless required by applicable law or agreed to in writing, software
10 # distributed under the License is distributed on an "AS IS" BASIS,
11 # WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
12 # See the License for the specific language governing permissions and
13 # limitations under the License.
14
15 from ament_copyright.main import main
16 import pytest
17
18
19 # Remove the `skip` decorator once the source file(s) have a copyright header
20 @pytest.mark.skip(reason='No copyright header has been placed in the generated source file.')
21 @pytest.mark.copyright
22 @pytest.mark.linter
23 def test_copyright():
24     rc = main(argv=['.', 'test'])
25     assert rc == 0, 'Found errors'

```

software/src/control/test/test_flake8.py (26 lines)

```

1 # Copyright 2017 Open Source Robotics Foundation, Inc.
2 #
3 # Licensed under the Apache License, Version 2.0 (the "License");
4 # you may not use this file except in compliance with the License.
5 # You may obtain a copy of the License at
6 #
7 #     http://www.apache.org/licenses/LICENSE-2.0
8 #
9 # Unless required by applicable law or agreed to in writing, software
10 # distributed under the License is distributed on an "AS IS" BASIS,
11 # WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
12 # See the License for the specific language governing permissions and
13 # limitations under the License.
14
15 from ament_flake8.main import main_with_errors
16 import pytest
17
18
19 @pytest.mark.flake8
20 @pytest.mark.linter
21 def test_flake8():
22     rc, errors = main_with_errors(argv=[])
23     assert rc == 0, \
24         'Found %d code style errors / warnings:\n' % len(errors) + \
25         '\n'.join(errors)

```

software/src/control/test/test_pep257.py (24 lines)

```

1 # Copyright 2015 Open Source Robotics Foundation, Inc.
2 #
3 # Licensed under the Apache License, Version 2.0 (the "License");
4 # you may not use this file except in compliance with the License.
5 # You may obtain a copy of the License at
6 #
7 #     http://www.apache.org/licenses/LICENSE-2.0
8 #
9 # Unless required by applicable law or agreed to in writing, software
10 # distributed under the License is distributed on an "AS IS" BASIS,
11 # WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
12 # See the License for the specific language governing permissions and
13 # limitations under the License.
14
15 from ament_pep257.main import main
16 import pytest
17
18
19 @pytest.mark.linter
20 @pytest.mark.pep257
21 def test_pep257():
22     rc = main(argv=['.', 'test'])
23     assert rc == 0, 'Found code style errors / warnings'

```

Package: description

software/src/description/CMakeLists.txt (34 lines)

```
1 cmake_minimum_required(VERSION 3.8)
2 project(description)
3
4 if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
5   add_compile_options(-Wall -Wextra -Wpedantic)
6 endif()
7
8 # find dependencies
9 find_package(ament_cmake REQUIRED)
10 # uncomment the following section in order to fill in
11 # further dependencies manually.
12 # find_package(<dependency> REQUIRED)
13
14 if(BUILD_TESTING)
15   find_package(ament_lint_auto REQUIRED)
16   # the following line skips the linter which checks for copyrights
17   # comment the line when a copyright and license is added to all source files
18   set(ament_cmake_copyright_FOUND TRUE)
19   # the following line skips cpplint (only works in a git repo)
20   # comment the line when this package is in a git repo and when
21   # a copyright and license is added to all source files
22   set(ament_cmake_cpplint_FOUND TRUE)
23   ament_lint_auto_find_test_dependencies()
24 endif()
25
26 ament_package()
27
28 # Install URDF and meshes
29 install(DIRECTORY
30   urdf
31   meshes
32   DESTINATION share/${PROJECT_NAME})
33 )
```

software/src/description/package.xml (28 lines)

```
1 <?xml version="1.0"?>
2 <?xml-model href="http://download.ros.org/schema/package_format3.xsd" schematypens="http://www.w3.org/2001/XMLSchema"?>
3 <package format="3">
4   <name>description</name>
5   <version>0.0.0</version>
6   <description>Robot description assets: URDF/xacro, MuJoCo MJCF, and STL meshes for the fishing arm.</description>
7   <maintainer email="nathange784@gmail.com">nzge</maintainer>
8   <license>MIT</license>
9
10   <buildtool_depend>ament_cmake</buildtool_depend>
11
12   <!-- xacro is needed to process fishing-robot.urdf.xacro -->
13   <exec_depend>xacro</exec_depend>
14
15   <!-- Hardware plugin referenced by the URDF on the real-robot path -->
16   <exec_depend>dynamixel_hardware</exec_depend>
17
18   <!-- Simulation plugin referenced by the URDF (built from source; see note) -->
19   <!-- <exec_depend>mujoco_ros2_control</exec_depend> -->
20
21   <test_depend>ament_lint_auto</test_depend>
22   <test_depend>ament_lint_common</test_depend>
23
24   <export>
25     <build_type>ament_cmake</build_type>
26   </export>
27 </package>
```

software/src/description/urdf/fishing-robot.urdf.xacro (203 lines)

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <robot xmlns:xacro="http://www.ros.org/wiki/xacro" name="fishing_robot">
3
4   <!-- Architecture flags (overridden by the launch file). -->
5   <xacro:arg name="use_sim" default="true"/>
6   <xacro:arg name="headless" default="false"/>
```

```

7
8 <link name="base_link">
9   <inertial>
10     <origin xyz="9.64494158785974E-06 -0.0124364350363364 -0.00800838743932182" rpy="0 0 0" />
11     <mass value="0.10242435780476" />
12     <inertia ixx="3.44658980710862E-05" ixy="1.15685827535353E-08" ixz="-4.74633377970236E-09" iyy="7.35797461812803E-05" iyz=
13   </inertial>
14   <visual>
15     <origin xyz="0 0 0" rpy="0 0 0" />
16     <geometry>
17       <mesh filename="package://description/meshes/base_link.STL" />
18     </geometry>
19     <material name=""><color rgba="0.752941176470588 0.752941176470588 0.752941176470588 1" /></material>
20   </visual>
21   <collision>
22     <origin xyz="0 0 0" rpy="0 0 0" />
23     <geometry>
24       <mesh filename="package://description/meshes/base_link_collision.STL" />
25     </geometry>
26   </collision>
27 </link>
28
29 <link name="Link_1">
30   <inertial>
31     <origin xyz="1.98732975738997E-05 -0.000248210549754878 0.025285547707494" rpy="0 0 0" />
32     <mass value="0.0497087584246774" />
33     <inertia ixx="1.48173848978349E-05" ixy="4.74633377970562E-09" ixz="1.15685827535328E-08" iyy="1.25308227827468E-05" iyz=
34   </inertial>
35   <visual>
36     <origin xyz="0 0 0" rpy="0 0 0" />
37     <geometry>
38       <mesh filename="package://description/meshes/Link_1.STL" />
39     </geometry>
40     <material name=""><color rgba="0.752941176470588 0.752941176470588 0.752941176470588 1" /></material>
41   </visual>
42   <collision>
43     <origin xyz="0 0 0" rpy="0 0 0" />
44     <geometry>
45       <mesh filename="package://description/meshes/Link_1_collision.STL" />
46     </geometry>
47   </collision>
48 </link>
49
50 <joint name="Joint_1" type="revolute">
51   <origin xyz="0 0 0.02335" rpy="0 0 0" />
52   <parent link="base_link" />
53   <child link="Link_1" />
54   <axis xyz="0 0 1" />
55   <limit lower="-3.1416" upper="3.1416" effort="2.5" velocity="5.76" />
56 </joint>
57
58 <link name="Link_2">
59   <inertial>
60     <origin xyz="-0.000376762904282731 0.0228500014053185 0.0758975605697149" rpy="0 0 0" />
61     <mass value="0.0374844556058819" />
62     <inertia ixx="9.81277949436277E-05" ixy="4.17865645290201E-13" ixz="8.66764648076052E-08" iyy="9.58497532082512E-05" iyz=
63   </inertial>
64   <visual>
65     <origin xyz="0 0 0" rpy="0 0 0" />
66     <geometry>
67       <mesh filename="package://description/meshes/Link_2.STL" />
68     </geometry>
69     <material name=""><color rgba="0.752941176470588 0.752941176470588 0.752941176470588 1" /></material>
70   </visual>
71   <collision>
72     <origin xyz="0 0 0" rpy="0 0 0" />
73     <geometry>
74       <mesh filename="package://description/meshes/Link_2_collision.STL" />
75     </geometry>
76   </collision>
77 </link>
78
79 <joint name="Joint_2" type="revolute">
80   <origin xyz="0 -0.02335 0.03875" rpy="0 0 0" />
81   <parent link="Link_1" />
82   <child link="Link_2" />
83   <axis xyz="0 1 0" />
84   <limit lower="-3.1416" upper="3.1416" effort="2.5" velocity="5.76" />
85 </joint>
86
87 <!-- Massless frame at the rod tip where the line attaches and the load cell
88       measures tension. Matches the MuJoCo 'rod_tip_site' and sensor frame_id. -->
89 <link name="rod_tip_link">
90   <joint name="rod_tip_joint" type="fixed">
91     <origin xyz="0 0.024 0.185" rpy="0 0 0" />

```

```

92     <parent link="Link_2" />
93     <child link="rod_tip_link" />
94 </joint>
95
96 <!-- Simulation-only: the virtual fish. ros2_control requires every commanded
97 joint to exist in the URDF, so we declare the fish's slide joint here (it
98 also exists in the MJCF, where it is physically simulated). Not part of the
99 real arm, hence gated on use_sim. -->
100 <xacro:if value="$(arg use_sim)">
101     <!-- The fish frame MUST match the MJCF 'fish' body (pos 0 0.4 0.025, slide
102 axis 0 -1 0) so the load cell's TF-based line-length estimate agrees
103 with the physically simulated line. Mismatch here makes the measured
104 tension diverge from what the rod actually sees. -->
105     <link name="fish_link" />
106     <joint name="fish_swim" type="prismatic">
107         <origin xyz="0 0.4 0.025" rpy="0 0 0" />
108         <parent link="base_link" />
109         <child link="fish_link" />
110         <axis xyz="0 -1 0" />
111         <limit lower="-0.35" upper="0.35" effort="10.0" velocity="10.0" />
112     </joint>
113 </xacro:if>
114
115 <ros2_control name="FishingRobotHardware" type="system">
116     <hardware>
117         <xacro:if value="$(arg use_sim)">
118             <plugin>mujoco_ros2_control/MujocoSystemInterface</plugin>
119             <!-- The MJCF scene the plugin loads, steps and renders. -->
120             <param name="mujoco_model">$(find description)/urdf/fishing-robot_sim.xml</param>
121             <!-- Apply this MJCF keyframe as the initial state on boot (no manual
122 keyframe selection needed in the viewer). -->
123             <param name="initial_keyframe">starting_pose</param>
124             <!-- Run with (false) or without (true) the MuJoCo GUI window. -->
125             <param name="headless">$(arg headless)</param>
126         </xacro:if>
127         <xacro:unless value="$(arg use_sim)">
128             <plugin>dynamixel_hardware/DynamixelHardware</plugin>
129             <param name="usb_port">/dev/ttyUSB0</param>
130             <param name="baud_rate">57600</param>
131             <param name="use_dummy">>false</param>
132         </xacro:unless>
133     </hardware>
134
135     <!-- Simulation: both joints are actuated in MuJoCo. -->
136     <xacro:if value="$(arg use_sim)">
137         <joint name="Joint_1">
138             <param name="id">1</param>
139             <command_interface name="position">
140                 <param name="min">-3.1416</param>
141                 <param name="max">3.1416</param>
142             </command_interface>
143             <state_interface name="position"/>
144             <state_interface name="velocity"/>
145             <state_interface name="effort"/>
146         </joint>
147
148         <joint name="Joint_2">
149             <param name="id">2</param>
150             <command_interface name="position">
151                 <param name="min">-3.1416</param>
152                 <param name="max">3.1416</param>
153             </command_interface>
154             <state_interface name="position"/>
155             <state_interface name="velocity"/>
156             <state_interface name="effort"/>
157         </joint>
158     </xacro:if>
159
160     <!-- Hardware: 1-DOF pitch only (project_w_t_controller.ipynb: DXL_ID=1 @ 57600).
161 Base Joint_1 is not on the bus; only the rod pitch servo is connected. -->
162     <xacro:unless value="$(arg use_sim)">
163         <joint name="Joint_2">
164             <param name="id">1</param>
165             <command_interface name="position">
166                 <param name="min">-3.1416</param>
167                 <param name="max">3.1416</param>
168             </command_interface>
169             <state_interface name="position"/>
170             <state_interface name="velocity"/>
171             <state_interface name="effort"/>
172         </joint>
173     </xacro:unless>
174
175     <!-- Simulation-only interfaces: the virtual fish and the line load cell exist
176 in the MuJoCo scene, not on the physical arm. -->

```

```

177 <xacro:if value="$(arg use_sim)">
178 <!-- The fish's pull: 1-DOF slide joint driven by an effort command
179 (forward_command_controller -> MuJoCo 'fish_force' motor). -->
180 <joint name="fish_swim">
181 <command_interface name="effort"/>
182 <state_interface name="position"/>
183 <state_interface name="velocity"/>
184 <state_interface name="effort"/>
185 </joint>
186
187 <!-- Line tension load cell: MuJoCo force+torque at the rod tip mapped to a
188 single FTS, published by force_torque_sensor_broadcaster. The plugin
189 reads MJCF sensors 'tension_sensor_force' and 'tension_sensor_torque'. -->
190 <sensor name="tension_sensor">
191 <param name="mujoco_type">fts</param>
192 <param name="mujoco_sensor_name">tension_sensor</param>
193 <state_interface name="force.x"/>
194 <state_interface name="force.y"/>
195 <state_interface name="force.z"/>
196 <state_interface name="torque.x"/>
197 <state_interface name="torque.y"/>
198 <state_interface name="torque.z"/>
199 </sensor>
200 </xacro:if>
201 </ros2_control>
202
203 </robot>

```

software/src/description/urdf/fishing-robot_hw.urdf (114 lines)

```

1 <?xml version="1.0" ?>
2 <!-- ===== -->
3 <!-- | This document was autogenerated by xacro from fishing-robot.urdf.xacro | -->
4 <!-- | EDITING THIS FILE BY HAND IS NOT RECOMMENDED | -->
5 <!-- ===== -->
6 <robot name="fishing_robot">
7 <link name="base_link">
8 <inertial>
9 <origin rpy="0 0 0" xyz="9.64494158785974E-06 -0.0124364350363364 -0.00800838743932182"/>
10 <mass value="0.10242435780476"/>
11 <inertia ixx="3.44658980710862E-05" ixy="1.15685827535353E-08" ixz="-4.74633377970236E-09" iyy="7.35797461812803E-05" iyz=
12 </inertial>
13 <visual>
14 <origin rpy="0 0 0" xyz="0 0 0"/>
15 <geometry>
16 <mesh filename="package://description/meshes/base_link.STL"/>
17 </geometry>
18 <material name="">
19 <color rgba="0.752941176470588 0.752941176470588 0.752941176470588 1"/>
20 </material>
21 </visual>
22 <collision>
23 <origin rpy="0 0 0" xyz="0 0 0"/>
24 <geometry>
25 <mesh filename="package://description/meshes/base_link_collision.STL"/>
26 </geometry>
27 </collision>
28 </link>
29 <link name="Link_1">
30 <inertial>
31 <origin rpy="0 0 0" xyz="1.98732975738997E-05 -0.000248210549754878 0.025285547707494"/>
32 <mass value="0.0497087584246774"/>
33 <inertia ixx="1.48173848978349E-05" ixy="4.74633377970562E-09" ixz="1.15685827535328E-08" iyy="1.25308227827468E-05" iyz=
34 </inertial>
35 <visual>
36 <origin rpy="0 0 0" xyz="0 0 0"/>
37 <geometry>
38 <mesh filename="package://description/meshes/Link_1.STL"/>
39 </geometry>
40 <material name="">
41 <color rgba="0.752941176470588 0.752941176470588 0.752941176470588 1"/>
42 </material>
43 </visual>
44 <collision>
45 <origin rpy="0 0 0" xyz="0 0 0"/>
46 <geometry>
47 <mesh filename="package://description/meshes/Link_1_collision.STL"/>
48 </geometry>
49 </collision>
50 </link>
51 <joint name="Joint_1" type="revolute">
52 <origin rpy="0 0 0" xyz="0 0 0.02335"/>
53 <parent link="base_link"/>

```

```

54   <child link="Link_1"/>
55   <axis xyz="0 0 1"/>
56   <limit effort="2.5" lower="-3.1416" upper="3.1416" velocity="5.76"/>
57 </joint>
58 <link name="Link_2">
59   <inertial>
60     <origin rpy="0 0 0" xyz="-0.000376762904282731 0.0228500014053185 0.0758975605697149"/>
61     <mass value="0.0374844556058819"/>
62     <inertia ixx="9.81277949436277E-05" ixy="4.17865645290201E-13" ixz="8.66764648076052E-08" iyy="9.58497532082512E-05" iyz=
63   </inertial>
64   <visual>
65     <origin rpy="0 0 0" xyz="0 0 0"/>
66     <geometry>
67       <mesh filename="package://description/meshes/Link_2.STL"/>
68     </geometry>
69     <material name="">
70       <color rgba="0.752941176470588 0.752941176470588 0.752941176470588 1"/>
71     </material>
72   </visual>
73   <collision>
74     <origin rpy="0 0 0" xyz="0 0 0"/>
75     <geometry>
76       <mesh filename="package://description/meshes/Link_2_collision.STL"/>
77     </geometry>
78   </collision>
79 </link>
80 <joint name="Joint_2" type="revolute">
81   <origin rpy="0 0 0" xyz="0 -0.02335 0.03875"/>
82   <parent link="Link_1"/>
83   <child link="Link_2"/>
84   <axis xyz="0 1 0"/>
85   <limit effort="2.5" lower="-3.1416" upper="3.1416" velocity="5.76"/>
86 </joint>
87 <ros2_control name="FishingRobotHardware" type="system">
88   <hardware>
89     <plugin>dynamixel_hardware/DynamixelHardware</plugin>
90     <param name="usb_port">/dev/ttyUSB0</param>
91     <param name="baud_rate">57600</param>
92     <param name="use_dummy">false</param>
93   </hardware>
94   <joint name="Joint_1">
95     <param name="id">1</param>
96     <command_interface name="position">
97       <param name="min">-3.1416</param>
98       <param name="max">3.1416</param>
99     </command_interface>
100    <state_interface name="position"/>
101    <state_interface name="velocity"/>
102  </joint>
103  <joint name="Joint_2">
104    <param name="id">2</param>
105    <command_interface name="position">
106      <param name="min">-3.1416</param>
107      <param name="max">3.1416</param>
108    </command_interface>
109    <state_interface name="position"/>
110    <state_interface name="velocity"/>
111  </joint>
112 </ros2_control>
113 </robot>

```

software/src/description/urdf/fishing-robot_sim.urdf (114 lines)

```

1  <?xml version="1.0" ?>
2  <!-- ===== -->
3  <!-- | This document was autogenerated by xacro from fishing-robot.urdf.xacro | -->
4  <!-- | EDITING THIS FILE BY HAND IS NOT RECOMMENDED | -->
5  <!-- ===== -->
6  <robot name="fishing_robot">
7    <link name="base_link">
8      <inertial>
9        <origin rpy="0 0 0" xyz="9.64494158785974E-06 -0.0124364350363364 -0.00800838743932182"/>
10       <mass value="0.10242435780476"/>
11       <inertia ixx="3.44658980710862E-05" ixy="1.15685827535353E-08" ixz="-4.74633377970236E-09" iyy="7.35797461812803E-05" iyz=
12     </inertial>
13     <visual>
14       <origin rpy="0 0 0" xyz="0 0 0"/>
15       <geometry>
16         <mesh filename="../../meshes/base_link.STL"/>
17       </geometry>
18       <material name="">
19         <color rgba="0.752941176470588 0.752941176470588 0.752941176470588 1"/>
20       </material>

```

```

21 </visual>
22 <collision>
23   <origin rpy="0 0 0" xyz="0 0 0"/>
24   <geometry>
25     <mesh filename="../../meshes/base_link_collision.STL"/>
26   </geometry>
27 </collision>
28 </link>
29
30 <link name="Link_1">
31   <inertial>
32     <origin rpy="0 0 0" xyz="1.98732975738997E-05 -0.000248210549754878 0.025285547707494"/>
33     <mass value="0.0497087584246774"/>
34     <inertia ixx="1.48173848978349E-05" ixy="4.74633377970562E-09" ixz="1.15685827535328E-08" iyy="1.25308227827468E-05" iyz=
35   </inertial>
36   <visual>
37     <origin rpy="0 0 0" xyz="0 0 0"/>
38     <geometry>
39       <mesh filename="../../meshes/Link_1.STL"/>
40     </geometry>
41     <material name="">
42       <color rgba="0.752941176470588 0.752941176470588 0.752941176470588 1"/>
43     </material>
44   </visual>
45   <collision>
46     <origin rpy="0 0 0" xyz="0 0 0"/>
47     <geometry>
48       <mesh filename="../../meshes/Link_1_collision.STL"/>
49     </geometry>
50   </collision>
51 </link>
52 <joint name="Joint_1" type="revolute">
53   <origin rpy="0 0 0" xyz="0 0 0.02335"/>
54   <parent link="base_link"/>
55   <child link="Link_1"/>
56   <axis xyz="0 0 1"/>
57   <limit effort="2.5" lower="-3.1416" upper="3.1416" velocity="5.76"/>
58 </joint>
59
60 <link name="Link_2">
61   <inertial>
62     <origin rpy="0 0 0" xyz="-0.000376762904282731 0.0228500014053185 0.0758975605697149"/>
63     <mass value="0.0374844556058819"/>
64     <inertia ixx="0.81277949436277E-05" ixy="4.17865645290201E-13" ixz="8.66764648076052E-08" iyy="9.58497532082512E-05" iyz=
65   </inertial>
66   <visual>
67     <origin rpy="0 0 0" xyz="0 0 0"/>
68     <geometry>
69       <mesh filename="../../meshes/Link_2.STL"/>
70     </geometry>
71     <material name="">
72       <color rgba="0.752941176470588 0.752941176470588 0.752941176470588 1"/>
73     </material>
74   </visual>
75   <collision>
76     <origin rpy="0 0 0" xyz="0 0 0"/>
77     <geometry>
78       <mesh filename="../../meshes/Link_2_collision.STL"/>
79     </geometry>
80   </collision>
81 </link>
82 <joint name="Joint_2" type="revolute">
83   <origin rpy="0 0 0" xyz="0 -0.02335 0.03875"/>
84   <parent link="Link_1"/>
85   <child link="Link_2"/>
86   <axis xyz="0 1 0"/>
87   <limit effort="2.5" lower="-3.1416" upper="3.1416" velocity="5.76"/>
88 </joint>
89
90 <ros2_control name="FishingRobotHardware" type="system">
91   <hardware>
92     <plugin>mujoco_ros2_control/MujocoSystem</plugin>
93   </hardware>
94   <joint name="Joint_1">
95     <param name="id">1</param>
96     <command_interface name="position">
97       <param name="min">-3.1416</param>
98       <param name="max">3.1416</param>
99     </command_interface>
100    <state_interface name="position"/>
101    <state_interface name="velocity"/>
102  </joint>
103  <joint name="Joint_2">
104    <param name="id">2</param>
105    <command_interface name="position">

```

```

106     <param name="min">-3.1416</param>
107     <param name="max">3.1416</param>
108   </command_interface>
109   <state_interface name="position"/>
110   <state_interface name="velocity"/>
111 </joint>
112 </ros2_control>
113 </robot>

```

software/src/description/urdf/fishing-robot_sim.xml (121 lines)

```

1 <mujoco model="fishing_robot">
2   <compiler angle="radian" meshdir="../meshes"/>
3
4   <option gravity="0 0 -9.81"/>
5
6   <!-- Enlarge the offscreen framebuffer so the diagnostics renderer can capture
7     HD (1280x720) isometric frames; MuJoCo defaults to 640x480. -->
8   <visual>
9     <global offwidth="1920" offheight="1080"/>
10 </visual>
11
12 <asset>
13   <mesh name="base_link_collision" file="base_link_collision.STL"/>
14   <mesh name="Link_1_collision" file="Link_1_collision.STL"/>
15   <mesh name="Link_2_collision" file="Link_2_collision.STL"/>
16
17   <mesh name="base_link_visual" file="base_link.STL"/>
18   <mesh name="Link_1_visual" file="Link_1.STL"/>
19   <mesh name="Link_2_visual" file="Link_2.STL"/>
20
21   <!-- Fish body mesh: fish.STL is authored in centimetres (~29 cm long), unlike
22     the arm STLs which are already in metres. scale=0.004 yields ~12 cm. -->
23   <mesh name="fish_mesh" file="fish.STL" scale="0.004 0.004 0.004"/>
24
25   <!-- Checkered floor for the ground plane -->
26   <texture name="grid" type="2d" builtin="checker" rgb1="0.2 0.25 0.3" rgb2="0.3 0.35 0.4" width="300" height="300"/>
27   <material name="grid" texture="grid" texrepeat="8 8" reflectance="0.1"/>
28 </asset>
29
30 <worldbody>
31   <light pos="0 0 1.5" dir="0 0 -1" diffuse="0.8 0.8 0.8"/>
32
33   <!-- Ground at the robot base level (z=0). Its default contact groups
34     (contype=1/conaffinity=1) collide with the arm's collision meshes,
35     so the rod cannot swing below the base plane. -->
36   <geom name="ground" type="plane" pos="0 0 -0.0253" size="2 2 0.01" material="grid"/>
37
38   <!-- ===== The Robot Arm ===== -->
39   <body name="base_link" pos="0 0 0">
40     <geom type="mesh" mesh="base_link_collision" group="3"/>
41     <geom type="mesh" mesh="base_link_visual" rgba="0.75 0.75 0.75 1" contype="0" conaffinity="0"/>
42
43     <body name="Link_1" pos="0 0 0.02335">
44       <inertial pos="1.98733e-05 -0.000248211 0.0252855" quat="0.997703 0.0677224 -0.00113881 0.00132904" mass="0.0497088" di
45       <joint name="Joint_1" pos="0 0 0" axis="0 0 1" range="-3.1416 3.1416" actuatorfrange="-2.5 2.5"/>
46       <geom type="mesh" mesh="Link_1_collision" group="3"/>
47       <geom type="mesh" mesh="Link_1_visual" rgba="0.75 0.75 0.75 1" contype="0" conaffinity="0"/>
48
49       <body name="Link_2" pos="0 -0.02335 0.03875">
50         <inertial pos="-0.000376763 0.02285 0.0758976" quat="1 0 -0.000462491 0" mass="0.0374845" diaginertia="9.81279e-05 9.
51         <joint name="Joint_2" pos="0 0 0" axis="0 1 0" range="-3.1416 3.1416"
52           actuatorfrange="-2.5 2.5" damping="1.8" armature="0.03"/>
53         <geom type="mesh" mesh="Link_2_collision" group="3"/>
54         <geom type="mesh" mesh="Link_2_visual" rgba="0.75 0.75 0.75 1" contype="0" conaffinity="0"/>
55
56         <!-- Rod tip: near end of the line + load-cell measurement point.
57           Mirrors 'rod_tip_link' in the URDF. -->
58         <site name="rod_tip_site" pos="0 0.024 0.185" size="0.01" rgba="0 1 0 1"/>
59       </body>
60     </body>
61   </body>
62
63   <!-- ===== The Fish =====
64     Environmental disturbance at the FAR end of the line. Rendered with the
65     generated fish mesh; the euler rotates its nose (+X) to face the robot
66     (-Y). A slide joint lets the fish swim along Y; the 'fish_force'
67     actuator lets planning/fish_agent pull on it, tensioning the line. -->
68   <body name="fish" pos="0 0.4 0.025">
69     <joint name="fish_swim" type="slide" axis="0 -1 0" range="-0.35 0.35" damping="0.5"/>
70     <!-- contype/conaffinity 0: the fish ignores ground collision (it only
71       slides in Y); otherwise it would penetrate/fight the floor. -->
72     <geom name="fish_geom" type="mesh" mesh="fish_mesh" euler="0 0 -1.5708"

```



```

73         rgba="0.85 0.5 0.2 1" mass="0.25" contype="0" conaffinity="0"/>
74         <site name="fish_site" pos="0 0 0" size="0.004" rgba="1 0 0 1"/>
75     </body>
76 </worldbody>
77
78 <!-- ===== The Fishing Line =====
79     Elastic spatial tendon: rod tip <-> fish. springlength is the slack
80     length; beyond it the line pulls like a spring (stiffness/damping). -->
81 <!-- Elastic spatial tendon: rod tip <-> fish. springlength sets the slack
82     length; when geometric distance exceeds it the line pulls. Tuned so the
83     starting_pose keyframe (Joint_2=1.745) begins with a taut line (~0.22 m). -->
84 <tendon>
85     <spatial name="fishing_line" width="0.0015" rgba="0.05 0.05 0.05 1"
86         stiffness="220" damping="1.5" springlength="0.20">
87         <site site="rod_tip_site"/>
88         <site site="fish_site"/>
89     </spatial>
90 </tendon>
91
92 <actuator>
93     <!-- Arm servos: the ros2_control 'position' command interface for Joint_1 /
94         Joint_2 maps natively onto these MuJoCo position actuators. -->
95     <position joint="Joint_1" name="Joint_1" kp="25" dampratio="1.0" ctrlrange="-3.1416 3.1416"/>
96     <position joint="Joint_2" name="Joint_2" kp="85" dampratio="1.0" ctrlrange="-3.1416 3.1416"/>
97     <!-- The fish's pull: ros2_control 'effort' interface for the fish_swim joint
98         maps onto this motor (commanded via a forward_command_controller). -->
99     <!-- gear=-1 so positive effort pulls the fish away from the rod (+Y),
100         tensioning the line. fish_agent publishes positive Newton values. -->
101     <motor name="fish_force" joint="fish_swim" gear="-1" ctrlrange="-5 5"/>
102 </actuator>
103
104 <!-- ===== Line tension (the load cell) =====
105     Force transmitted through the rod-tip site = what the load cell reads.
106     mujoco_ros2_control combines the _force/_torque pair into a single FTS,
107     exposed by force_torque_sensor_broadcaster. 'line_length' aids debugging. -->
108 <sensor>
109     <force name="tension_sensor_force" site="rod_tip_site"/>
110     <torque name="tension_sensor_torque" site="rod_tip_site"/>
111     <tendonpos name="line_length" tendon="fishing_line"/>
112 </sensor>
113
114 <!-- qpos order: Joint_1, Joint_2, fish_swim. Rod starts upright with the
115     line cast out to the fish. -->
116 <keyframe>
117     <key name="starting_pose" qpos="1.57 1.745 0"/>
118 </keyframe>
119
120 </mujoco>

```

Package: diagnostics

software/src/diagnostics/config/diagnostics_params.yaml (34 lines)

```
1 # Recorder configuration.
2 # output_dir -> BASE dir; each run writes a <timestamp>/ subfolder inside it.
3 # Lives in the repo so recordings are versioned with the project.
4 # mjcf_path -> '' auto-resolves to description share / urdf / fishing-robot_sim.xml
5 # render_script-> '' auto-resolves to diagnostics share / scripts / render_animation.py
6 recorder:
7   ros_parameters:
8     output_dir: "/home/nzge/fishing-bot/software/recordings"
9     mjcf_path: ""
10    render_script: ""
11    # Python that can import mujoco (the project virtualenv) for the renderer.
12    venv_python: "/home/nzge/fishing-bot/.venv/bin/python"
13    auto_render: true
14    video_format: "mp4"      # 'mp4' (needs imageio-ffmpeg) or 'gif'
15    video_fps: 30
16    qpos_joints: ["Joint_1", "Joint_2", "fish_swim"]
17    arm_joints: ["Joint_1", "Joint_2"]
18
19 # Bring-up self-test (run with hardware_check:=true). Small, slow, open-loop
20 # moves + a tension sampling window; tune the safety margins for your rig.
21 hardware_check:
22   ros_parameters:
23     # Hardware is 1-DOF pitch (Joint_2 / Dynamixel ID 1); base not on the bus.
24     joints: ["Joint_2"]
25     move_delta: 0.2          # rad, per-joint test move
26     move_time: 2.0           # s, trajectory duration (slow = gentle)
27     settle_time: 1.0        # s, extra time to arrive
28     position_tolerance: 0.08 # rad, pass band on reaching the target
29     startup_timeout: 15.0    # s, wait for controllers/hardware
30     tension_check_duration: 3.0 # s, sensor sampling window
31     tension_min: -50.0
32     tension_max: 50.0
33     require_tension_variation: false
```

software/src/diagnostics/diagnostics/__init__.py (1 lines)

```
1
```

software/src/diagnostics/diagnostics/hardware_check.py (189 lines)

```
1 #!/usr/bin/env python3
2 """Hardware bring-up self-test (open-loop, no closed-loop control).
3
4 Verifies that the ROS 2 pipeline talks to the physical layer:
5
6 * MOTOR - command each joint a small, slow, absolute move via the trajectory
7           controller and confirm /joint_states reports it reaching the target.
8 * SENSOR - sample /fishing_arm/tension and confirm it is publishing finite
9           values within a sane range.
10
11 It deliberately does NOT run the admittance controller (the bringup launch
12 disables it when hardware_check:=true), so these are pure function checks, not
13 control behaviour. The exact same test runs in simulation, so you can validate
14 it with use_sim:=true before trusting it on real hardware.
15
16 Prints a PASS/FAIL summary and exits 0 (all passed) or 1 (something failed).
17 """
18 import math
19 import sys
20 import threading
21 import time
22
23 import rclpy
24 from rclpy.executors import SingleThreadedExecutor
25 from rclpy.node import Node
26 from sensor_msgs.msg import JointState
27 from trajectory_msgs.msg import JointTrajectory, JointTrajectoryPoint
28 from interfaces.msg import FishingTension
29
30
31 class HardwareCheck(Node):
32     def __init__(self):
```

```

33     super().__init__('hardware_check')
34
35     self.declare_parameter('joints', ['Joint_1', 'Joint_2'])
36     self.declare_parameter('move_delta', 0.2)           # rad, small & safe
37     self.declare_parameter('move_time', 2.0)           # s, slow command
38     self.declare_parameter('settle_time', 1.0)         # s, extra to arrive
39     self.declare_parameter('position_tolerance', 0.08)  # rad
40     self.declare_parameter('startup_timeout', 15.0)    # s, wait for states
41     self.declare_parameter('tension_check_duration', 3.0)
42     self.declare_parameter('tension_min', -50.0)
43     self.declare_parameter('tension_max', 50.0)
44     self.declare_parameter('require_tension_variation', False)
45     self.declare_parameter('traj_topic', '/position_trajectory_controller/joint_trajectory')
46
47     self.joints = list(self.get_parameter('joints').value)
48     self.move_delta = self.get_parameter('move_delta').value
49     self.move_time = self.get_parameter('move_time').value
50     self.settle_time = self.get_parameter('settle_time').value
51     self.position_tolerance = self.get_parameter('position_tolerance').value
52     self.startup_timeout = self.get_parameter('startup_timeout').value
53     self.tension_check_duration = self.get_parameter('tension_check_duration').value
54     self.tension_min = self.get_parameter('tension_min').value
55     self.tension_max = self.get_parameter('tension_max').value
56     self.require_tension_variation = bool(
57         self.get_parameter('require_tension_variation').value)
58     self.traj_topic = self.get_parameter('traj_topic').value
59
60     self.joint_pos = {}
61     self.tension = []
62     self._collect_tension = False
63
64     self.create_subscription(JointState, '/joint_states', self._js_cb, 50)
65     self.create_subscription(FishingTension, '/fishing_arm/tension', self._ten_cb, 50)
66     self.pub = self.create_publisher(JointTrajectory, self.traj_topic, 10)
67
68     def _js_cb(self, msg):
69         for name, pos in zip(msg.name, msg.position):
70             self.joint_pos[name] = pos
71
72     def _ten_cb(self, msg):
73         if self._collect_tension:
74             self.tension.append(msg.tension_newtons)
75
76     def _send(self, positions):
77         msg = JointTrajectory()
78         msg.joint_names = list(positions.keys())
79         point = JointTrajectoryPoint()
80         point.positions = [float(positions[j]) for j in msg.joint_names]
81         point.time_from_start.sec = int(self.move_time)
82         point.time_from_start.nanosec = int((self.move_time % 1.0) * 1e9)
83         msg.points.append(point)
84         self.pub.publish(msg)
85
86     def _wait_for(self, condition, timeout):
87         deadline = time.time() + timeout
88         while time.time() < deadline:
89             if condition():
90                 return True
91             time.sleep(0.05)
92         return False
93
94     def run_checks(self):
95         results = []
96
97         if not self._wait_for(lambda: all(j in self.joint_pos for j in self.joints),
98                                self.startup_timeout):
99             missing = [j for j in self.joints if j not in self.joint_pos]
100             print(f'[hw-check] FAIL: no /joint_states for {missing} within '
101                  f'{self.startup_timeout:.0f}s. Are the controllers + hardware up?')
102             return False
103
104         # Wait for the trajectory controller to actually subscribe, otherwise the
105         # first command is published into the void (pub/sub discovery race) and the
106         # first joint never moves.
107         if not self._wait_for(lambda: self.pub.get_subscription_count() > 0,
108                                self.startup_timeout):
109             print(f'[hw-check] FAIL: no subscriber on {self.traj_topic} - is '
110                  f'position_trajectory_controller active?')
111             return False
112         time.sleep(0.5)
113
114         start = {j: self.joint_pos[j] for j in self.joints}
115         print(f'[hw-check] Initial positions: '
116              f'{ {j: round(start[j], 3) for j in self.joints} }')
117

```

```

118 # ---- MOTOR checks: move each joint, confirm it tracks, then return ----
119 for j in self.joints:
120     target = dict(start)
121     target[j] = start[j] + self.move_delta
122     print(f'[hw-check] Commanding {j} {self.move_delta:+.3f} rad '
123           f'-> {target[j]:.3f} ...')
124     self._send(target)
125     time.sleep(self.move_time + self.settle_time)
126     err = abs(self.joint_pos[j] - target[j])
127     ok = err < self.position_tolerance
128     results.append((f'motor:{j}', ok,
129                   f'reached err={err:.3f} rad (tol {self.position_tolerance}'))
130     self._send(start)
131     time.sleep(self.move_time + self.settle_time)
132
133 # ---- SENSOR check: confirm tension publishes finite, in-range values ----
134 print(f'[hw-check] Sampling /fishing_arm/tension for '
135       f'{self.tension_check_duration:.1f}s ...')
136 self.tension = []
137 self._collect_tension = True
138 time.sleep(self.tension_check_duration)
139 self._collect_tension = False
140
141 n = len(self.tension)
142 finite = all(math.isfinite(x) for x in self.tension)
143 lo = min(self.tension) if n else float('nan')
144 hi = max(self.tension) if n else float('nan')
145 in_range = n > 0 and lo >= self.tension_min and hi <= self.tension_max
146 varied = (hi - lo) > 1e-6 if n else False
147 sensor_ok = (n > 0 and finite and in_range
148              and (varied or not self.require_tension_variation))
149 detail = f'{n} samples'
150 if n:
151     detail += f', range [{lo:.2f}, {hi:.2f}] N'
152 if not finite:
153     detail += ', NON-FINITE values'
154 results.append(('sensor:tension', sensor_ok, detail))
155
156 # ---- Summary ----
157 print('\n==== HARDWARE CHECK SUMMARY =====')
158 all_ok = True
159 for name, ok, detail in results:
160     all_ok = all_ok and ok
161     print(f' [{"PASS" if ok else "FAIL"}] {name:18s} {detail}')
162 print('==== ' + ('ALL CHECKS PASSED' if all_ok else 'SOME CHECKS FAILED')
163       + ' =====\n')
164 return all_ok
165
166
167 def main(args=None):
168     rclpy.init(args=args)
169     node = HardwareCheck()
170     executor = SingleThreadedExecutor()
171     executor.add_node(node)
172     spin_thread = threading.Thread(target=executor.spin, daemon=True)
173     spin_thread.start()
174
175     passed = False
176     try:
177         passed = node.run_checks()
178     except KeyboardInterrupt:
179         pass
180     finally:
181         node.destroy_node()
182         if rclpy.ok():
183             rclpy.shutdown()
184     sys.exit(0 if passed else 1)
185
186
187 if __name__ == '__main__':
188     main()

```

software/src/diagnostics/diagnostics/recorder.py (223 lines)

```

1 #!/usr/bin/env python3
2 """Recorder: logs the run, plots it, and triggers the MuJoCo animation render.
3
4 Toggled by record=true in the bringup launch. It subscribes to the joint
5 states and the line-tension sensor, buffers the time series, and on shutdown:
6
7 1. Writes a PNG with joint positions, motor effort (torque) and the measured
8   vs target line tension (so you can see if tension is held constant).
9 2. Saves the raw time series + the joint trajectory to an .npz file.

```

```

10 3. Auto-spawns the offscreen animation renderer (render_animation.py) under
11 the .venv python, because MuJoCo lives there, not in the ROS python env.
12
13 Everything here uses only rclpy + numpy + matplotlib (all available to the ROS
14 python), keeping the node light; the heavy MuJoCo rendering is a separate
15 process fed the .npz.
16 """
17 import datetime
18 import os
19 import subprocess
20
21 import numpy as np
22 import rclpy
23 from rclpy.node import Node
24 from sensor_msgs.msg import JointState
25 from interfaces.msg import FishingTension
26
27
28 class Recorder(Node):
29     def __init__(self):
30         super().__init__('recorder')
31
32         # Where artifacts are written. Empty -> a timestamped dir under ~.
33         self.declare_parameter('output_dir', '')
34         # MJCF + render script. Empty -> resolved from the package shares.
35         self.declare_parameter('mjcf_path', '')
36         self.declare_parameter('render_script', '')
37         # The python that can import mujoco (the project virtualenv).
38         self.declare_parameter('venv_python', os.path.expanduser('~/.fishing-bot/.venv/bin/python'))
39         self.declare_parameter('auto_render', True)
40         self.declare_parameter('video_format', 'mp4') # 'mp4' or 'gif'
41         self.declare_parameter('video_fps', 30)
42         # MJCF qpos order (used to rebuild the trajectory for the renderer).
43         self.declare_parameter('qpos_joints', ['Joint_1', 'Joint_2', 'fish_swim'])
44         # Joints whose effort/torque we plot.
45         self.declare_parameter('arm_joints', ['Joint_1', 'Joint_2'])
46
47         self.output_dir = self.get_parameter('output_dir').value
48         self.mjcf_path = self.get_parameter('mjcf_path').value
49         self.render_script = self.get_parameter('render_script').value
50         self.venv_python = self.get_parameter('venv_python').value
51         self.auto_render = bool(self.get_parameter('auto_render').value)
52         self.video_format = self.get_parameter('video_format').value
53         self.video_fps = int(self.get_parameter('video_fps').value)
54         self.qpos_joints = list(self.get_parameter('qpos_joints').value)
55         self.arm_joints = list(self.get_parameter('arm_joints').value)
56
57         self._resolve_paths()
58
59         # Buffers.
60         self.js_t = []
61         self.qpos_rows = [] # one row per JointState msg
62         self.eff = {j: [] for j in self.arm_joints}
63         self.eff_t = []
64         self.ten_t = []
65         self.ten = []
66         self.ten_target = []
67
68         self.create_subscription(JointState, '/joint_states', self._js_cb, 50)
69         self.create_subscription(FishingTension, '/fishing_arm/tension', self._ten_cb, 50)
70
71         self.get_logger().info(
72             f'Recorder active. Artifacts -> {self.output_dir} '
73             f'f'(auto_render={self.auto_render}, format={self.video_format}).')
74
75     def _resolve_paths(self):
76         from ament_index_python.packages import get_package_share_directory
77         # output_dir is a BASE directory; every run gets its own timestamped
78         # subfolder so recordings never overwrite each other.
79         base = self.output_dir or os.path.expanduser('~/.fishing_recordings')
80         stamp = datetime.datetime.now().strftime('%Y%m%d_%H%M%S')
81         self.output_dir = os.path.join(base, stamp)
82         os.makedirs(self.output_dir, exist_ok=True)
83         if not self.mjcf_path:
84             self.mjcf_path = os.path.join(
85                 get_package_share_directory('description'), 'urdf', 'fishing-robot_sim.xml')
86         if not self.render_script:
87             self.render_script = os.path.join(
88                 get_package_share_directory('diagnostics'), 'scripts', 'render_animation.py')
89
90     def _now(self):
91         # Stamp every sample with the recorder's own clock so all topics share
92         # one time base, regardless of each publisher's use_sim_time setting.
93         return self.get_clock().now().nanoseconds * 1e-9
94

```

```

95 def _js_cb(self, msg):
96     name_to_pos = dict(zip(msg.name, msg.position))
97     if not all(j in name_to_pos for j in self.qpos_joints):
98         return
99     now = self._now()
100     self.js_t.append(now)
101     self.qpos_rows.append([name_to_pos[j] for j in self.qpos_joints])
102     if msg.effort and len(msg.effort) == len(msg.name):
103         name_to_eff = dict(zip(msg.name, msg.effort))
104         self.eff_t.append(now)
105         for j in self.arm_joints:
106             self.eff[j].append(name_to_eff.get(j, float('nan'))))
107
108 def _ten_cb(self, msg):
109     self.ten_t.append(self._now())
110     self.ten.append(msg.tension_newtons)
111     self.ten_target.append(msg.target_tension_newtons)
112
113 def save_and_render(self):
114     if not self.js_t and not self.ten_t:
115         print('[recorder] No data captured; nothing to save.')
116         return
117
118     # Normalize all clocks to start at t=0 for readability.
119     t0 = min([t for t in (self.js_t[:1] + self.ten_t[:1])] or [0.0])
120     js_t = np.array(self.js_t) - t0
121     qpos = np.array(self.qpos_rows) if self.qpos_rows else np.empty((0, len(self.qpos_joints)))
122     eff_t = np.array(self.eff_t) - t0 if self.eff_t else np.array([])
123     ten_t = np.array(self.ten_t) - t0 if self.ten_t else np.array([])
124     ten = np.array(self.ten)
125     ten_target = np.array(self.ten_target)
126
127     npz_path = os.path.join(self.output_dir, 'recording.npz')
128     np.savez(
129         npz_path,
130         times=js_t, qpos=qpos, qpos_joints=np.array(self.qpos_joints),
131         eff_t=eff_t, eff=np.array([self.eff[j] for j in self.arm_joints]),
132         arm_joints=np.array(self.arm_joints),
133         ten_t=ten_t, tension=ten, tension_target=ten_target,
134         mjcf_path=self.mjcf_path)
135     print(f'[recorder] Saved data -> {npz_path}')
136
137     self._plot(js_t, qpos, eff_t, ten_t, ten, ten_target)
138     self._spawn_render(npz_path)
139
140 def _plot(self, js_t, qpos, eff_t, ten_t, ten, ten_target):
141     try:
142         import matplotlib
143         matplotlib.use('Agg')
144         import matplotlib.pyplot as plt
145     except ImportError as exc:
146         print(f'[recorder] matplotlib unavailable ({exc}); skipping plots.')
147         return
148
149     fig, axes = plt.subplots(3, 1, figsize=(10, 9), sharex=True)
150
151     ax = axes[0]
152     for i, j in enumerate(self.qpos_joints):
153         if qpos.size:
154             ax.plot(js_t, qpos[:, i], label=j)
155     ax.set_ylabel('position [rad / m]')
156     ax.set_title('Joint positions')
157     ax.legend(loc='upper right')
158     ax.grid(True, alpha=0.3)
159
160     ax = axes[1]
161     for i, j in enumerate(self.arm_joints):
162         if eff_t.size and len(self.eff[j]) == len(eff_t):
163             ax.plot(eff_t, self.eff[j], label=j)
164     ax.set_ylabel('effort [N.m]')
165     ax.set_title('Motor effort (torque)')
166     ax.legend(loc='upper right')
167     ax.grid(True, alpha=0.3)
168
169     ax = axes[2]
170     if ten_t.size:
171         ax.plot(ten_t, ten, label='measured', color='tab:blue')
172         if np.any(np.asarray(ten_target) != 0.0):
173             ax.plot(ten_t, ten_target, label='target', color='tab:red', linestyle='--')
174     ax.set_ylabel('tension [N]')
175     ax.set_xlabel('time [s]')
176     ax.set_title('Line tension (constant-tension tracking)')
177     ax.legend(loc='upper right')
178     ax.grid(True, alpha=0.3)
179

```

```

180     fig.tight_layout()
181     png_path = os.path.join(self.output_dir, 'diagnostics.png')
182     fig.savefig(png_path, dpi=120)
183     plt.close(fig)
184     print(f'[recorder] Saved plots -> {png_path}')
185
186     def _spawn_render(self, npz_path):
187         video_path = os.path.join(self.output_dir, f'animation.{self.video_format}')
188         if not self.auto_render:
189             print('[recorder] auto_render disabled. Render manually with:')
190             print(f' {self.venv_python} {self.render_script} {npz_path} '
191                   f'--output {video_path} --fps {self.video_fps}')
192             return
193         if not os.path.exists(self.venv_python):
194             print(f'[recorder] venv python not found at {self.venv_python}; '
195                   f'cannot auto-render. Run manually:')
196             print(f' <python-with-mujoco> {self.render_script} {npz_path} '
197                   f'--output {video_path} --fps {self.video_fps}')
198             return
199         cmd = [self.venv_python, self.render_script, npz_path,
200               '--output', video_path, '--fps', str(self.video_fps)]
201         # Detach into its own session so the launch teardown (process-group
202         # SIGINT/SIGKILL) does not kill the render before it finishes.
203         subprocess.Popen(cmd, start_new_session=True)
204         print(f'[recorder] Rendering animation in background -> {video_path}')
205
206
207     def main(args=None):
208         rclpy.init(args=args)
209         node = Recorder()
210         try:
211             rclpy.spin(node)
212         except (KeyboardInterrupt, rclpy.executors.ExternalShutdownException):
213             pass
214         finally:
215             node.save_and_render()
216             node.destroy_node()
217             if rclpy.ok():
218                 rclpy.shutdown()
219
220
221 if __name__ == '__main__':
222     main()

```

software/src/diagnostics/launch/diagnostics.launch.py (23 lines)

```

1 from launch import LaunchDescription
2 from launch.substitutions import PathJoinSubstitution
3 from launch_ros.actions import Node
4 from launch_ros.substitutions import FindPackageShare
5
6
7 def generate_launch_description():
8     # Recorder settings (paths, video format/fps, venv python) live in config.
9     recorder_params = PathJoinSubstitution(
10         [FindPackageShare('diagnostics'), 'config', 'diagnostics_params.yaml'])
11
12     recorder_node = Node(
13         package='diagnostics',
14         executable='recorder',
15         name='recorder',
16         output='screen',
17         parameters=[recorder_params],
18     )
19
20     return LaunchDescription([
21         recorder_node,
22     ])

```

software/src/diagnostics/package.xml (27 lines)

```

1 <?xml version="1.0"?>
2 <?xml-model href="http://download.ros.org/schema/package_format3.xsd" schematypens="http://www.w3.org/2001/XMLSchema"?>
3 <package format="3">
4   <name>diagnostics</name>
5   <version>0.0.1</version>
6   <description>Recording + visualization tools: time-series plots and an
7     isometric MuJoCo animation, toggled with record:=true.</description>
8   <maintainer email="nathange784@gmail.com">nzge</maintainer>
9   <license>Apache-2.0</license>

```

```

10
11 <exec_depend>rclpy</exec_depend>
12 <exec_depend>sensor_msgs</exec_depend>
13 <exec_depend>trajectory_msgs</exec_depend>
14 <exec_depend>interfaces</exec_depend>
15 <exec_depend>python3-numpy</exec_depend>
16 <exec_depend>python3-matplotlib</exec_depend>
17
18 <test_depend>ament_copyright</test_depend>
19 <test_depend>ament_flake8</test_depend>
20 <test_depend>ament_pep257</test_depend>
21 <test_depend>python3-pytest</test_depend>
22
23 <export>
24   <build_type>ament_python</build_type>
25 </export>
26 </package>

```

software/src/diagnostics/scripts/motor_test.py (96 lines)

```

1  # -*- coding: utf-8 -*-
2  """1D0F-FK&IK.ipynb
3
4  Automatically generated by Colab.
5
6  Original file is located at
7  https://colab.research.google.com/drive/1mn1KUgn-dta6Zbn3yhciQf_AI59JnrKn
8  """
9
10 import numpy as np
11
12 # Coordinate frame definition:
13 #   +x axis: forward direction, pointing from the base toward the rod/fish
14 #   +y axis: sideways direction, perpendicular to the side-view plane
15 #   +z axis: vertical upward direction
16 #
17 # Rod pitch angle q:
18 #   q = 0 deg: rod points horizontally along the +x direction
19 #   q > 0 deg: rod pitches upward toward +z
20 #   q < 0 deg: rod pitches downward toward -z
21 #
22 # Since the current mechanism only has rod pitch and no base yaw,
23 # the rod tip motion is constrained to the x-z plane, so y remains 0.
24 #
25 # Current placeholder parameters:
26 #   rod_length = 0.3 m
27 #   q_min = -20 deg
28 #   q_max = 80 deg
29 class FishingRodKinematics:
30     def __init__(self, rod_length=0.3, q_min=np.deg2rad(-20), q_max=np.deg2rad(80)):
31         self.L = rod_length
32         self.q_min = q_min
33         self.q_max = q_max
34
35     def rot_y(self, q):
36         c = np.cos(q)
37         s = np.sin(q)
38
39         return np.array([
40             [ c, 0, -s, 0],
41             [ 0, 1,  0, 0],
42             [ s, 0,  c, 0],
43             [ 0, 0,  0, 1]
44         ])
45
46     def trans_x(self, L):
47         return np.array([
48             [1, 0, 0, L],
49             [0, 1, 0, 0],
50             [0, 0, 1, 0],
51             [0, 0, 0, 1]
52         ])
53
54     def forward_kinematics(self, q):
55         q = np.clip(q, self.q_min, self.q_max)
56
57         T = self.rot_y(q) @ self.trans_x(self.L)
58         tip_position = T[:3, 3]
59
60         return T, tip_position
61
62     def inverse_kinematics(self, target):
63         x, y, z = target

```



```

64         q = np.arctan2(z, x)
65         q = np.clip(q, self.q_min, self.q_max)
66
67         T, tip_position = self.forward_kinematics(q)
68
69         return q, T, tip_position
70
71
72
73 if __name__ == "__main__":
74     rod = FishingRodKinematics(
75         rod_length=0.3,
76         q_min=np.deg2rad(-20),
77         q_max=np.deg2rad(80)
78     )
79
80     q = np.deg2rad(25)
81     T, tip = rod.forward_kinematics(q)
82
83     print("Transformation matrix:")
84     print(T)
85
86     print("Rod tip position:")
87     print(tip)
88
89     target = np.array([0.24, 0.0, 0.12])
90     q_sol, T_sol, tip_sol = rod.inverse_kinematics(target)
91
92     print("IK angle in degrees:")
93     print(np.rad2deg(q_sol))
94
95     print("Reachable tip position:")
96     print(tip_sol)

```

software/src/diagnostics/scripts/render_animation.py (124 lines)

```

1  #!/usr/bin/env python3
2  """Offscreen MuJoCo animation renderer (runs under the .venv python).
3
4  Reads a recording.npz produced by the recorder node, replays the joint
5  trajectory through the MJCF, and writes an isometric video auto-framed so the
6  whole arm + line + fish stay in view while being zoomed in as far as possible.
7
8  This is intentionally NOT a ROS node: it needs `mujoco` (which lives in the
9  project virtualenv, not the ROS python). The recorder spawns it for you, or run
10 it manually:
11
12     ~/fishing-bot/.venv/bin/python render_animation.py recording.npz \
13     --output animation.mp4 --fps 30
14
15 Requires `imageio` (and `imageio-ffmpeg` for mp4) in that same environment:
16
17     ~/fishing-bot/.venv/bin/pip install imageio imageio-ffmpeg
18 """
19 import argparse
20 import math
21 import os
22 import sys
23
24 # Pick a GL backend before importing mujoco: EGL when headless, GLFW otherwise.
25 if 'MUJOCO_GL' not in os.environ:
26     os.environ['MUJOCO_GL'] = 'glfw' if os.environ.get('DISPLAY') else 'egl'
27
28 import numpy as np # noqa: E402
29
30
31 def _compute_framing(model, data, mujoco, qpos, indices):
32     """Bounding sphere of every geom except the ground, over all frames."""
33     ground_id = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_GEOM, 'ground')
34     lo = np.full(3, np.inf)
35     hi = np.full(3, -np.inf)
36     for i in indices:
37         data.qpos[:] = qpos[i]
38         mujoco.mj_forward(model, data)
39         for g in range(model.ngeom):
40             if g == ground_id:
41                 continue
42             p = data.geom_xpos[g]
43             r = float(model.geom_rbound[g])
44             lo = np.minimum(lo, p - r)
45             hi = np.maximum(hi, p + r)
46     center = (lo + hi) / 2.0
47     radius = float(np.linalg.norm(hi - center))

```

```

48     return center, max(radius, 1e-3)
49
50
51 def main():
52     parser = argparse.ArgumentParser(description=__doc__)
53     parser.add_argument('npz', help='recording.npz from the recorder node')
54     parser.add_argument('--output', default='animation.mp4')
55     parser.add_argument('--mjcf', default='', help='override MJCF path')
56     parser.add_argument('--fps', type=int, default=30)
57     parser.add_argument('--width', type=int, default=1280)
58     parser.add_argument('--height', type=int, default=720)
59     parser.add_argument('--azimuth', type=float, default=135.0)
60     parser.add_argument('--elevation', type=float, default=-20.0)
61     parser.add_argument('--margin', type=float, default=1.15,
62                         help='>1 leaves padding around the scene')
63     args = parser.parse_args()
64
65     try:
66         import mujoco
67         import imageio
68     except ImportError as exc:
69         sys.exit(f'[render] Missing dependency: {exc}. '
70                 f'Install with: pip install mujoco imageio imageio-ffmpeg')
71
72     rec = np.load(args.npz, allow_pickle=True)
73     qpos = rec['qpos']
74     times = rec['times']
75     mjcf_path = args.mjcf or str(rec['mjcf_path'])
76     if qpos.size == 0:
77         sys.exit('[render] recording has no qpos frames.')
78
79     model = mujoco.MjModel.from_xml_path(mjcf_path)
80     if qpos.shape[1] != model.nq:
81         sys.exit(f'[render] qpos width {qpos.shape[1]} != model.nq {model.nq}')
82     data = mujoco.MjData(model)
83
84     # Subsample the recorded frames down to the target playback fps.
85     n = len(times)
86     duration = float(times[-1] - times[0]) if n > 1 else 1.0
87     src_rate = n / duration if duration > 0 else float(args.fps)
88     stride = max(1, int(round(src_rate / args.fps)))
89     indices = list(range(0, n, stride))
90
91     center, radius = _compute_framing(model, data, mujoco, qpos, indices)
92     fovy = float(model.vis.global_.fovy)
93     distance = radius / math.sin(math.radians(fovy / 2.0)) * args.margin
94
95     cam = mujoco.MjvCamera()
96     cam.type = mujoco.mjtCamera.mjCAMERA_FREE
97     cam.lookat[:] = center
98     cam.distance = distance
99     cam.azimuth = args.azimuth
100    cam.elevation = args.elevation
101
102    renderer = mujoco.Renderer(model, height=args.height, width=args.width)
103    frames = []
104    for i in indices:
105        data.qpos[:] = qpos[i]
106        mujoco.mj_forward(model, data)
107        renderer.update_scene(data, camera=cam)
108        frames.append(renderer.render())
109    renderer.close()
110
111    out = args.output
112    try:
113        imageio.mimsave(out, frames, fps=args.fps)
114    except Exception as exc: # ffmpeg missing for mp4, etc. -> fall back to gif
115        gif = os.path.splitext(out)[0] + '.gif'
116        print(f'[render] {out} failed ({exc}); writing {gif} instead.')
117        imageio.mimsave(gif, frames, fps=args.fps, loop=0)
118        out = gif
119    print(f'[render] Wrote {out} ({len(frames)} frames @ {args.fps} fps).')
120
121
122 if __name__ == '__main__':
123     main()

```

software/src/diagnostics/scripts/sensor_test.py (96 lines)

```

1  # -*- coding: utf-8 -*-
2  """1D0F-FK&IK.ipynb
3
4  Automatically generated by Colab.

```

```

5
6 Original file is located at
7     https://colab.research.google.com/drive/1mn1KUgn-dta6Zbn3yhciQf_AI59JnrKn
8     ""
9
10 import numpy as np
11
12 # Coordinate frame definition:
13 #   +x axis: forward direction, pointing from the base toward the rod/fish
14 #   +y axis: sideways direction, perpendicular to the side-view plane
15 #   +z axis: vertical upward direction
16 #
17 # Rod pitch angle q:
18 #   q = 0 deg: rod points horizontally along the +x direction
19 #   q > 0 deg: rod pitches upward toward +z
20 #   q < 0 deg: rod pitches downward toward -z
21 #
22 # Since the current mechanism only has rod pitch and no base yaw,
23 # the rod tip motion is constrained to the x-z plane, so y remains 0.
24 #
25 # Current placeholder parameters:
26 #   rod_length = 0.3 m
27 #   q_min = -20 deg
28 #   q_max = 80 deg
29 class FishingRodKinematics:
30     def __init__(self, rod_length=0.3, q_min=np.deg2rad(-20), q_max=np.deg2rad(80)):
31         self.L = rod_length
32         self.q_min = q_min
33         self.q_max = q_max
34
35     def rot_y(self, q):
36         c = np.cos(q)
37         s = np.sin(q)
38
39         return np.array([
40             [ c, 0, -s, 0],
41             [ 0, 1,  0, 0],
42             [ s, 0,  c, 0],
43             [ 0, 0,  0, 1]
44         ])
45
46     def trans_x(self, L):
47         return np.array([
48             [1, 0, 0, L],
49             [0, 1, 0, 0],
50             [0, 0, 1, 0],
51             [0, 0, 0, 1]
52         ])
53
54     def forward_kinematics(self, q):
55         q = np.clip(q, self.q_min, self.q_max)
56
57         T = self.rot_y(q) @ self.trans_x(self.L)
58         tip_position = T[:3, 3]
59
60         return T, tip_position
61
62     def inverse_kinematics(self, target):
63         x, y, z = target
64
65         q = np.arctan2(z, x)
66         q = np.clip(q, self.q_min, self.q_max)
67
68         T, tip_position = self.forward_kinematics(q)
69
70         return q, T, tip_position
71
72
73 if __name__ == "__main__":
74     rod = FishingRodKinematics(
75         rod_length=0.3,
76         q_min=np.deg2rad(-20),
77         q_max=np.deg2rad(80)
78     )
79
80     q = np.deg2rad(25)
81     T, tip = rod.forward_kinematics(q)
82
83     print("Transformation matrix:")
84     print(T)
85
86     print("Rod tip position:")
87     print(tip)
88
89     target = np.array([0.24, 0.0, 0.12])

```

```

90     q_sol, T_sol, tip_sol = rod.inverse_kinematics(target)
91
92     print("IK angle in degrees:")
93     print(np.rad2deg(q_sol))
94
95     print("Reachable tip position:")
96     print(tip_sol)

```

software/src/diagnostics/scripts/test_joint_state.py (1 lines)

```

1

```

software/src/diagnostics/setup.cfg (5 lines)

```

1 [develop]
2 script_dir=$base/lib/diagnostics
3 [install]
4 install_scripts=$base/lib/diagnostics

```

software/src/diagnostics/setup.py (35 lines)

```

1 import os
2 from glob import glob
3 from setuptools import find_packages, setup
4
5 package_name = 'diagnostics'
6
7 setup(
8     name=package_name,
9     version='0.0.1',
10    packages=find_packages(exclude=['test']),
11    data_files=[
12        ('share/ament_index/resource_index/packages',
13         ['resource/' + package_name]),
14        ('share/' + package_name, ['package.xml']),
15        (os.path.join('share', package_name, 'launch'), glob('launch/*.launch.py')),
16        (os.path.join('share', package_name, 'config'), glob('config/*.yaml')),
17        # The render script runs under the .venv python (it needs mujoco), so it
18        # is shipped as a data file rather than a console_scripts entry point.
19        (os.path.join('share', package_name, 'scripts'), glob('scripts/*.py')),
20    ],
21    install_requires=['setuptools', 'rclpy', 'sensor_msgs', 'trajectory_msgs', 'interfaces'],
22    zip_safe=True,
23    maintainer='nzge',
24    maintainer_email='nathange784@gmail.com',
25    description='Recording + visualization tools for the robotic fishing arm',
26    license='Apache-2.0',
27    tests_require=['pytest'],
28    entry_points={
29        'console_scripts': [
30            'recorder = diagnostics.recorder:main',
31            'hardware_check = diagnostics.hardware_check:main',
32        ],
33    },
34 )

```

Package: interfaces

software/src/interfaces/CMakeLists.txt (33 lines)

```
1 cmake_minimum_required(VERSION 3.8)
2 project(interfaces)
3
4 if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
5   add_compile_options(-Wall -Wextra -Wpedantic)
6 endif()
7
8 # find dependencies
9 find_package(ament_cmake REQUIRED)
10 find_package(rosidl_default_generators REQUIRED)
11 find_package(std_msgs REQUIRED)
12
13 rosidl_generate_interfaces(${PROJECT_NAME}
14   "msg/FishingTension.msg"
15   DEPENDENCIES std_msgs
16 )
17
18 if(BUILD_TESTING)
19   find_package(ament_lint_auto REQUIRED)
20   # the following line skips the linter which checks for copyrights
21   # comment the line when a copyright and license is added to all source files
22   set(ament_cmake_copyright_FOUND TRUE)
23   # the following line skips cpplint (only works in a git repo)
24   # comment the line when this package is in a git repo and when
25   # a copyright and license is added to all source files
26   set(ament_cmake_cpplint_FOUND TRUE)
27   ament_lint_auto_find_test_dependencies()
28 endif()
29
30 ament_export_dependencies(rosidl_default_runtime)
31
32 ament_package()
```

software/src/interfaces/msg/FishingTension.msg (3 lines)

```
1 std_msgs/Header header
2 float32 tension_newtons
3 float32 target_tension_newtons
```

software/src/interfaces/package.xml (25 lines)

```
1 <?xml version="1.0"?>
2 <?xml-model href="http://download.ros.org/schema/package_format3.xsd" schematypens="http://www.w3.org/2001/XMLSchema"?>
3 <package format="3">
4   <name>interfaces</name>
5   <version>0.0.0</version>
6   <description>Custom ROS 2 message definitions shared across the fishing arm stack (e.g. FishingTension).</description>
7   <maintainer email="nathange784@gmail.com">nzge</maintainer>
8   <license>MIT</license>
9
10   <buildtool_depend>ament_cmake</buildtool_depend>
11   <buildtool_depend>rosidl_default_generators</buildtool_depend>
12
13   <depend>std_msgs</depend>
14
15   <exec_depend>rosidl_default_runtime</exec_depend>
16   <member_of_group>rosidl_interface_packages</member_of_group>
17
18   <test_depend>ament_lint_auto</test_depend>
19   <test_depend>ament_lint_common</test_depend>
20
21   <export>
22     <build_type>ament_cmake</build_type>
23   </export>
24 </package>
```

Package: planning

software/src/planning/config/fish_params.yaml (10 lines)

```
1 # Disturbance profile for the virtual "fish" (simulation only).
2 # The fish applies this force to its body in MuJoCo; the line transmits it to
3 # the rod tip where the load cell measures the resulting tension.
4 fish_agent:
5   ros_parameters:
6     mean_force: 3.5      # N, steady pull – centred on controller target
7     amplitude: 1.2       # N, sinusoidal "fight" about the mean
8     frequency: 0.5       # Hz, how fast the fish thrashes
9     publish_frequency: 50.0 # Hz, command update rate
```

software/src/planning/launch/fish.launch.py (23 lines)

```
1 from launch import LaunchDescription
2 from launch.substitutions import PathJoinSubstitution
3 from launch_ros.actions import Node
4 from launch_ros.substitutions import FindPackageShare
5
6
7 def generate_launch_description():
8     # Disturbance profile lives in config (No Magic Numbers).
9     fish_params = PathJoinSubstitution(
10         [FindPackageShare('planning'), 'config', 'fish_params.yaml'])
11
12     fish_agent_node = Node(
13         package='planning',
14         executable='fish_agent',
15         name='fish_agent',
16         output='screen',
17         parameters=[fish_params],
18     )
19
20     return LaunchDescription([
21         fish_agent_node,
22     ])
```

software/src/planning/package.xml (22 lines)

```
1 <?xml version="1.0"?>
2 <?xml-model href="http://download.ros.org/schema/package_format3.xsd" schematypens="http://www.w3.org/2001/XMLSchema"?>
3 <package format="3">
4   <name>planning</name>
5   <version>0.0.0</version>
6   <description>Disturbance and target generator (the virtual fish) for simulation runs.</description>
7   <maintainer email="nathange784@gmail.com">nzge</maintainer>
8   <license>MIT</license>
9
10  <exec_depend>rclpy</exec_depend>
11  <exec_depend>std_msgs</exec_depend>
12
13  <test_depend>ament_copyright</test_depend>
14  <test_depend>ament_flake8</test_depend>
15  <test_depend>ament_pep257</test_depend>
16  <test_depend>python3-pytest</test_depend>
17
18  <export>
19    <build_type>ament_python</build_type>
20  </export>
21 </package>
```

software/src/planning/planning/__init__.py (1 lines)

```
1
```

software/src/planning/planning/fish_agent.py (70 lines)

```
1 #!/usr/bin/env python3
2 """The 'Fish': an environmental disturbance source.
3
```

```

4 The fish is attached to the far end of the fishing line; the near end is tied
5 to the rod tip. The fish does NOT talk to the controller. Instead it generates
6 a pulling force that is applied to the fish body in the MuJoCo simulation. That
7 force tensions the line tendon, and the load cell at the rod tip *measures* the
8 resulting tension. The controller only ever reacts to that measured tension.
9
10 Output: std_msgs/Float64MultiArray on /fish_effort_controller/commands (effort,
11 in Newtons, applied to the fish's MuJoCo slide joint). ros2_control's
12 forward_command_controller forwards it to the 'fish_force' motor.
13 """
14 import math
15
16 import rclpy
17 from rclpy.node import Node
18 from std_msgs.msg import Float64MultiArray
19
20
21 class FishAgent(Node):
22     def __init__(self):
23         super().__init__('fish_agent')
24
25         # Disturbance profile lives in config (No Magic Numbers).
26         self.declare_parameter('mean_force', 3.0)
27         self.declare_parameter('amplitude', 2.0)
28         self.declare_parameter('frequency', 0.5)
29         self.declare_parameter('publish_frequency', 50.0)
30
31         self.mean_force = self.get_parameter('mean_force').value
32         self.amplitude = self.get_parameter('amplitude').value
33         self.frequency = self.get_parameter('frequency').value
34         self.publish_frequency = self.get_parameter('publish_frequency').value
35
36         self.pub = self.create_publisher(
37             Float64MultiArray, '/fish_effort_controller/commands', 10)
38         self.timer = self.create_timer(1.0 / self.publish_frequency, self.timer_cb)
39         self.start_time = self.get_clock().now()
40
41         self.get_logger().info(
42             f'Virtual Fish Agent generated: mean={self.mean_force} N, '
43             f'amplitude={self.amplitude} N, frequency={self.frequency} Hz.')
44
45     def timer_cb(self):
46         elapsed = (self.get_clock().now() - self.start_time).nanoseconds / 1e9
47         # Sine "fight" superimposed on a steady pull along the line.
48         force = self.mean_force + self.amplitude * math.sin(
49             2.0 * math.pi * self.frequency * elapsed)
50
51         msg = Float64MultiArray()
52         msg.data = [float(force)]
53         self.pub.publish(msg)
54
55
56 def main(args=None):
57     rclpy.init(args=args)
58     node = FishAgent()
59     try:
60         rclpy.spin(node)
61     except KeyboardInterrupt:
62         pass
63     finally:
64         node.destroy_node()
65         rclpy.shutdown()
66
67
68 if __name__ == '__main__':
69     main()

```

software/src/planning/setup.cfg (5 lines)

```

1 [develop]
2 script_dir=$base/lib/planning
3 [install]
4 install_scripts=$base/lib/planning

```

software/src/planning/setup.py (35 lines)

```

1 import os
2 from glob import glob
3 from setuptools import find_packages, setup
4

```

```

5 package_name = 'planning'
6
7 setup(
8     name=package_name,
9     version='0.0.0',
10    packages=find_packages(exclude=['test']),
11    data_files=[
12        ('share/ament_index/resource_index/packages',
13         ['resource/' + package_name]),
14        ('share/' + package_name, ['package.xml']),
15        (os.path.join('share', package_name, 'launch'), glob('launch/*.launch.py')),
16        (os.path.join('share', package_name, 'config'), glob('config/*.yaml')),
17    ],
18    install_requires=['setuptools'],
19    zip_safe=True,
20    maintainer='nzge',
21    maintainer_email='nathange784@gmail.com',
22    description='Disturbance/target generator (the "Fish") for the robotic fishing arm',
23    license='MIT',
24    extras_require={
25        'test': [
26            'pytest',
27        ],
28    },
29    entry_points={
30        'console_scripts': [
31            'fish_agent = planning.fish_agent:main',
32        ],
33    },
34 )

```


Package: sensors

software/src/sensors/config/sensor_params.yaml (37 lines)

```
1 # Configuration for the Fishing Arm Load Cell (HX711 via Arduino serial).
2 load_cell_node:
3   ros_parameters:
4     # --- Arduino serial (matches project_w_t_controller.ipynb) ---
5     arduino_port: "/dev/ttyACM0"
6     arduino_baud: 9600
7     arduino_timeout_s: 0.05
8     tare_on_start: true
9     tare_samples: 20
10    sensor_sign: 1.0
11    sensor_line_filter_window: 2
12
13    # --- Physical calibration ---
14    # Used when tare_on_start=false; overwritten by live tare otherwise.
15    calibration_offset: 0.0
16    # N per raw ADC count. Notebook used 4.8e-5; tune on your rig.
17    calibration_scale: 0.000048
18
19    # --- Signal processing ---
20    use_moving_average: true
21    window_size: 10
22    publish_frequency: 80.0
23
24    # --- Safety / operational limits ---
25    tension_max_threshold: 20.0
26    noise_floor: 0.1
27
28    frame_id: "rod_tip_link"
29    target_tension: 3.5
30
31    # --- Simulation stretch model (must match fishing-robot_sim.xml tendon) ---
32    sim_line_stiffness: 220.0
33    sim_line_sprlength: 0.20
34    sim_line_damping: 1.5
35    sim_rod_frame: "rod_tip_link"
36    sim_fish_frame: "fish_link"
```

software/src/sensors/launch/sensor.launch.py (29 lines)

```
1 from launch import LaunchDescription
2 from launch.actions import DeclareLaunchArgument
3 from launch.substitutions import LaunchConfiguration, PathJoinSubstitution
4 from launch_ros.actions import Node
5 from launch_ros.substitutions import FindPackageShare
6
7
8 def generate_launch_description():
9     # Tension source: 'hardware' (HX711) or 'sim_fts' (MuJoCo force sensor).
10    source = LaunchConfiguration('source')
11    source_arg = DeclareLaunchArgument('source', default_value='hardware')
12
13    # Calibration offsets and limits live in config (No Magic Numbers).
14    sensor_params = PathJoinSubstitution(
15        [FindPackageShare('sensors'), 'config', 'sensor_params.yaml'])
16
17    load_cell_node = Node(
18        package='sensors',
19        executable='load_cell_publisher',
20        name='load_cell_node',
21        output='screen',
22        parameters=[sensor_params, {'source': source}],
23    )
24
25    return LaunchDescription([
26        source_arg,
27        load_cell_node,
28    ])
```

software/src/sensors/package.xml (26 lines)

```
1 <?xml version="1.0"?>
2 <?xml-model href="http://download.ros.org/schema/package_format3.xsd" schematypens="http://www.w3.org/2001/XMLSchema"?>
```

```

3 <package format="3">
4   <name>sensors</name>
5   <version>0.0.0</version>
6   <description>Line-tension sensing: HX711 load cell on hardware, MuJoCo FTS in simulation.</description>
7   <maintainer email="nathange784@gmail.com">nzge</maintainer>
8   <license>MIT</license>
9
10  <exec_depend>rclpy</exec_depend>
11  <exec_depend>std_msgs</exec_depend>
12  <exec_depend>geometry_msgs</exec_depend>
13  <exec_depend>interfaces</exec_depend>
14  <exec_depend>tf2_ros</exec_depend>
15  <exec_depend>python3-serial</exec_depend>
16
17  <test_depend>ament_copyright</test_depend>
18  <test_depend>ament_flake8</test_depend>
19  <test_depend>ament_pep257</test_depend>
20  <test_depend>python3-pytest</test_depend>
21
22  <export>
23    <build_type>ament_python</build_type>
24  </export>
25 </package>

```

software/src/sensors/sensors/__init__.py (6 lines)

```

1 def __init__(self):
2     super().__init__('load_cell_node')
3     # This automatically looks for 'calibration_scale' in the YAML file.
4     # If it's not found, it defaults to 1.0.
5     self.declare_parameter('calibration_scale', 1.0)
6     self.scale = self.get_parameter('calibration_scale').get_parameter_value().double_value

```

software/src/sensors/sensors/arduino_load_cell.py (94 lines)

```

1 """Arduino HX711 serial reader (matches project_w_t_controller.ipynb protocol).
2
3 The Arduino firmware prints lines like:  raw: 12345
4 """
5 from __future__ import annotations
6
7 import re
8 import time
9 from collections import deque
10 from typing import Optional, Tuple
11
12 import serial
13
14 SENSOR_LINE_PATTERN = re.compile(r'raw:\s*(-?\d+)')
15
16
17 def parse_sensor_line(line: str) -> Optional[int]:
18     match = SENSOR_LINE_PATTERN.search(line)
19     if not match:
20         return None
21     return int(match.group(1))
22
23
24 class ArduinoLoadCellReader:
25     """Non-blocking reader for HX711 counts over Arduino serial."""
26
27     def __init__(
28         self,
29         port: str,
30         baud: int = 9600,
31         timeout_s: float = 0.05,
32         filter_window: int = 2,
33         tare_samples: int = 20,
34     ):
35         self.port = port
36         self.baud = baud
37         self.timeout_s = timeout_s
38         self.filter_window = max(1, filter_window)
39         self.tare_samples = max(1, tare_samples)
40         self._ser: Optional[serial.Serial] = None
41         self.buffer: deque[int] = deque(maxlen=self.filter_window)
42         self.zero_offset: Optional[float] = None
43
44     @property
45     def is_open(self) -> bool:

```

```

46         return self._ser is not None and self._ser.is_open
47
48     def open(self) -> None:
49         self._ser = serial.Serial(
50             self.port,
51             self.baud,
52             timeout=self.timeout_s,
53         )
54         # Allow the Arduino to reset after USB enumeration.
55         time.sleep(2.0)
56         self._ser.reset_input_buffer()
57
58     def close(self) -> None:
59         if self._ser is not None:
60             self._ser.close()
61             self._ser = None
62
63     def _readline_raw(self) -> Optional[int]:
64         if not self.is_open:
65             return None
66         line = self._ser.readline().decode('utf-8', errors='ignore').strip()
67         if not line:
68             return None
69         return parse_sensor_line(line)
70
71     def tare(self) -> float:
72         """Average ``tare_samples`` readings with no line load."""
73         samples = []
74         deadline = time.time() + max(5.0, self.tare_samples * self.timeout_s * 4)
75         while len(samples) < self.tare_samples and time.time() < deadline:
76             raw = self._readline_raw()
77             if raw is not None:
78                 samples.append(raw)
79         if not samples:
80             raise RuntimeError(
81                 f'No load-cell samples from {self.port} during tare - '
82                 'is the Arduino running and printing "raw: <count>" lines?',
83             )
84         self.zero_offset = float(sum(samples) / len(samples))
85         return self.zero_offset
86
87     def read_filtered_raw(self) -> Optional[float]:
88         """Return the latest filtered raw ADC count, or None if no new sample."""
89         raw = self._readline_raw()
90         if raw is None:
91             return None
92         self._buffer.append(raw)
93         return float(sum(self._buffer) / len(self._buffer))

```

software/src/sensors/sensors/load_cell_publisher.py (260 lines)

```

1  #!/usr/bin/env python3
2  import math
3  from collections import deque
4  from typing import Optional
5
6  import rclpy
7  import serial
8  from rclpy.node import Node
9  from geometry_msgs.msg import WrenchStamped
10 from interfaces.msg import FishingTension
11 from tf2_ros import Buffer, TransformListener
12
13 from sensors.arduino_load_cell import ArduinoLoadCellReader
14
15
16 class LoadCellPublisher(Node):
17     def __init__(self):
18         # Node name matches the key in sensor_params.yaml so params load correctly.
19         super().__init__('load_cell_node')
20
21         # Defensive Coding: Parameterization (No Magic Numbers).
22         self.declare_parameter('calibration_offset', 0.0)
23         self.declare_parameter('calibration_scale', 1.0)
24         self.declare_parameter('use_moving_average', True)
25         self.declare_parameter('window_size', 10)
26         self.declare_parameter('publish_frequency', 80.0)
27         self.declare_parameter('tension_max_threshold', 20.0)
28         self.declare_parameter('noise_floor', 0.1)
29         self.declare_parameter('frame_id', 'rod_tip_link')
30         # Target line tension (published for diagnostics overlays; the controllers
31         # own the real setpoint, this just annotates the recorded signal).
32         self.declare_parameter('target_tension', 3.5)

```

```

33 # 'hardware' = HX711 via Arduino serial; 'sim_fts' = MuJoCo line-tension estimate.
34 self.declare_parameter('source', 'hardware')
35 self.declare_parameter('wrench_topic', '/tension_sensor_broadcaster/wrench')
36 # Arduino HX711 path (project_w_t_controller.ipynb protocol).
37 self.declare_parameter('arduino_port', '/dev/ttyACM0')
38 self.declare_parameter('arduino_baud', 9600)
39 self.declare_parameter('arduino_timeout_s', 0.05)
40 self.declare_parameter('tare_on_start', True)
41 self.declare_parameter('tare_samples', 20)
42 self.declare_parameter('sensor_sign', 1.0)
43 self.declare_parameter('sensor_line_filter_window', 2)
44 # Sim stretch model (matches fishing-robot_sim.xml tendon parameters).
45 self.declare_parameter('sim_line_stiffness', 220.0)
46 self.declare_parameter('sim_line_sprlength', 0.20)
47 self.declare_parameter('sim_line_damping', 1.5)
48 self.declare_parameter('sim_rod_frame', 'rod_tip_link')
49 self.declare_parameter('sim_fish_frame', 'fish_link')
50
51 self.calibration_offset = self.get_parameter('calibration_offset').value
52 self.calibration_scale = self.get_parameter('calibration_scale').value
53 self.use_moving_average = self.get_parameter('use_moving_average').value
54 self.window_size = int(self.get_parameter('window_size').value)
55 self.publish_frequency = self.get_parameter('publish_frequency').value
56 self.tension_max_threshold = self.get_parameter('tension_max_threshold').value
57 self.noise_floor = self.get_parameter('noise_floor').value
58 self.frame_id = self.get_parameter('frame_id').value
59 self.target_tension = float(self.get_parameter('target_tension').value)
60 self.source = self.get_parameter('source').value
61 self.wrench_topic = self.get_parameter('wrench_topic').value
62 self.arduino_port = self.get_parameter('arduino_port').value
63 self.arduino_baud = int(self.get_parameter('arduino_baud').value)
64 self.arduino_timeout_s = float(self.get_parameter('arduino_timeout_s').value)
65 self.tare_on_start = bool(self.get_parameter('tare_on_start').value)
66 self.tare_samples = int(self.get_parameter('tare_samples').value)
67 self.sensor_sign = float(self.get_parameter('sensor_sign').value)
68 self.sensor_line_filter_window = int(
69     self.get_parameter('sensor_line_filter_window').value)
70 self.sim_line_stiffness = self.get_parameter('sim_line_stiffness').value
71 self.sim_line_sprlength = self.get_parameter('sim_line_sprlength').value
72 self.sim_line_damping = self.get_parameter('sim_line_damping').value
73 self.sim_rod_frame = self.get_parameter('sim_rod_frame').value
74 self.sim_fish_frame = self.get_parameter('sim_fish_frame').value
75
76 self.publisher_ = self.create_publisher(FishingTension, '/fishing_arm/tension', 10)
77 self.buffer = deque(maxlen=self.window_size)
78
79 self._latest_force = None
80 self._prev_line_length = None
81 self._prev_line_time = None
82 self._arduino: Optional[ArduinoLoadCellReader] = None
83 self._last_filtered_raw: Optional[float] = None
84 self._last_tension = 0.0
85 self._hardware_samples = 0
86
87 if self.source == 'sim_fts':
88     self._tf_buffer = Buffer()
89     self._tf_listener = TransformListener(self._tf_buffer, self)
90     self.wrench_sub = self.create_subscription(
91         WrenchStamped, self.wrench_topic, self._wrench_cb, 10)
92 elif self.source == 'hardware':
93     self._init_hardware_reader()
94
95 self.timer_period = 1.0 / self.publish_frequency
96 self.timer = self.create_timer(self.timer_period, self.timer_callback)
97 self.time_counter = 0.0
98
99 self.get_logger().info(
100     f'Load Cell Publisher initialized at {self.publish_frequency:.1f} Hz '
101     f'(source: {self.source}, frame: {self.frame_id}).')
102
103 def _init_hardware_reader(self) -> None:
104     reader = ArduinoLoadCellReader(
105         port=self.arduino_port,
106         baud=self.arduino_baud,
107         timeout_s=self.arduino_timeout_s,
108         filter_window=self.sensor_line_filter_window,
109         tare_samples=self.tare_samples,
110     )
111     try:
112         reader.open()
113     except serial.SerialException as exc:
114         raise RuntimeError(
115             f'Failed to open Arduino load cell on {self.arduino_port}: {exc}',
116         ) from exc
117

```

```

118     zero_offset = self.calibration_offset
119     if self.tare_on_start:
120         self.get_logger().info(
121             'Taring load cell - keep the line unloaded for a moment...')
122         zero_offset = reader.tare()
123         self.get_logger().info(f'Tare complete. zero_offset={zero_offset:.1f}')
124     else:
125         reader.zero_offset = zero_offset
126         self.get_logger().info(
127             f'Using fixed calibration_offset={zero_offset:.1f} (tare_on_start=false)',
128         )
129
130     self._arduino = reader
131     self.calibration_offset = zero_offset
132     self.get_logger().info(
133         f'Arduino HX711 reader active on {self.arduino_port} @ {self.arduino_baud} baud, '
134         f'scale={self.calibration_scale:.6g} N/count.',
135     )
136
137     def destroy_node(self) -> bool:
138         if self._arduino is not None:
139             self._arduino.close()
140             self._arduino = None
141         return super().destroy_node()
142
143     def _wrench_cb(self, msg):
144         f = msg.wrench.force
145         self._latest_force = (f.x, f.y, f.z)
146
147     def _line_length_from_tf(self) -> Optional[float]:
148         try:
149             tf = self._tf_buffer.lookup_transform(
150                 self.sim_rod_frame,
151                 self.sim_fish_frame,
152                 rclpy.time.Time(),
153             )
154         except Exception:
155             return None
156         t = tf.transform.translation
157         return math.sqrt(t.x * t.x + t.y * t.y + t.z * t.z)
158
159     def _sim_stretch_tension(self) -> Optional[float]:
160         """Hooke's-law line model: k*(L-L0) + c*dL/dt (matches MJCF tendon)."""
161         length = self._line_length_from_tf()
162         if length is None:
163             return None
164
165         stretch = max(length - self.sim_line_sprlength, 0.0)
166         tension = self.sim_line_stiffness * stretch
167
168         now = self.get_clock().now()
169         if self._prev_line_length is not None and self._prev_line_time is not None:
170             dt = (now - self._prev_line_time).nanoseconds / 1e9
171             if dt > 1e-6:
172                 rate = (length - self._prev_line_length) / dt
173                 if rate > 0.0:
174                     tension += self.sim_line_damping * rate
175
176         self._prev_line_length = length
177         self._prev_line_time = now
178         return max(tension, 0.0)
179
180     def _raw_to_newtons(self, filtered_raw: float) -> float:
181         return (
182             self.sensor_sign
183             * self.calibration_scale
184             * (filtered_raw - self.calibration_offset)
185         )
186
187     def read_hardware_tension_newtons(self) -> Optional[float]:
188         """Read one Arduino sample; return None if no new line this cycle."""
189         if self._arduino is None:
190             return None
191         filtered_raw = self._arduino.read_filtered_raw()
192         if filtered_raw is None:
193             return None
194         self._last_filtered_raw = filtered_raw
195         self._hardware_samples += 1
196         return self._raw_to_newtons(filtered_raw)
197
198     def read_tension_newtons(self) -> float:
199         if self.source == 'sim_fts':
200             stretch_tension = self._sim_stretch_tension()
201             if stretch_tension is not None:
202                 return stretch_tension

```

```

203         if self._latest_force is None:
204             return 0.0
205         fx, fy, fz = self._latest_force
206         return math.sqrt(fx * fx + fy * fy + fz * fz)
207
208     tension = self.read_hardware_tension_newtons()
209     if tension is not None:
210         self._last_tension = tension
211         return tension
212     # Hold last reading until the next serial line arrives.
213     return self._last_tension
214
215 def timer_callback(self):
216     tension = self.read_tension_newtons()
217
218     if abs(tension) < self.noise_floor:
219         tension = 0.0
220
221     if self.use_moving_average:
222         self.buffer.append(tension)
223         tension = sum(self.buffer) / len(self.buffer)
224
225     msg = FishingTension()
226     msg.header.stamp = self.get_clock().now().to_msg()
227     msg.header.frame_id = self.frame_id
228     msg.tension_newtons = float(tension)
229     msg.target_tension_newtons = self.target_tension
230     self.publisher_.publish(msg)
231
232     if tension > self.tension_max_threshold:
233         self.get_logger().warn(
234             f'Line tension {tension:.2f} N exceeds threshold '
235             f'{self.tension_max_threshold:.2f} N.')
236
237     if int(self.time_counter * self.publish_frequency) % int(self.publish_frequency) == 0:
238         detail = ''
239         if self.source == 'hardware' and self._last_filtered_raw is not None:
240             detail = f' (raw={self._last_filtered_raw:.0f})'
241         self.get_logger().info(f'Publishing line tension: {tension:.2f} N{detail}')
242
243     self.time_counter += self.timer_period
244
245
246 def main(args=None):
247     rclpy.init(args=args)
248     node = LoadCellPublisher()
249     try:
250         rclpy.spin(node)
251     except KeyboardInterrupt:
252         pass
253     finally:
254         node.destroy_node()
255         rclpy.shutdown()
256
257
258 if __name__ == '__main__':
259     main()

```

software/src/sensors/setup.cfg (5 lines)

```

1 [develop]
2 script_dir=$base/lib/sensors
3 [install]
4 install_scripts=$base/lib/sensors

```

software/src/sensors/setup.py (35 lines)

```

1 import os
2 from glob import glob
3 from setuptools import find_packages, setup
4
5 package_name = 'sensors'
6
7 setup(
8     name=package_name,
9     version='0.0.1',
10    packages=find_packages(exclude=['test']),
11    data_files=[
12        # Register the package marker
13        ('share/ament_index/resource_index/packages',

```

```

14         ['resource/' + package_name]),
15         # Include package.xml
16         ('share/' + package_name, ['package.xml']),
17         # Include all launch files (if you create a launch directory)
18         (os.path.join('share', package_name, 'launch'), glob('launch/*.launch.py')),
19         # Include all config files (if you create a config directory)
20         (os.path.join('share', package_name, 'config'), glob('config/*.yaml')),
21     ],
22     install_requires=['setuptools', 'rclpy', 'std_msgs', 'interfaces', 'tf2_ros', 'pyserial'],
23     zip_safe=True,
24     maintainer='nzge',
25     maintainer_email='nathange784@gmail.com',
26     description='Sensor node package for robotic fishing arm',
27     license='Apache-2.0', # Or your preferred license
28     tests_require=['pytest'],
29     entry_points={
30         'console_scripts': [
31             # The format is 'command_name = package_name.script_name:main_function'
32             'load_cell_publisher = sensors.load_cell_publisher:main',
33         ],
34     },
35 )

```